

UNIVERSITY OF PATRAS - SCHOOL OF ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING
AND INFORMATICS



ΠΑΝΕΠΙΣΤΗΜΙΟ
ΠΑΤΡΩΝ
UNIVERSITY OF PATRAS

Diploma Thesis

by the student of the Department of Computer Engineering and Informatics
School of Engineering, University of Patras

Angelos Dimoglis

Student ID: 1059603

Title

**FPGA-Targeted Design and Simulation of a FlexRay Communication
Controller in VHDL**

Supervisor

Associate Professor Nikolaos Sklavos, University of Patras

Diploma Thesis Number:

Patras, September 2022

*English translation of a Diploma Thesis originally written in Greek and completed at the
Department of Computer Engineering and Informatics, University of Patras, in September
2022. The Greek original is the official version.*

Thesis Details

Title: FPGA-Targeted Design and Simulation of a FlexRay Communication Controller in VHDL

Student: Angelos Dimoglis

Supervising Committee
Associate Professor Nikolaos Sklavos, University of Patras

This document is an English translation of the author's Diploma Thesis, originally written in Greek under the title 'Υλοποίηση Communication Controller σε FPGA, με βάση το πρωτόκολλο FlexRay' and completed at the Department of Computer Engineering and Informatics, University of Patras, in September 2022. The translation was prepared in 2026. The title has been adapted and minor errors corrected; the technical content is otherwise unchanged. The Greek original is the official version of record.

Abstract

Nowadays, modern vehicles incorporate more and more features, such as automated parking, pedestrian detection, etc. The FlexRay communication protocol is now one of the widely used protocols in the automotive industry for the operation of the aforementioned features. This is because the protocol offers faster data transmission and higher reliability. At the same time, however, the FlexRay protocol requires higher bandwidth and consumes more energy compared to previously used protocols, resulting in a degradation of the overall system performance. For this reason, it is proposed to extend the capabilities of the communication controllers at each node of the network where the FlexRay protocol operates. In this way, the overall quality and efficiency of the system would be improved. Most off-the-shelf devices do not allow such an extension, but it would be feasible through the use of programmable logic, and more specifically a Field-Programmable Gate Array (FPGA). The purpose of this Diploma Thesis is therefore the design and simulation of a corresponding Communication Controller, using the hardware description language VHDL.

Acknowledgements

First of all, I would like to thank my supervisor, Professor Nikolaos Sklavos, who pointed me towards the right research sources on the subject, gave me valuable advice, and shared his view on every decision I made. I also owe thanks to the faculty of the Department of Computer Engineering & Informatics, who gave me the knowledge I needed to complete my thesis successfully. Finally, I would like to thank my family for the support they provided throughout my studies.

Contents

List of Figures	xiii
List of Tables	xv
1 Introduction	1
1.1 The Problem to Be Solved	1
1.2 Objective of the Thesis	2
1.3 Thesis Outline	2
2 Embedded Systems	3
2.1 Terminology	3
2.2 Applications	4
2.3 Challenges	5
2.4 Common Characteristics	7
3 The FlexRay Protocol	9
3.1 The Origins of the FlexRay Protocol	9
3.1.1 Required Specifications	10
3.2 General Characteristics	12
3.3 Communication Channels and Cycles	15
3.3.1 Communication Channel	15
3.3.2 Communication Cycle	15
3.4 Segments	15
3.4.1 Slots and Minislots	16
3.4.2 Static Segments and Slots	16
3.4.3 Dynamic Segments and Minislots	17
3.4.4 The Symbol Window Segment	17
3.4.5 The Network Idle Time Segment	18
3.5 Communication Frames	18
3.5.1 Frame Components	19
3.6 The Architecture of a Node	20
4 An Approach to Solving the Problem	25
4.1 Improvements to the FlexRay Protocol	25
4.1.1 Topology	25
4.1.2 Scheduling	26
4.1.3 Network-Level Optimisations	26
4.2 The Communication Controller	26
4.2.1 Architecture	26
4.2.2 Implementation and Optimisations	27
4.2.3 Data Path Extensions	30

4.3	Experimental Results	31
4.3.1	Use Case: Fault Detection & Fallback Operation	31
4.4	Conclusions	32
5	Design and Simulation of the Communication Controller	33
5.1	General Architecture	33
5.2	The Data Frame Structure	34
5.3	Protocol Management Module	35
5.3.1	Commands from the Host/Microcontroller	35
5.3.2	The Finite State Machine	35
5.3.3	Testbench	37
5.4	Controller Host Interface	38
5.4.1	Testbench	39
5.5	Medium Interface Layer	39
5.5.1	Output Layer	40
5.5.2	Input Layer	41
5.5.3	Testbench	42
5.6	Wakeup Pattern Generator	44
5.6.1	Testbench	45
5.7	Wakeup Pattern Recognition	45
5.7.1	Testbench	47
5.8	Clock Synchronization	47
5.8.1	Principle of Operation	47
5.8.2	Data Framing	48
5.8.3	Testbench	49
5.9	System Testing	50
5.9.1	Testbench	51
5.10	System Synthesis	54
5.10.1	General Information	54
5.10.2	Synthesis Results	55
6	Conclusions & Future Work	57
6.1	Conclusions	57
6.2	Future Work	57
	Appendices	59
A	Source Code	61
A.1	Protocol Management Module	61
A.2	Controller Host Interface	65
A.3	Medium Interface Layer	66
A.4	Wakeup Pattern Generator	77
A.5	Wakeup Pattern Recognition	81
A.6	Clock Synchronization	84
A.7	Communication Controller (Integration of the Components)	90
B	Notes on the Source Code	97
B.1	The Finite State Machines	97
B.2	Signal Mappings in the System	98
B.3	The Validity of the sync_status Signal	98

C	FPGA Technology	101
C.1	What Is an FPGA?	101
C.2	The Classification of FPGAs	101
C.3	The Structure of an FPGA	102
D	Block Coding	103
D.1	4B/5B Encoding	103
E	The OSI Model	105
E.1	Introduction	105
E.2	Description of the Layers	106
E.2.1	Physical Layer	106
E.2.2	Data Link Layer	106
E.2.3	Network Layer	106
E.2.4	Transport Layer	107
E.2.5	Session Layer	107
E.2.6	Presentation Layer	107
E.2.7	Application Layer	107
	Bibliography	109

List of Figures

2.1	The relationship between embedded systems and Cyber-Physical Systems	3
2.2	Embedded systems in cars Source: Modern Car Technology	4
2.3	Differences between personal computers and embedded systems	8
3.1	Range of problems to be solved	10
3.2	Division of the cycle into a static and a dynamic segment	12
3.3	The dual parallel hierarchy of the FlexRay protocol	13
3.4	The elements of the FlexRay protocol	14
3.5	The structure of the communication cycles	15
3.6	Slots and minislots	16
3.7	End of the Symbol Window	18
3.8	Local clock phase correction	18
3.9	Transmission of communication frames	19
3.10	Components of communication frames	19
3.11	The connections within a FlexRay node	21
3.12	The logical relationships between the components of a FlexRay node	22
3.13	Internal architecture of the Protocol Controller	23
4.1	The FlexRay communication cycle	25
4.2	Active star topology	25
4.3	Switch topology	25
4.4	Architecture of the modified Communication Controller	27
4.5	Operating states of the Communication Controller	28
4.6	Host commands	28
4.7	Architecture of Clock Synchronization	29
4.8	Architecture of the Medium Interface Layer	30
4.9	Data path extensions of the Communication Controller	30
4.10	Latency of critical data processing	32
5.1	General architecture of the Communication Controller	33
5.2	The data frame used	34
5.3	Architecture of the Protocol Management Module	35
5.4	State diagram of the FSM	36
5.5	Testbench of the Next-state logic submodule (FSM), file: pmm/fsm_tb.vhd	37
5.6	Testbench of the Protocol Management Module, file: pmm/pmm_tb.vhd	38
5.7	Architecture of the Controller Host Interface	38
5.8	Testbench of the Controller Host Interface, file: chi/main_tb.vhd	39
5.9	Architecture of the Medium Interface Layer	39
5.10	Frame scheduling circuit	41
5.11	Architecture of the error detection submodule	42
5.12	Testbench for Scheduling (Output Layer), file: mil/scheduler_tb.vhd	42

5.13	Testbench for the Frame ID Generator (Output Layer), file: mil/fid_gen_tb.vhd . . .	43
5.14	Testbench for the MIL (Output Layer), file: mil/mil_main_tb.vhd	43
5.15	Testbench #1 for the MIL (Input Layer), file: mil/mil_main_tb.vhd	43
5.16	Testbench #2 for the MIL (Input Layer), file: mil/mil_main_tb.vhd	44
5.17	Architecture of the Wakeup Pattern Generator	44
5.18	Data frame of the Wakeup Pattern Generator	45
5.19	Testbench of the Wakeup Pattern Generator, file: wupg/wupg_main_tb.vhd	45
5.20	Architecture of the Wakeup Pattern Recognition	46
5.21	State diagram of the FSM in the WUPR	46
5.22	Testbench of the WUPR, file: wup_recognition/wupr_main_tb.vhd	47
5.23	Architecture of the Microtick Counter	47
5.24	Data frame of Clock Synchronization	48
5.25	Finite state machine for the validity bit	48
5.26	Testbench #1 of the Microtick Counter, file: microtick_counter_tb.vhd	49
5.27	Testbench #2 of the Microtick Counter (changed global_value), file: microtick_counter_tb.vhd	49
5.28	Execution flow during system testing	50
5.29	System testbench (1/5)	51
5.30	System testbench (2/5)	51
5.31	System testbench (3/5)	52
5.32	Data frame of the Wakeup Pattern Generator	52
5.33	System testbench (4/5)	53
5.34	System testbench (5/5)	53
5.35	Data frame of Clock Synchronization	54
5.36	Logic synthesis & technology mapping from an RTL description	55
5.37	Power consumption	56
5.38	Dynamic power consumption	56
B.1	Circuit diagram of design style #1	97
B.2	Circuit diagram of design style #2	97
C.1	The position of FPGAs among semiconductor devices	101
C.2	Structure of a traditional FPGA	102
D.1	Block coding	103
E.1	The OSI reference model	105

List of Tables

5.1	State table of the FSM in the PMM	36
5.2	State encoding	37
5.3	Encoding of the Host commands	37
5.4	Truth table of the frame scheduling circuit	40
5.5	State table of the FSM in the WUPR	46
5.6	Resource utilisation by hierarchy	55
5.7	Resource utilisation summary	55
B.1	CHI-PMM signal mapping	98
B.2	CHI-CS signal mapping	98
B.3	PMM-CS signal mapping	98
B.4	CHI-MIL signal mapping	98
B.5	CS-MIL signal mapping	98
B.6	WUPR-MIL signal mapping	98
B.7	WUPG-MIL signal mapping	98
B.8	WUPG-PMM signal mapping	98
D.1	4B/5B encoding: data sequence mappings	104
D.2	4B/5B encoding: control sequence mappings	104

1. Introduction

1.1 The Problem to Be Solved

In recent years, *embedded systems* have grown enormously, thanks to transistor scaling and lower manufacturing costs. An embedded system is defined as a device that combines hardware and software, and possibly other electrical and mechanical parts, in order to carry out one specific function. This is the main point that sets them apart from general-purpose computers, which can be used for any application the end user wants.

Embedded systems owe their success to microprocessors. The first microprocessor was built by Intel in 1971. The order had in fact been for a series of integrated circuits that would perform the functions of a calculator. Intel, however, delivered a single chip, a microprocessor, which incorporated all of the desired functions.[1]

The microprocessor was highly successful over the following decade, since its advantages were not slow to be recognised. Beyond gathering the functions onto a single chip, functional upgrades and maintenance become easier through the use of software. Protection of intellectual property also improves. The hardware may remain vulnerable to attack, but the software can be hidden from malicious end users.[2]

These reasons explain why embedded systems are found in numerous everyday applications, even when we do not notice them directly. Typical examples are the microwave oven, the television, the printer, credit card readers (POS terminals), and medical devices that monitor vital signs.

An embedded system is usually a component of some larger system. Modern vehicles, for example, contain many embedded systems of differing kinds, known as Electronic Control Units (ECUs). Like any ordinary embedded system, each ECU in a car is responsible for carrying out one specific function. Common functions in modern vehicles include engine control, brake control, automated parking, pedestrian detection with automated braking, and so on.

Most ECUs are interdependent, however, so they need to communicate with one another in order to carry out their function successfully. This is achieved through a communication protocol. The protocols in common use are event-triggered ones, such as the Controller Area Network (CAN). The need to integrate more applications, and safety-critical applications in particular, raised the requirements for determinism, predictability and reliability. This led to the adoption of time-triggered protocols such as time-triggered Ethernet and FlexRay.

Attractive as the new protocols are, their adoption comes at a cost. Time-triggered protocols use more bandwidth and consume more energy, which significantly degrades the overall performance of the system.

One solution to this problem would be to extend the capabilities of the Communication Controller at each ECU node, so as to improve the overall quality and efficiency of the system. The Communication Controller would then be able to perform functions that would normally run on the microprocessor of the corresponding node. In this way it takes part of the workload off the processing logic, which removes several bottlenecks. The solution also opens the door to further capabilities that the FlexRay standard did not provide, such as fallback operation and fault tolerance.

The main obstacle to this approach is that most off-the-shelf devices, especially platform-based

ones, are not flexible enough to accept the degree of customisation that extending the Communication Controller requires. Programmable logic therefore becomes necessary, for example through Field-Programmable Gate Array (FPGA) technology.[3]

1.2 Objective of the Thesis

The objective of this thesis is the design and simulation of a Communication Controller that meets the specifications described above. The system targets FPGA technology and was described in the hardware description language VHDL. This Communication Controller conforms to the FlexRay protocol and incorporates additional functions that improve the performance and efficiency of the system. Among these functions are synchronisation with the remaining nodes (ECUs) in the network, error detection, and data encoding and decoding.

1.3 Thesis Outline

Chapter 2: This chapter presents embedded systems. It explains the terminology in use (Cyber-Physical Systems, the IoT, and so on), and covers possible applications, design challenges, and the characteristics that embedded systems have in common.

Chapter 3: This chapter presents the FlexRay protocol, focusing on the reasons it was created, its main characteristics, and the structures it uses. The general architecture of a node within a FlexRay network is also illustrated.

Chapter 4: This chapter examines in more detail the approach to solving the problem presented in Section 1.1, based on the paper [3]. Several improvements to the functionality of the FlexRay protocol are proposed, a possible Communication Controller implementation based on that approach is presented, and tests evaluating the approach follow.

Chapter 5: This chapter is the main subject of the thesis, namely the design and simulation of a Communication Controller based on the specifications presented in Chapter 4. It covers the general architecture of the system and the architectures of the individual submodules, together with a brief description of how they operate. Simulations confirming the correct operation of the submodules, and of the complete system, follow. The chapter closes with the synthesis of the system and a review of FPGA resource utilisation.

Chapter 6: This chapter sets out the conclusions drawn from the thesis, followed by possible future improvements for the continued development of the Communication Controller.

Appendix A: This appendix contains the VHDL code used to describe the submodules and the system, along with the corresponding testbenches used to simulate them.

Appendix B: This appendix records a number of noteworthy remarks about the code presented in Appendix A.

Appendix C: This appendix presents the fundamentals of FPGA technology, since the FPGA is the platform targeted by this thesis.

Appendix D: This appendix describes block coding, and the 4B/5B encoding used in the design of the Communication Controller.

Appendix E: This appendix presents the OSI model, which is referred to in Chapter 3. It explains the important role the model plays in networking, and describes the layers it consists of and how they came about.

2. Embedded Systems

2.1 Terminology

Until the late 1980s, information processing was carried out mainly by mainframe computers and bulky magnetic tape drives. Later, Weiser coined the term ‘Ubiquitous Computing’, meaning the ability to perform computation anywhere and at any time. He also predicted that computers would be built into products, which would allow them to become invisible within the environment.

Thanks to the widespread use of small devices combined with the Internet, many of these visions have already become reality. The use of such devices is now pervasive, in the sense that it plays an important role in many aspects of everyday life. The same could be said of artificial intelligence.

Rapid progress in miniaturisation has made it possible to integrate information processing with the environment through the use of computers. This kind of information processing is referred to as an *embedded system*. According to Marwedel, embedded systems are defined as information processing systems that are embedded into products.

Typical examples of products that contain embedded systems are cars, trains, aircraft and telecommunications equipment. All embedded systems, whatever their application, share a large number of characteristics, such as real-time constraints, dependability, and efficiency requirements.

The *link* to physical systems plays a major role in embedded systems, and it is captured by the following quotation:

‘Embedded software is software integrated with physical processes. The technical problem is managing time and concurrency in computational systems.’

Nevertheless, this link has been emphasised even more in recent years through the term *Cyber-Physical Systems*. Cyber-Physical Systems are defined as systems that integrate computation with physical processes.

Emphasising this link matters, because it is often ignored in the world of applications that run on servers, personal computers and mobile phones. A Cyber-Physical System consists of an embedded system (the information processing part) and a (dynamic) physical environment. The figure below shows this more clearly.

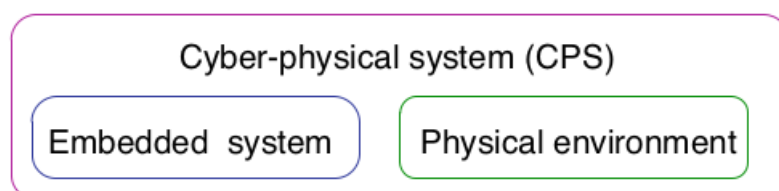


Figure 2.1: The relationship between embedded systems and Cyber-Physical Systems

Interconnection and *cooperation* are two terms that the European Commission has explicitly stressed:

‘Cyber-Physical Systems concern the next generation of embedded information and communication systems, which are interconnected and cooperate through the Internet of Things, offering citizens and businesses a wide range of innovative applications and services.’

It follows from this quotation that, besides the relationship between the cyber world and the physical world, the notion of communication also carries great weight. This notion comes across more clearly in the definition of the *Internet of Things* (IoT).

The Internet of Things describes the pervasive presence of a variety of devices (sensors, actuators, mobile phones and so on) which, through dedicated protocols, are able to interact and cooperate towards common goals.

The Internet of Things is the term that bridges sensors (which capture information for the Internet) and actuators (which are controlled over the Internet).

The development of IoT technology has also led to what is known as *Industry 4.0*. Industry 4.0 aims at more flexible production, in which the product lifecycle is supported by the IoT from the design phase onwards.

2.2 Applications

There is a wealth of information processing applications in which Cyber-Physical Systems and the Internet of Things could be used. Since a complete list would be very long, only some typical examples are given here, grouped by application area.

Transportation and Mobility:

- **Automotive:** Cars sold in developed countries are expected to contain a large number of electronic systems. Among them are the systems for airbag control, engine control, navigation, air conditioning and anti-theft protection. Autonomous driving, which is found more and more often on motorways, is another major achievement. Embedded systems can improve comfort, prevent accidents, and reduce the impact on the environment.

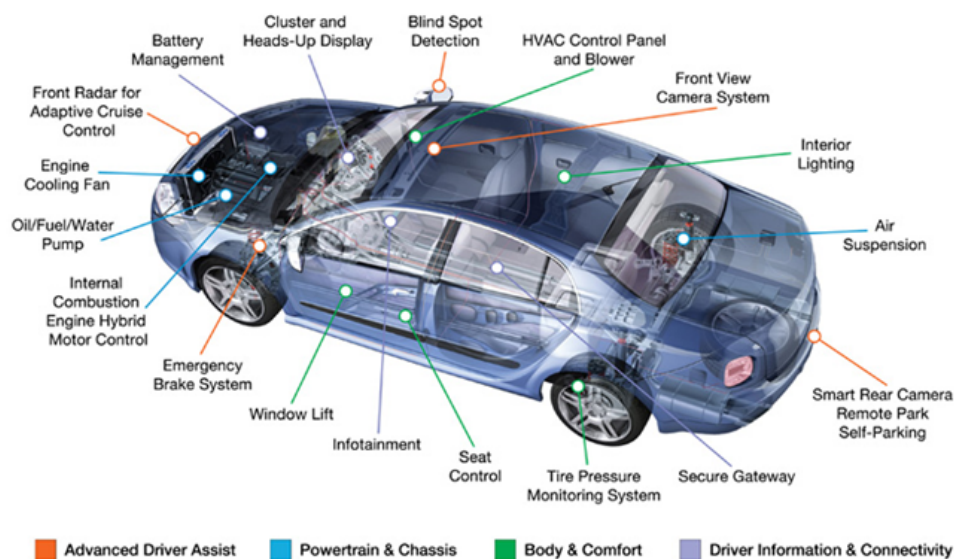


Figure 2.2: Embedded systems in cars

Source: [Modern Car Technology](#)

- **Avionics:** The ever-rising price of aircraft is due to the growing amount of information

processing equipment they carry, such as flight control, collision avoidance and autopilot systems. Here the most important factor is the dependability of the embedded systems.

- **Innovations in mobility:** Information and communication technology makes other ways of getting around possible as well. Electric scooters (e-scooters) are a typical example of a cyber-physical system. They make it possible to travel many kilometres without any particular skill. The Internet of Things and Cyber-Physical Systems are also important ingredients of taxi services.

Mechanical Engineering: Mechanical and manufacturing equipment has been combined with embedded systems for decades. Using Internet of Things technology, Cyber-Physical Systems, or both, allows the production line to be optimised even further. Logistics makes it possible to automate many processes in the factory. Radio-frequency identification (RFID) combined with the Internet, for example, makes it easier to identify products worldwide.

Smart Buildings: Information processing can make buildings more comfortable, reduce their energy consumption, and raise the level of security. There is a trend towards merging the functions of many different devices (air conditioning, lighting, access control and so on) into a single system for centralised control. Possible optimisations include automatic control of lighting and air conditioning, depending on whether anyone is present in a given room. Lists of empty rooms could also be displayed at the building entrance in an emergency. A fair amount of energy could be saved this way. One main goal, then, is the construction of zero-energy buildings, which would help reduce the carbon dioxide in the atmosphere and limit global warming.

Health: Medical devices are becoming ever more important, especially in ageing societies. Advanced sensors combined with machine learning can monitor vital values accurately and predict or detect possible diseases. Treatments can also be improved with artificial intelligence, which makes personalised therapy possible.

Telecommunications: Mobile phones have been one of the fastest-growing markets of recent years. Radio frequency signals, digital signal processing and low power consumption are central research topics. Telecommunications is unquestionably a domain of the Internet of Things.

Consumer Electronics: Audio and video equipment is a major sector of the consumer electronics industry. More and more information processing is being built into such devices, because advanced digital signal processing techniques offer new services and better quality. Mobile phones, televisions and game consoles are special cases of embedded systems. Devices of this kind have to meet real-time requirements and operate adequately with limited resources (such as electrical energy and data rate).

2.3 Challenges

Unfortunately, designing embedded systems and integrating them into Cyber-Physical Systems and the Internet of Things involves a number of challenges. Which challenges arise depends on the kind of application. Common difficulties that may come up when designing an embedded system are listed below.

- Cyber-Physical Systems and Internet of Things systems must be **dependable**. A system is considered dependable if it delivers its expected service with high probability and does not cause harm. The main reason dependability has to be guaranteed is that these systems are directly connected to the environment, so their operation can have an immediate effect on it. Dependability requires the following:

Security: Information security can be defined as preserving the confidentiality, integrity and availability of information. Interconnecting the components of an Internet of Things system opens the way to quite dangerous attacks, such as theft, tampering and destruction. These attacks are a serious concern in the design and development of IoT systems. A secure system requires the following:

- Entity authentication
- Secure access to the Internet
- Secure communication
- Secure storage (confidentiality and integrity of data)
- Content security (access restriction)

Confidentiality: The property that information has not been disclosed to any unauthorised entity or process. Confidentiality is usually ensured through encryption.

Safety: Safety is defined as the absence of unacceptable risk, that is, of harm to people's health. The harm may arise directly, or indirectly as a result of damage to property or to the environment.

Reliability: This term concerns system failure caused by operation that deviates from the specifications set during the design phase. Reliability is the probability that the system does not fail within a given time interval. Malicious attacks are not taken into account when assessing reliability, only events that occur during normal operation of the system.

Repairability: The probability that a failed system can be repaired within a given time interval.

Availability: The probability that the system is available. High availability requires both high reliability and high repairability.

- There is often a **mismatch between models of the physical world and models in cyberspace**. Some examples follow:
 - Many cyber-physical systems must meet **real-time constraints**. Failing to meet them could degrade the quality of service the system provides (audio quality, for instance) or cause harm to the user (for instance, if a vehicle does not operate within its specifications).
 - Many embedded systems are **hybrid**, meaning they involve both analogue and digital signal processing. Unfortunately, analogue quantities cannot be represented exactly. An approximation is therefore used, in the hope of still obtaining the desired result.
 - A great many cyber-physical systems are **control loops**, meaning they operate with feedback. It has been shown, however, that continuous signal sampling is not very efficient and can waste a lot of energy.
 - **Traditional programming languages** are not the best way to describe concurrent, timed systems.
- Another important issue is the **efficient use of resources**. This first requires identifying which resources are needed. The metrics to consider when evaluating how efficiently resources are used are the following:
 - Energy
 - Run time
 - Code size
 - Weight
 - Cost

- There are also several **implications beyond the technical issues** that must be taken into account. These concern society, the law, people, and the environment.
 - Integrating many units from different manufacturers raises the question of liability. Efforts are already under way to settle such questions for self-driving cars.
 - Social issues comprise the effects that information devices have on society. This is why the term *Cyber-Physical-Social Systems* was coined. Unfortunately, such issues tend to be identified long after a new technology has arrived.
 - Human issues concern the design of user-friendly interfaces that follow the principles of Human-Computer Interaction.
 - The contribution to global warming and the amount of waste produced must stay within acceptable limits. The same applies to resource usage.
- Cyber-physical and IoT systems usually consist of heterogeneous hardware and software components from different vendors, which have to work in changing environments. This **heterogeneity**, however, poses several challenges for correct cooperation between the components.

2.4 Common Characteristics

Embedded systems, cyber-physical systems and Internet of Things systems share a number of characteristics, regardless of their application area. Some of them are listed below:

- Cyber-physical and IoT systems use **sensors** and **actuators** to connect embedded systems to the environment. IoT systems are, in addition, connected to the Internet.
- Embedded systems are typically **reactive**. This means that they interact continuously with their environment and act at a pace set by that environment. Automata would be an ideal way to model such systems. Mathematical functions, by contrast, would be an unsuitable model.
- Embedded systems are usually designed to run one specific application. The processors that run control software in a car, for example, will always run the same software, and it cannot be modified. There are two reasons for this:
 1. Running additional programs would make those systems less dependable.
 2. Running additional programs is only possible if there are unused resources (memory, for example). An efficient system should normally have no unused resources.
- Most embedded systems do not use a mouse, keyboard or large screen as their user interface, but a **dedicated user interface** such as buttons or pedals. As a result, the user does not notice the information processing involved. This is consistent with the idea of the **invisible computer**.

The figure below illustrates the differences between personal computers or data centres and embedded systems.

	Embedded	PC-/server-like
Architectures	Frequently heterogeneous very compact	Mostly homogeneous not compact (x86, etc.)
x86 compatibility	Less relevant	Very relevant
Architecture fixed?	Rarely	Yes
Models of computation (MoCs)	C+multiple models (data flow, discrete events, ...)	Mostly von Neumann (C, C++, Java)
Optimization objectives	Multiple (energy, size, ...)	Average performance dominates
Safety-critical?	Possibly	Usually not
Real-time relevant	Frequently	Hardly
Apps. known at design time	Yes, for real-time systems	Only some (e.g., WORD)

Figure 2.3: Differences between personal computers and embedded systems

In practice, no embedded system has all of the characteristics mentioned above. A revised definition of embedded systems could therefore be given:

Information processing systems that have most of the common characteristics listed above are called embedded systems.^[4]

3. The FlexRay Protocol

3.1 The Origins of the FlexRay Protocol

FlexRay grew out of a study carried out by car manufacturers. The study set out to determine whether the communication protocols of the time (CAN, TTCAN, TCN and others) would be able to meet the requirements of future applications over the coming decades. It concluded that the protocols in use would not be able to meet those requirements. The reasons are listed below for each protocol.

- **CAN**
 - Its data rate is too low for new applications.
 - Providing determinism and redundancy in transmission is quite difficult.
- **TTCAN**
 - It has the same throughput limitations as CAN.
 - It lacks support for optical transmission, a redundant transmission channel, fault tolerance and a bus guardian.
- **TTP/C**
 - The information capacity of a frame is rather small.
 - It is not flexible enough. It cannot, for example, combine synchronous and asynchronous segments in a transmission, let a node use several slots to send data within the synchronous segment, or let nodes operate on single, dual or mixed channels.
- **Byteflight**
 - It offers very few functions.

For these reasons, manufacturers of cars, equipment and electronics formed the FlexRay consortium. Its aim was to create a communication protocol with a flexible configuration, meaning the ability to serve applications at four different levels:

- High throughput in digital data transmission, in order to improve, complete or extend applications that had been held back by the data rate of CAN.
- The ability to implement 'X-by-Wire' solutions.
- Solutions based on redundancy:
 - Repeated transmission of the same messages, made possible by the higher throughput.
 - Two separate communication channels carrying the same information in parallel.
 - Two communication channels used to make the most of the physical data rate defined by the protocol.
- The capacity to support all future functions in motor vehicles.

3.1.1 Required Specifications

The figure below shows the range of problems that the FlexRay protocol has to solve.

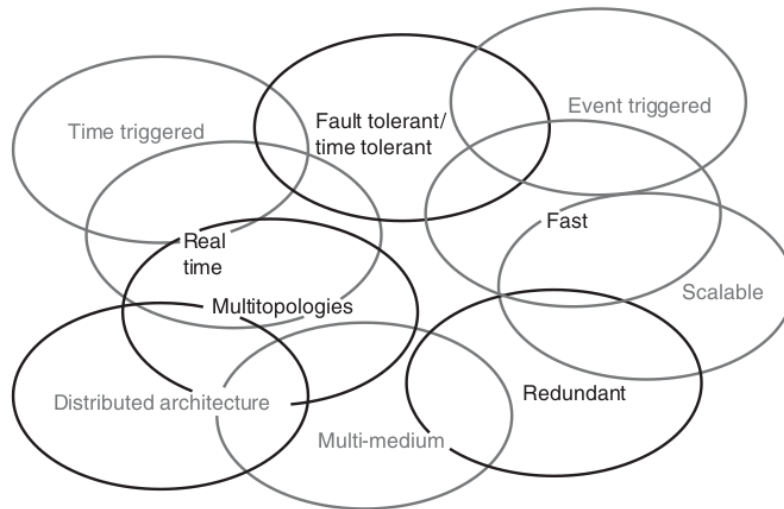


Figure 3.1: Range of problems to be solved

To address the issues shown in Figure 3.1, the consortium laid down a set of specifications. They are presented below and cover communication, topology, safety and the application.

Communication

- High throughput for digital data transmission (10Mbit/s).
- Synchronous and asynchronous data transmission.
- Data transmission must be deterministic, meaning that message delivery latencies are known and guaranteed.
- Different, simultaneous data rates at different nodes, with bandwidth easy to allocate to each node in the network.
- Support for both a static and a dynamic segment in data transmission.
- A configurable number of time slots per transmitting node and per cycle.
- Hardware and software redundancy.
- The ability for participants to enter a sleep state and to be woken up over the network.
- Management of resource consumption by the network participants.
- Fast error detection and signalling.
- Detection of synchronisation errors with respect to the global clock.
- Fault tolerance and time-triggered services provided in hardware.
- Containment of errors at the physical layer by means of the bus guardian.
- Support for a decentralised bus guardian at the physical layer.
- New nodes can be added without further configuration.

Topology

To give vehicle designers enough flexibility, FlexRay must support communication in the following topologies:

- Single channel
- Dual channel
- Bus, and bus with stubs
- Passive and active star
- Multiple star, with optional sub-buses
- Any combination of the above

Safety

For all systems of the ‘X-by-Wire’ type, FlexRay must provide the following:

- The ability to handle redundant communication.
- Deterministic access to the network.
- Avoidance of bus access collisions.
- Redundancy with variable geometry (single channel, dual channel, or a mixed system).
- Restarting a node involves more than just re-establishing communication.
- Communication with the other nodes is maintained, as long as it has not been compromised.
- The system is robust against transient faults and external radiation.
- Minimal exposure of the system to radiation, electromagnetic compatibility (EMC), and similar concerns.

Application

FlexRay must ensure that the application:

- is always responsible for all decisions concerning the safety and availability of the network.
- always makes the final decision.
- always has the communication system under its control, and not the other way round.
- keeps receiving data for as long as possible, because some information is critical to making the right decision.
- provides different operating modes for normal, continuous and degraded operation (for example, when errors are present).
- allows nodes to be configured to stay active for a number of cycles without receiving communication (or data) frames.

3.2 General Characteristics

To give the system the necessary flexibility, it must be capable of the following:

1. Real-time operation, that is, operation within specific and known time intervals. This ensures that the physical communication medium is used by only one entity at a time, so no collision can occur.
2. Event-driven communication, with an information rate that varies on demand. The communication time is then not known in advance, but it stays below a predefined limit.

In FlexRay, communication takes place in cycles called *communication cycles*. Within these cycles, communication happens both synchronously and asynchronously, in time slots predefined by the system designer. The communication cycles are divided into slots and minislots of equal duration, to which central processing units (CPUs) have exclusive access for transmitting data.

Designing and managing the network is the designer's responsibility, since the designer sets the values for the cycles and time slots. This principle of network access based on predefined time slots (TDMA) prevents message collisions on the physical transmission medium.

The FlexRay protocol includes two coupling models:

1. Time-triggered, where initiation and allocation are based on time.
2. Event-triggered, driven by external events.

To support both models within one communication cycle, the cycle has to be divided into two segments, the *static* and the *dynamic* one (for (1) and (2) respectively), as shown in the figure below.

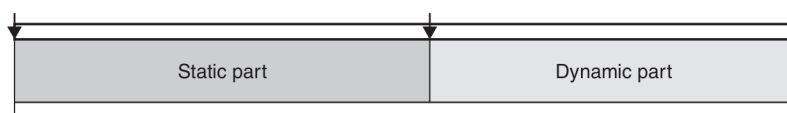


Figure 3.2: Division of the cycle into a static and a dynamic segment

FlexRay is built on a dual parallel hierarchy:

- In terms of functionality, data are encapsulated in communication frames, which are in turn placed in time slots. The time slots belong to segments of communication cycles, and the cycles in turn make up a communication channel.
- In terms of timing, the Global Time, Macroticks, Microticks, the local clock and so on are involved.

The structure can therefore be compared to a set of Russian nesting dolls, as the figure below shows.

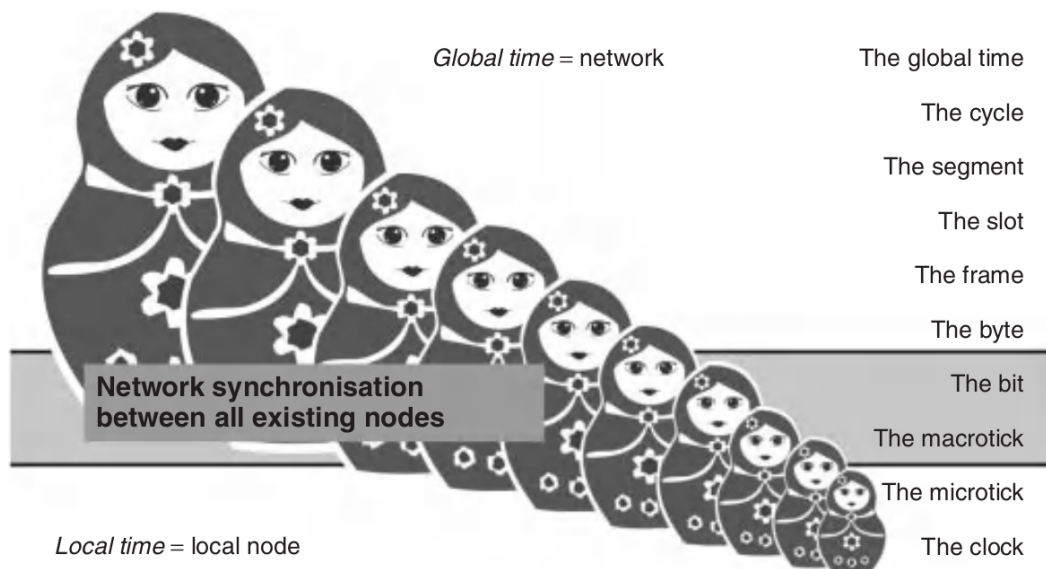


Figure 3.3: The dual parallel hierarchy of the FlexRay protocol

The next figure shows most of the elements of the FlexRay protocol. They cover every level, from the highest (the network) down to the lowest (the local microcontroller).

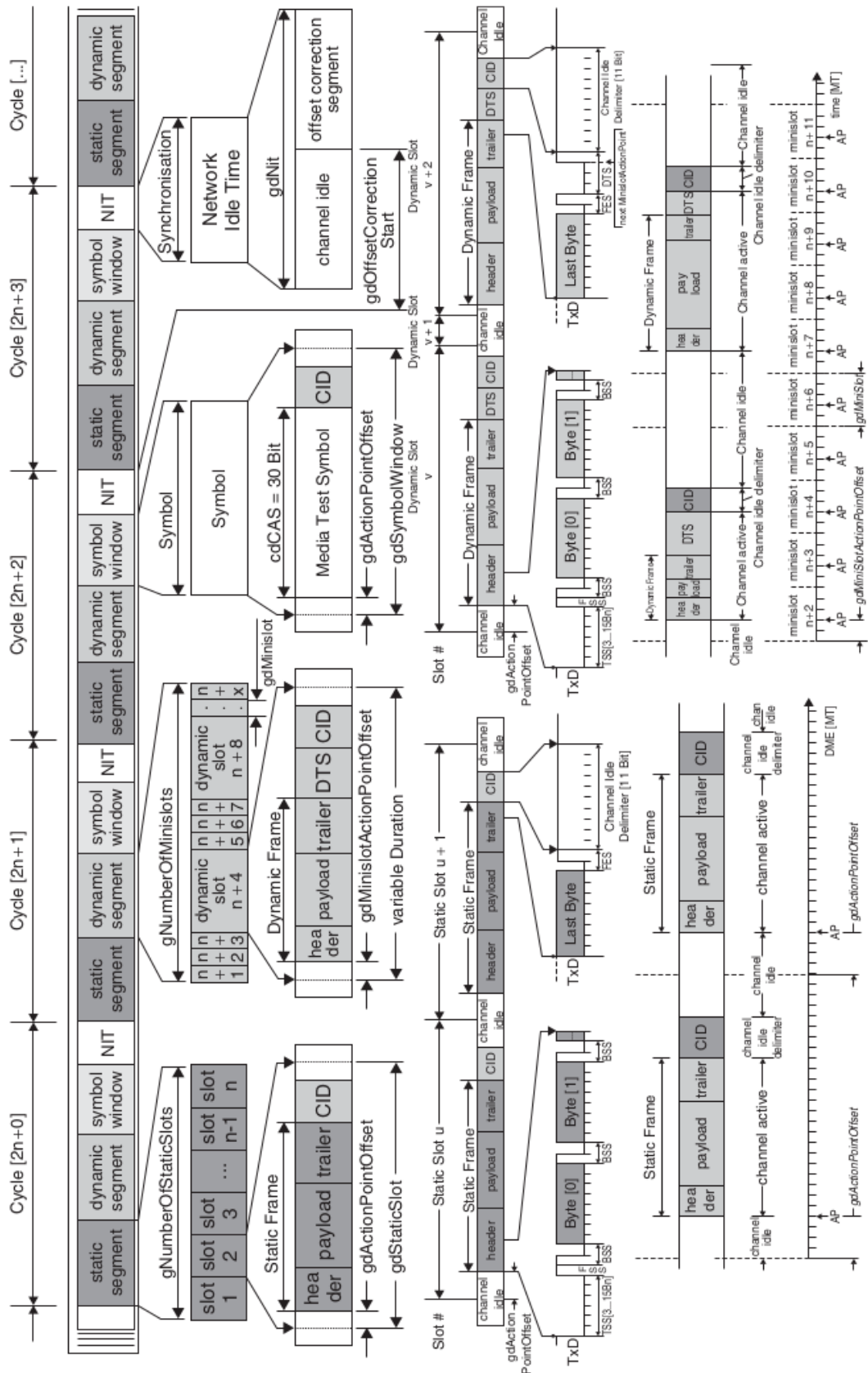


Figure 3.4: The elements of the FlexRay protocol

3.3 Communication Channels and Cycles

3.3.1 Communication Channel

The communication channel is the link between the various participants in the network. FlexRay supports two communication channels, A and B, each with its own constraints and freedoms. The system can nevertheless operate with a single channel. The design decision to have two channels was taken for two reasons:

1. Information can be supplemented, duplicated or offloaded from one channel to the other.
2. Future applications (such as X-by-Wire) can be added.

3.3.2 Communication Cycle

Communication is organised into communication cycles, which repeat and are all of equal duration. As a rule, the duration of a cycle is a constant tied to the ‘Global Time’.

The figure below shows the structure of the communication cycles.

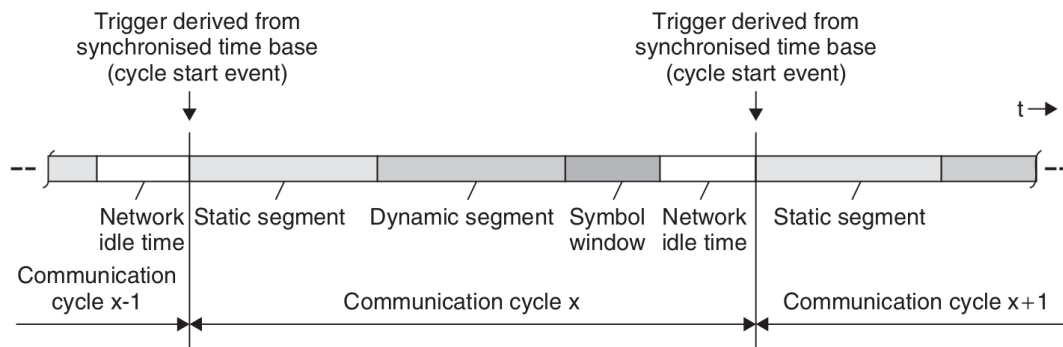


Figure 3.5: The structure of the communication cycles

From Figure 3.5 we see that the communication cycles are divided into four parts, called segments, which repeat at fixed time intervals. These segments are:

- The **static segment** (ST).
- The **dynamic segment** (DT), which is optional.
- The **symbol window** (SW), which is optional.
- The **network idle time** (NIT), which ends the cycle. This is an interval during which the network is idle.

Because of the requirements of real-time applications, the duration of the communication cycle is usually set between $10\mu s$ and $16ms$.

3.4 Segments

As mentioned in the previous section, the communication cycles are divided into segments (Figure 3.5) that repeat at fixed time intervals. The segments are explained in more detail below.

Static Segment: Used by deterministic real-time applications with a predefined bandwidth.

Dynamic Segment: Used by event-triggered applications, which follow probabilistic management with a variable bandwidth.

Symbol Window: Used by applications that employ a bus guardian (X-by-Wire applications, for example).

Network Idle Time: The phase during which the network remains idle.

3.4.1 Slots and Minislots

The static and dynamic segments are in turn divided into ‘slots’ and ‘minislots’ respectively, as Figure 3.6 shows. Slots and minislots carry static and dynamic communication frames respectively.

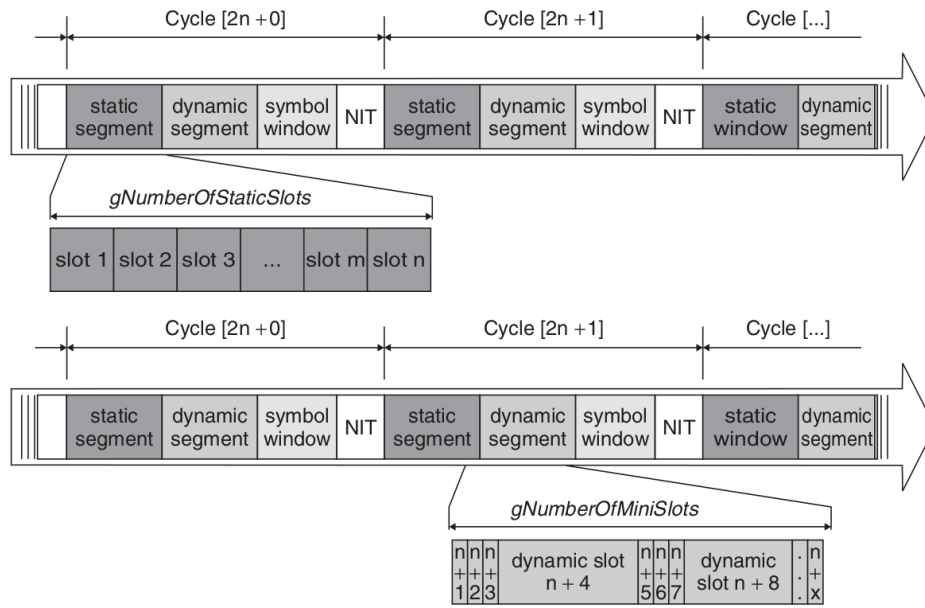


Figure 3.6: Slots and minislots

Static segment slots are used in real-time systems where the system designer must fix the time periods for messages in advance. Critical, unexpected messages may also arrive at any moment, however, so it must be possible to assign slots to them immediately within the communication cycle. This is where the minislots of the dynamic segment come in.

3.4.2 Static Segments and Slots

The static segment exists to guarantee high-performance deterministic communication, to establish the importance of messages, and to manage distributed systems and closed-loop control. Access to the medium during the static segment is governed by the Global Time Division Multiple Access (GT-DMA) principle.

The rules that govern the division of static segments into slots are listed below.

- All slots must have the same duration.
- All communication cycles must contain the same number of slots.
- Each slot is identified by a unique number (ID).
- In dual-channel communication, the slot format must be the same on both channels (duration, number and so on).

- The start of a slot is determined by the network's Global Time.
- The Global Time also determines the slot duration, on the basis of the clusters of participants.
- Only one node may transmit per slot.
- Each unique identifier (ID) represents a unique process, assigned according to the transmitting nodes.
- At least two static segment slots must be used during synchronisation.

3.4.3 Dynamic Segments and Minislots

Access to the medium during the dynamic segment is governed by the Flexible Time Division Multiple Access (FTDMA) principle. Before the dynamic segments are used, the system designer divides them offline into minislots. Minislots are short and all of equal duration. Their positions in time are numbered within the dynamic segment and relative to the full communication cycle.

Minislots represent 'possible instants' and depend on the application. When an unexpected event (one outside the normal timing, for instance) needs to be transmitted, a minislot can be used, thanks to its dynamic nature.

Some rules on defining and managing minislots follow:

- All communication cycles contain the same number of minislots.
- Their duration must be the same on both transmission channels.
- The access sequence during a dynamic segment may differ between the two communication channels.
- The unique function of output messages is closely tied to the node, given the dynamic slot and the channel parameters.
- The start of the dynamic segment can be linked to different operating modes, namely:
 - Mixed mode, in which static and dynamic segments coexist.
 - A mode in which there are only dynamic segments.
- When a communication element starts during a predefined minislot, it is called a dynamic slot.

3.4.4 The Symbol Window Segment

The Symbol Window segment is optional and is used to carry the Medium Access Test Symbol (MTS). The MTS is used to check that the bus guardian is working correctly. It has the same structure as the Collision Avoidance Symbol (CAS). Both symbols are 30 bits long. To end the Symbol Window, the MTS is followed by a Channel Idle Delimiter (CID), as shown in the figure below.

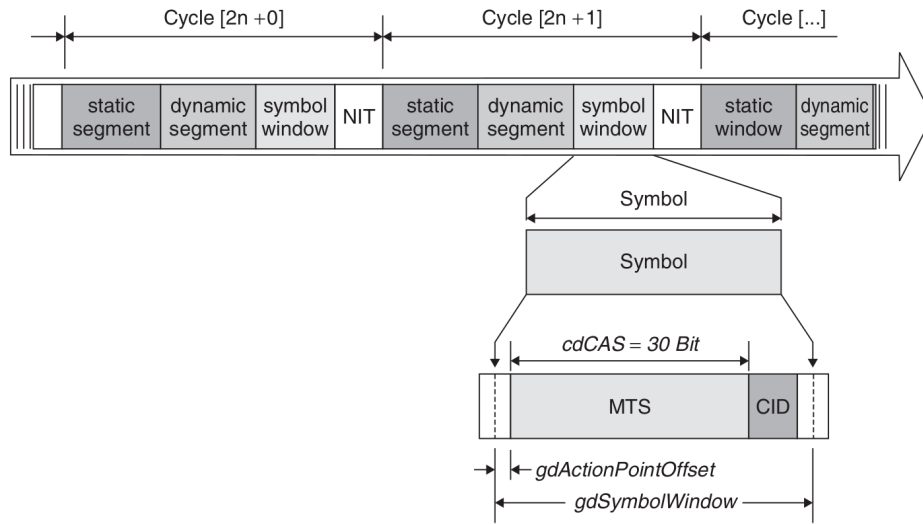


Figure 3.7: End of the Symbol Window

3.4.5 The Network Idle Time Segment

This segment marks the end of each communication cycle. During it there is no activity on the network, neither sending nor receiving of messages, because the network is idle.

In reality, though, there is intense activity inside the nodes, and this is what justifies the Network Idle Time segment. The nodes use the NIT to carry out the necessary synchronisation calculations and then correct the phase of their local clock. These actions are shown in Figure 3.8. Phase corrections affect every participant in the network, which is why the duration of the NIT depends on the size of the cluster.

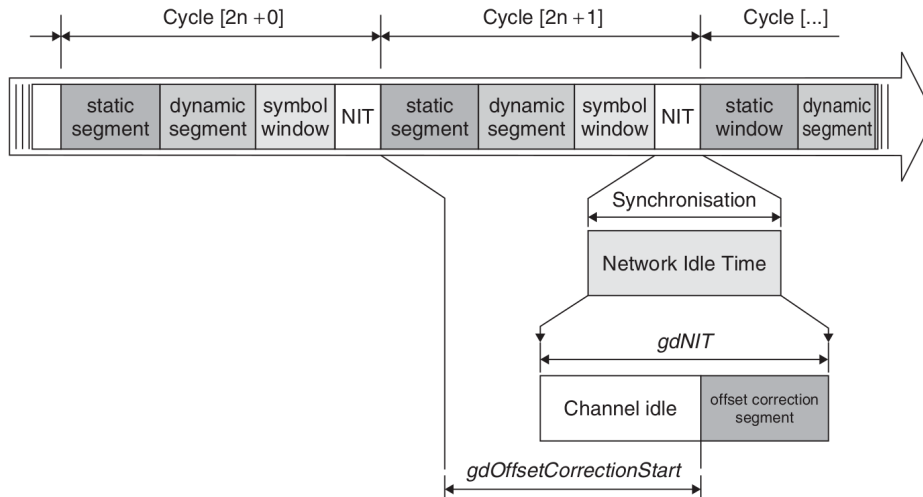


Figure 3.8: Local clock phase correction

3.5 Communication Frames

Communication frames carry the logical payload. They are transmitted in static segments, dynamic segments, or both, within static and dynamic slots respectively, as shown in Figure 3.9. The slots of the static segments and the minislots of the dynamic segments are occupied by static and dynamic communication frames respectively.

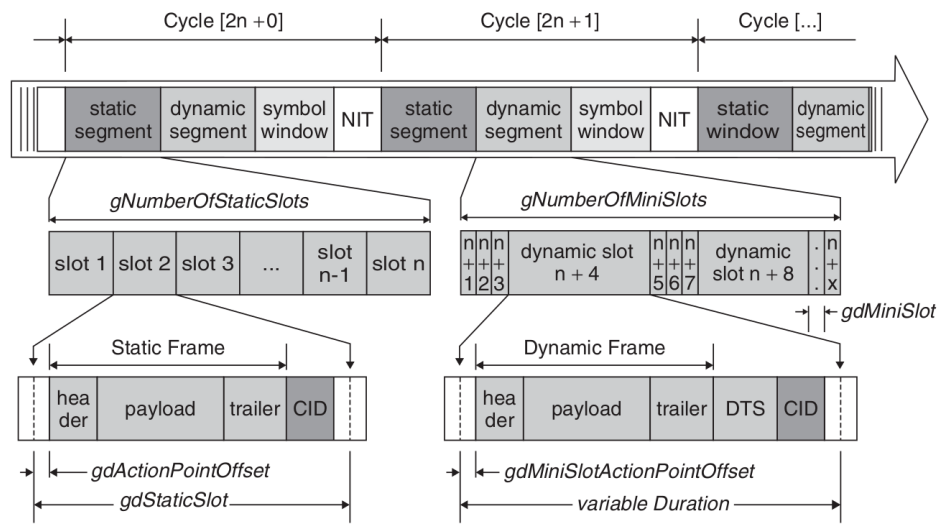


Figure 3.9: Transmission of communication frames

3.5.1 Frame Components

Static and dynamic frames consist of the *Header Field*, the *Payload* (the data itself), and the *Trailer*, which marks the end of the frame. These fields are shown in the figure below.

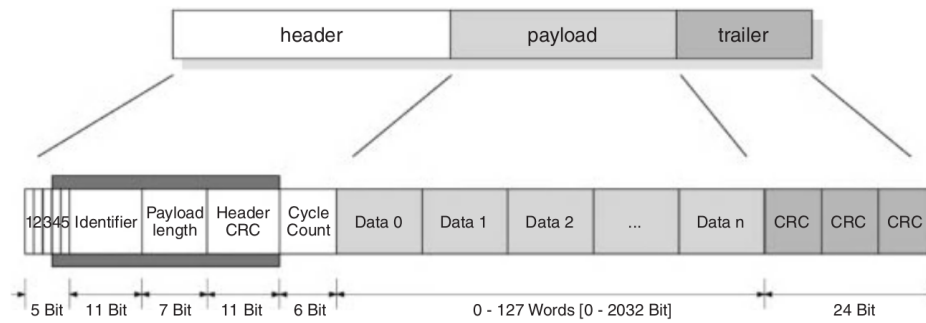


Figure 3.10: Components of communication frames

Header Field

The Header Field consists of the following parts:

- **Leading Indicator (5 bits):** The bits of the Leading Indicator are interpreted as follows:
 - Bit 1:** Reserved for future use (currently 1).
 - Bit 2:** Indicates the type of content in the Payload field. '0' stands for ordinary content and '1' for special content. In the latter case there are two options, depending on whether the frame is transmitted in the static or the dynamic segment.
 - Bit 3:** Indicates whether the frame is a null frame, meaning that the transmitting node has nothing to send since the previous cycle.
 - Bit 4:** Indicates whether the frame is a sync frame. If it is, the frame is to be used to synchronise the clocks of the nodes.
 - Bit 5:** Indicates whether the frame is a startup frame.
- **Slot/Frame ID (11 bits):** The unique identifier of the frame. It indicates the position of the slot in which the frame will be transmitted, within the static or the dynamic segment. Two controllers may not send frames with the same identifier (Frame ID) on the same communication channel.

- **Payload Length (7 bits):** Gives the number of words (the data length) contained in the payload. The length is counted in words, and one word consists of 2 bytes, so the value is the byte count divided by 2. A 7-bit field can take $2^7 = 128$ values (0 to 127), so the payload can be at most 127 words, or 254 bytes. Any value greater than 123 is treated as an error.
- **Header CRC (11 bits):** Used to check that the slot number and the Frame ID received from the transmitting node are consistent. This way the receiver does not go on to process data that has no value.
- **Cycle Count (6 bits):** The number of the communication cycle in which the frame is transmitted. It can also serve as a ‘continuity indicator’ for the communication. The controller that sends the frame increments this value automatically in every communication cycle.

Payload

This field carries the useful data. It can hold up to 127 words of 16 bits each, that is, 2032 bits at most.

Trailer/CRC

This field protects the frame being transmitted by means of a Hamming distance calculation. The distance must be 6 for data up to 248 bytes long, and 4 for longer data.

3.6 The Architecture of a Node

The typical architecture of a FlexRay node consists of four blocks:

- the *Host* microcontroller, which contains the application and the application-level management of the communication protocol.
- the *Protocol Manager*, or Communication Controller, which builds the cycles, segments and so on.
- the *Line Driver*.
- the *Bus Guardian*, which is optional.

Figure 3.11 shows the internal and external connections between these components (or blocks) in a FlexRay node. Quite often the protocol manager and the Host are contained in the same integrated circuit. Very few implementations today include a Bus Guardian.

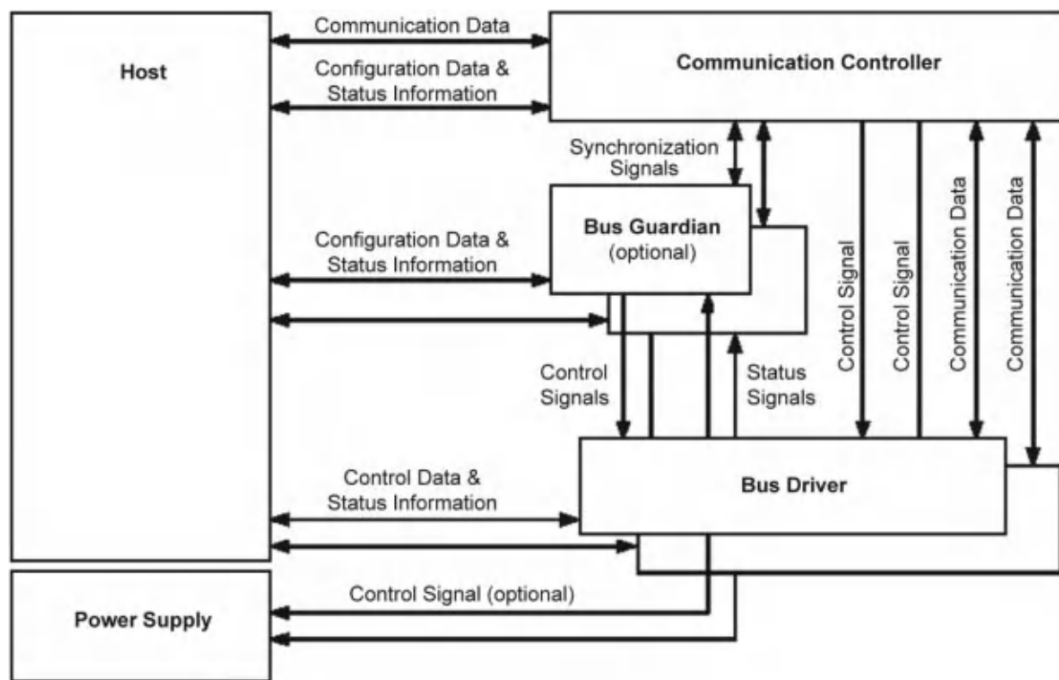


Figure 3.11: The connections within a FlexRay node

Figure 3.12 shows the logical relationships between these blocks in more detail. Reading Figure 3.12 from the bottom up, one can make out the separation between layers 1 and 2 on the one hand and layer 7 on the other, in terms of the OSI model.¹

¹See Appendix E.

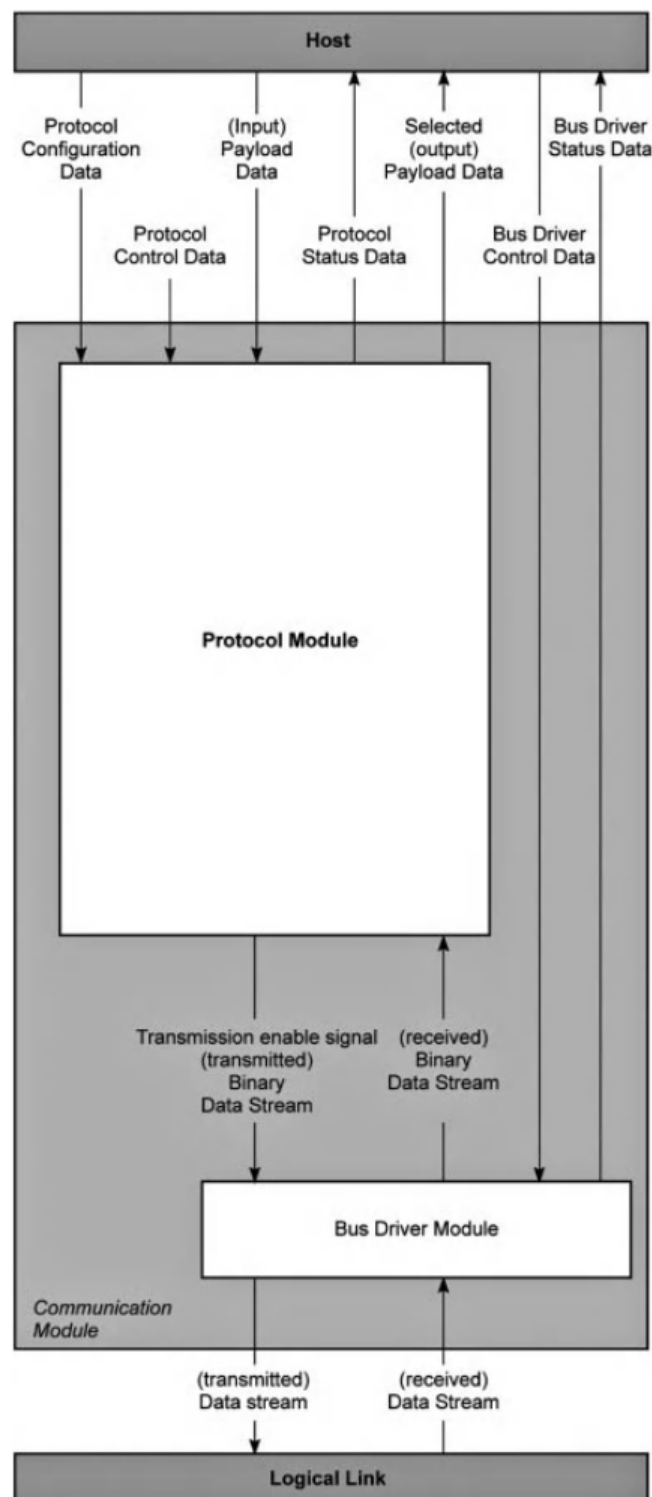


Figure 3.12: The logical relationships between the components of a FlexRay node

Figure 3.13 shows the internal architecture of the Protocol Controller, as proposed by the protocol specification. We see that every function is duplicated, since FlexRay is designed to support two communication channels.[5]

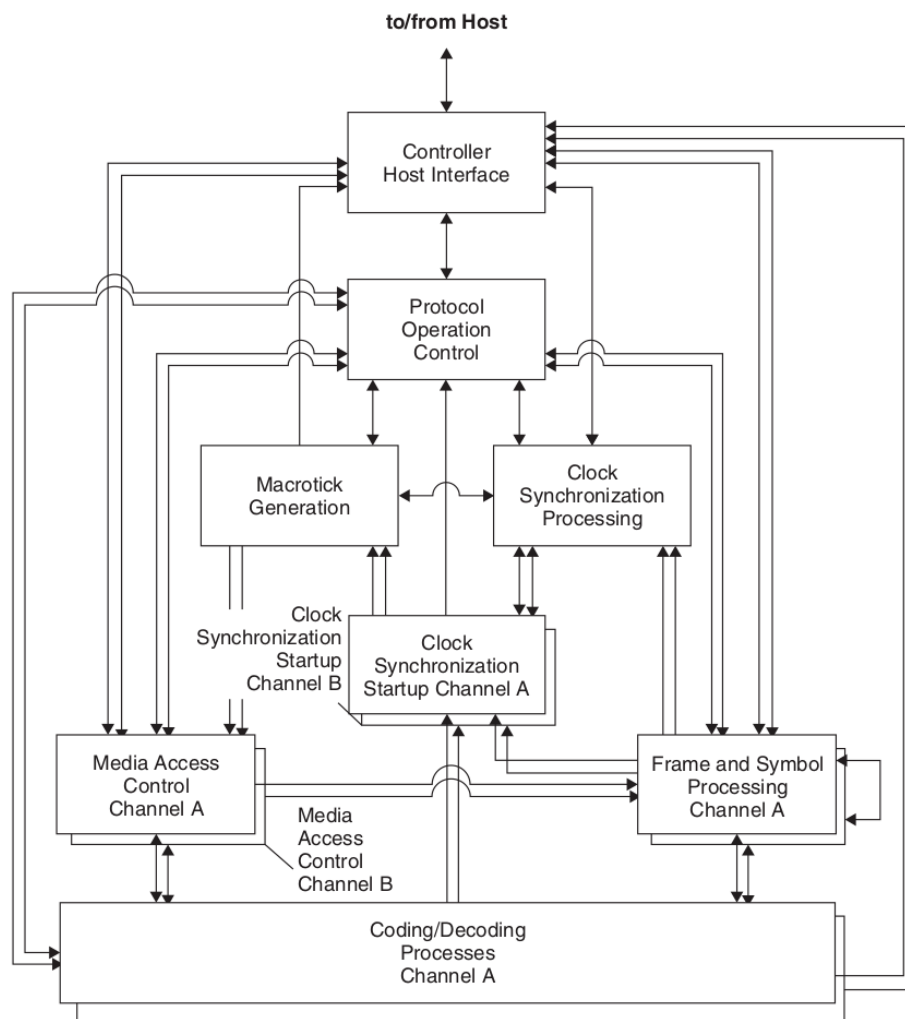


Figure 3.13: Internal architecture of the Protocol Controller

4. An Approach to Solving the Problem

This chapter presents the work of the authors of [3], namely their approach to solving the problem described in Section 1.1. The authors propose a number of improvements to the FlexRay protocol and then present an FPGA-based Communication Controller with extended capabilities. They also demonstrate the benefit of modifying the Communication Controller through experimental results from realistic, safety-focused use cases.

4.1 Improvements to the FlexRay Protocol

4.1.1 Topology

Each node/ECU typically contains an integrated or discrete Communication Controller, through which it can send and receive data frames. Data frames are transmitted in the slot or slots assigned to the node, as shown in the figure below.

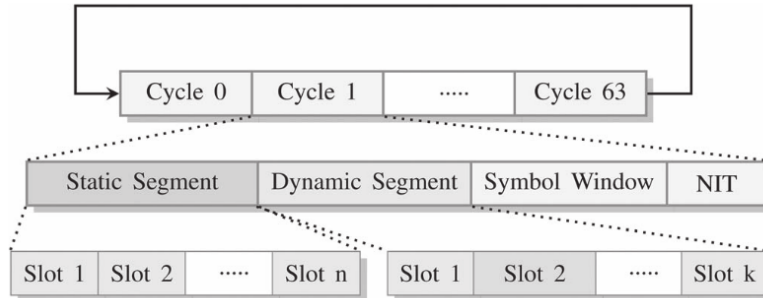


Figure 4.1: The FlexRay communication cycle

The FlexRay standard defines the active star topology, shown in the figure below.

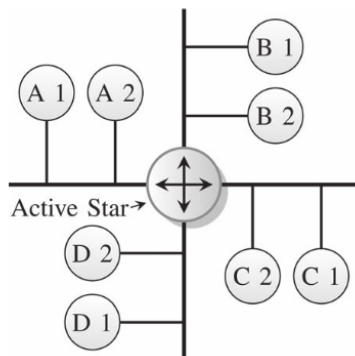


Figure 4.2: Active star topology

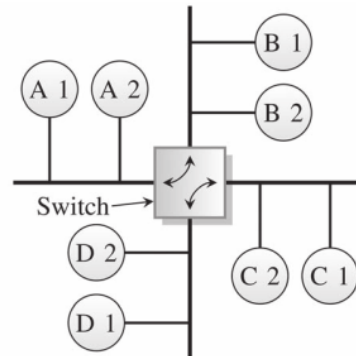


Figure 4.3: Switch topology

The problem with this topology is that the active star forwards information from one branch of the network to all the other branches. Nodes can therefore share the same slot only in different cycles. This design wastes a large share of the network's total bandwidth.

With a switch, on the other hand, as in Figure 4.3, the same slots can be used simultaneously by several nodes. This architecture allows the branches to be used in parallel, because the switch forwards each frame only to the branch that contains its receiver.

4.1.2 Scheduling

Many studies have looked at scheduling on the shared transmission medium, but they all aim mainly at optimising the number of slots used while meeting the given requirements.

Two general approaches have been presented. One seeks the minimum number of communication cycles needed to solve the problem, while using little bandwidth. The alternative aims to maximise the number of unused cycles, for maximum flexibility in the future.

4.1.3 Network-Level Optimisations

Extending the capabilities of the Communication Controller lets it handle various requests on behalf of the ECU. The ECU can then remain in the 'sleep' state, which improves the efficiency of the node.

4.2 The Communication Controller

As seen in Section 3.6, a node in a FlexRay network consists of a Communication Controller and a host ECU that runs a dedicated application. The Communication Controller ensures that the node complies with the FlexRay specification. It also lets the ECU communicate with the other ECUs in the network, by sending and receiving data on its behalf. The operating state of the Communication Controller is controlled by the host ECU.

4.2.1 Architecture

The figure below shows the architecture of the Communication Controller implemented by the researchers.

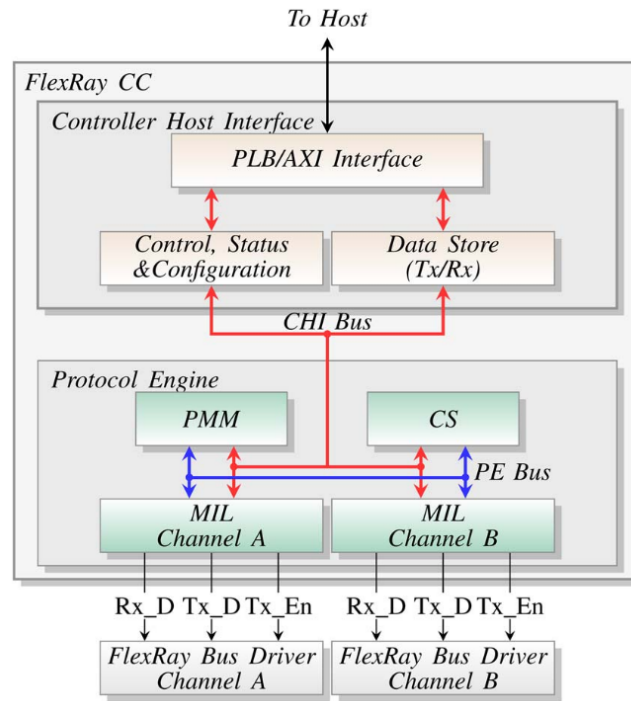


Figure 4.4: Architecture of the modified Communication Controller

The *Controller Host Interface* (CHI) communicates with the host and handles the commands and parameters used to configure the FlexRay node. During operation, the CHI continuously supplies the host with the current operating state and with control signals, so that the host can take corrective action if needed.

Clock Synchronization and the Medium Interface Layer are submodules of the *Protocol Engine* (PE) and are controlled by the *Protocol Management Module* (PMM). The PMM makes sure that changes of operating state follow the FlexRay protocol.

The *Medium Interface Layer* (MIL) is responsible for sending data to, and receiving data from, the Data Bus. It also encodes outgoing data and decodes incoming data.

Clock Synchronization (CS) generates the local clock and distributes it to the other modules of the Communication Controller. It is also responsible for synchronising this node with the rest of the network.

Synchronisation works by controlling the number of Microticks (the unit of local clock alignment) that make up one Macro-tick (the unit of alignment of the global clock, or of a cluster of the FlexRay network). FlexRay uses a distributed correction mechanism for synchronisation: each node makes the appropriate adjustments on its own, based on information it receives from the other nodes.

4.2.2 Implementation and Optimisations

The state of the PMM reflects the current operating state of the node. The diagram below shows every state the node can be in.

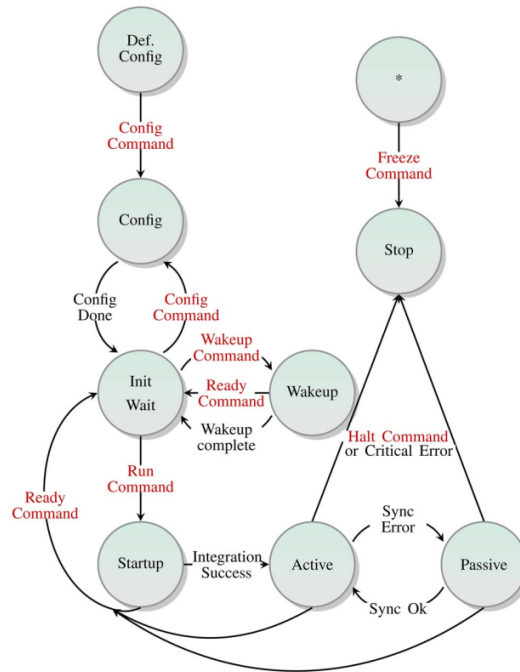


Figure 4.5: Operating states of the Communication Controller

Changes of operating state are caused either by host commands or by network conditions. The table below lists all the host commands and how each one affects the operating state.

Host Com-mand	Affected States	Final State	Processed at
ALL SLOTS	Active, Passive	no state change	End of cycle
ALLOW COLD-START	All states except Def. Config, Config, Stop	no state change	Immediate
CONFIG	Def. Config, Init Wait	Config	Immediate
CONFIG COMPLETE	Config	Init Wait	Immediate
DEFAULT CONFIG	Stop	Def. Config	Immediate
FREEZE	All States	Stop	Immediate
HALT	Active, Passive	Stop	End of cycle
READY	All states except Def. Config, Config, Init Wait, Stop	Init Wait	Immediate
RUN	Init Wait	Start-Up	Immediate
WAKEUP	Init Wait	Wake-Up	Immediate

Figure 4.6: Host commands

Wakeup State: Every node in a FlexRay network can enter the sleep state to save energy. When a node wants to communicate, it first moves to the wakeup state. In that state it sends a sequence

of symbols intended to wake the other nodes. When the sleeping nodes detect this sequence, they leave the sleep state so that they can respond to future requests.

Startup State: When the node enters the startup state, the synchronisation process begins. The Clock Synchronization module continuously checks the clock deviation relative to the time at which the frame is received. If the deviation is within limits, synchronisation has succeeded and the node moves to the *active* state. If not, it moves to the *passive* state. Throughout operation the node keeps switching between the active and passive states, depending on its synchronisation status.

The simplified architecture of the Clock Synchronization module is shown below.

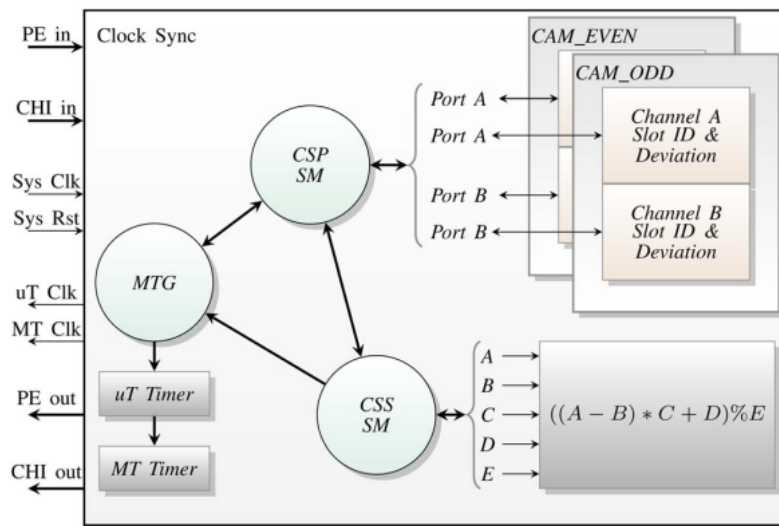


Figure 4.7: Architecture of Clock Synchronization

The *Macro-tick Generator*, or *MTG*, is the submodule that controls the clock cycle and Macro-tick counters. It is also responsible for correcting the frequency and phase values.

The *Clock Synchronization Process*, or *CSP*, handles initialisation at startup, as well as measuring and storing the deviation values. It also computes the phase and frequency values.

The *Clock Synchronization Startup*, or *CSS*, is responsible for starting a synchronised clock when the node synchronises, or when it establishes communication with an idle network.

In essence, the CSP finite state machine controls the operation of the CS module by interacting with the other two submodules, the MTG and the CSS.

The figure below shows the architecture of the Medium Interface Layer (MIL).

As Figure 4.9 shows, several pieces of information can be checked when a frame is received, and they decide the fate of the frame.

Msg ID filter: The unique identifier (ID) of the frame could carry error indications, so that an error recovery process starts immediately in safety-critical applications.

Appl. headers: The data segment of the frame could include headers with information on the data length and the sequence number of the frame. Such information is useful when transferring the data takes many cycles. The Communication Controller would then be able to recognise fragmented data and schedule its delivery to the host so that resources are used as well as possible.

Timestamp validation: Adding timestamps to data frames would help identify stale data, so that it can be discarded immediately and the host is spared from processing it.

4.3 Experimental Results

To evaluate the design and performance of this implementation, a complete Electronic Control Unit was modelled on a Xilinx Spartan 6 XC6SLX45 FPGA. The host was described as a state machine. For the network communication tests, a FlexRay DataBus was emulated inside the FPGA, receiving requests from a real FlexRay network. The Communication Controller is connected directly to the FlexRay DataBus and configured with the corresponding parameters for operating on the network.

The results of one of the three use cases presented in the paper [3] are given below. They show that the extensions added to the Communication Controller lead to smarter and more efficient systems than traditional implementations.

4.3.1 Use Case: Fault Detection & Fallback Operation

Safety-critical applications use fallback modes that guarantee minimal functionality if hardware or software faults occur. A critical parameter here is the time needed to activate the fallback mode once the fault has been detected.

This use case concerns an automated braking system. The following ECUs were used in the experiment:

1. The *Brake Sensor*, which controls the sensor units.
2. The *Brake Actuator*, which sends the appropriate commands to the brake system.

Each of these ECUs includes fallback logic that is activated when an error indication message is received. Error indication messages are sent by a fault detection unit, which continuously monitors the DataBus for erroneous data or commands.

For the experiment, erroneous data intended for the brake actuator were injected into the memory of the brake sensor, to be sent over the FlexRay DataBus. While the erroneous data were being transmitted, the fault detection unit detected them and then sent an error indication message.

With a conventional Communication Controller, the received error indication message is decoded and passed straight to the processor, where the data are processed to activate the fallback mode. The average latency in this case is 9.05 ms, marked "RT" in Figure 4.10.

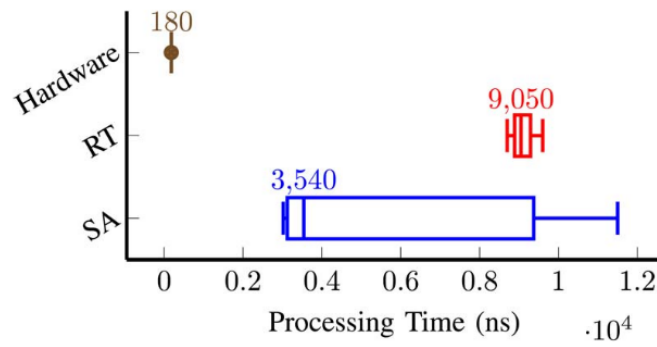


Figure 4.10: Latency of critical data processing

Moving the data processing into the Communication Controller, by contrast, cuts the latency considerably. In this case an interrupt to the processor is raised once the data have been processed, so that it switches to the fallback mode. The latency is much lower, and the request is served on average 50 times faster (Figure 4.10, marked "Hardware").

The experiment was also carried out with software processing of the critical data. As Figure 4.10 shows (marked "SA"), however, the latency is considerably higher.

4.4 Conclusions

In summary, extending the capabilities of the Communication Controller in an ECU achieves a significant reduction in energy consumption, and at the same time gives the FlexRay node additional functions that the protocol did not provide. This approach is therefore an ideal solution to the efficiency problem of the FlexRay protocol. Finally, the approach can also be applied to other time-triggered protocols, such as time-triggered Ethernet.

5. Design and Simulation of the Communication Controller

This chapter presents the main part of the thesis, namely the design and simulation of a Communication Controller based on the FlexRay protocol. The architecture and functionality are based on the Communication Controller of the paper [3], which was summarised in Chapter 4. The paper, however, gives only incomplete architectures for the submodules inside the Communication Controller. A custom approach was therefore followed in designing their architectures.

First, the general architecture of the Communication Controller to be designed is presented, together with the general function of each of its submodules. The structure of the data frame used is also presented. Each submodule is then analysed separately in terms of its architecture and operation, and a simulation of each submodule follows to confirm that it operates correctly. Finally, the complete system is simulated to confirm its correct operation, and the results of its synthesis are presented.

5.1 General Architecture

Figure 5.1 shows the general architecture of the Communication Controller. Most of the components inside the Communication Controller are inspired by the paper. The Wakeup Pattern Recognition (WUPR) and Wakeup Pattern Generator (WUPG), however, were added in this work, to give the Controller more functionality that would normally be carried out by the Host/Microcontroller. More information on these submodules is given below.

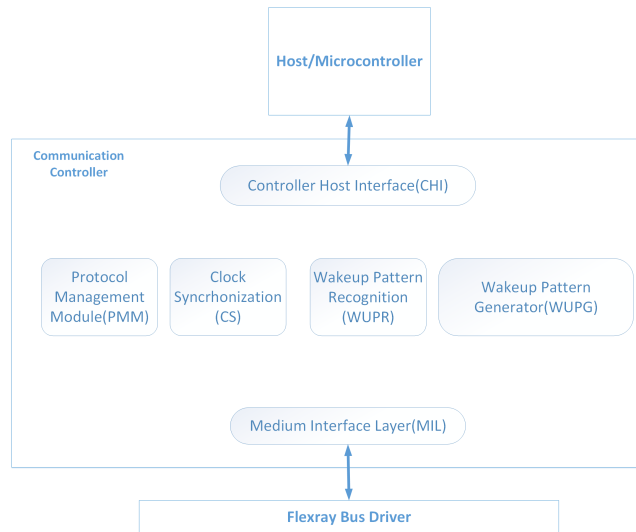


Figure 5.1: General architecture of the Communication Controller

Controller Host Interface (CHI): Acts as the interface to the Host. Through the CHI, the Host can continuously check the operating state of the Communication Controller, send it commands, and send it data frames that belong to the Host's application.

Medium Interface Layer (MIL): Mediates between the Communication Controller and the FlexRay Bus Driver/DataBus. It is responsible for passing data frames to and from the network, and it performs encoding and decoding, frame scheduling, and error detection.

Protocol Management Module (PMM): The brain of the Communication Controller. Through a finite state machine it keeps the system compliant with the FlexRay protocol. It interprets the commands received from the Host and controls the other components.

Clock Synchronization (CS): This submodule does exactly what its name suggests: it is responsible for synchronising the node that contains the Communication Controller with the rest of the network.

Wakeup Pattern Generator (WUPG): Generates a specific sequence of symbols intended for other nodes in the FlexRay network. The sequence is used to wake up the other nodes, which are in the ‘sleep’ state.

Wakeup Pattern Recognition (WUPR): Recognises the symbol sequence described above. When the sequence is recognised, a corresponding signal is sent to the Controller Host Interface and from there to the Host.

5.2 The Data Frame Structure

Figure 5.2 shows the structure of the data frame used.

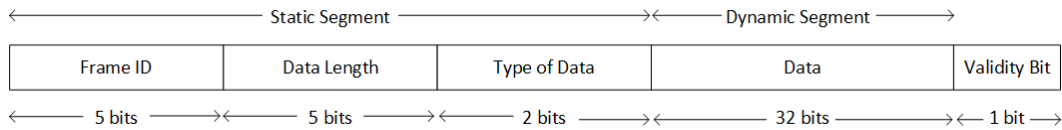


Figure 5.2: The data frame used

As the figure shows, the frame consists of three parts: the static segment, the dynamic segment, and a validity bit.

The *static segment* is essentially the header of the frame, and consists of the following fields:

Frame ID: A 5-bit number that serves as the unique identifier of the frame. In the present design this field serves no particular purpose. It was nevertheless added for future applications.

Data Length: Represents the length of the data contained in the frame, that is, the actual size of the dynamic segment.

Type of Data: This number represents the type of data contained in the frame. The field is used mainly to determine where the data frame is headed inside the Communication Controller. The type of data can be one of the following:

- Ordinary data, intended for the Controller Host Interface.
- A wakeup pattern, used by the Wakeup Pattern Recognition submodule to recognise the pattern (or symbol sequence).
- Synchronisation data, which are critical for synchronising the node and are processed by Clock Synchronization.

Since three types of data are used, at least 2 bits are clearly needed to represent them ($\log_2 3 = 1.58$). This is why the field is 2 bits long.

The *dynamic segment* consists of a single field: the data contained in the frame. The size of the data that can be carried in one frame clearly cannot exceed 32 bits.

Finally, the *validity bit* indicates whether the frame is valid, meaning that it contains real data and not ‘garbage’. It is used to decide whether the frame is allowed onto the Data Bus of the network.

5.3 Protocol Management Module

As mentioned in Section 5.1, the Protocol Management Module (PMM) contains a finite state machine that follows the FlexRay protocol. The machine stores the current operating state of the Communication Controller, interprets commands from the Host, and controls the other components through the appropriate signals. The architecture of the PMM is given in the figure below.

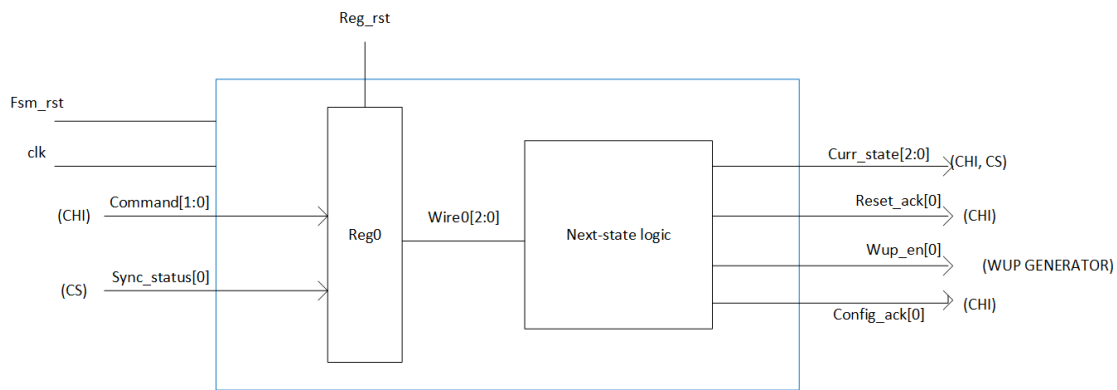


Figure 5.3: Architecture of the Protocol Management Module

In Figure 5.3, Reg0 is a register built from a D Flip-Flop. The Next-state Logic is the finite state machine. The figure also shows the names of the signals, their widths and their destinations. The output signal curr_state[2:0], for example, is 3 bits wide (bits 2 down to 0) and goes to two submodules, the Controller Host Interface (CHI) and Clock Synchronization (CS).

5.3.1 Commands from the Host/Microcontroller

All the possible commands that can be received from the Host are listed below.

WAKEUP: Sends a sequence of symbols to wake up the other nodes.

STARTUP: Starts the normal operation of the Communication Controller. Once normal operation begins, the node also starts synchronising with the rest of the network.

CONFIG: Initialises certain signals before normal operation begins. Once they have been initialised, an acknowledgement signal (ACK) is sent to the Host.

HALT: Stops the operation of the Communication Controller immediately.

5.3.2 The Finite State Machine

The finite state machine resides in the Next-state Logic submodule of Figure 5.3. Its specification is given in the state table below.

Current State	command	Input config_ack	sync_status	Next State	config_ack	Output WUP_EN	reset_ack
default_config	CONFIG			config			
config		1		sleep	1		
sleep	WAKEUP			wakeup		1	
wakeup	STARTUP			startup			
startup			1	active			
active	HALT		0	passive			
passive	HALT		1	active			
stop				default_config			1

Table 5.1: State table of the FSM in the PMM

To make the specification easier to follow, a state diagram is given below, showing the operation of the machine as a graph.

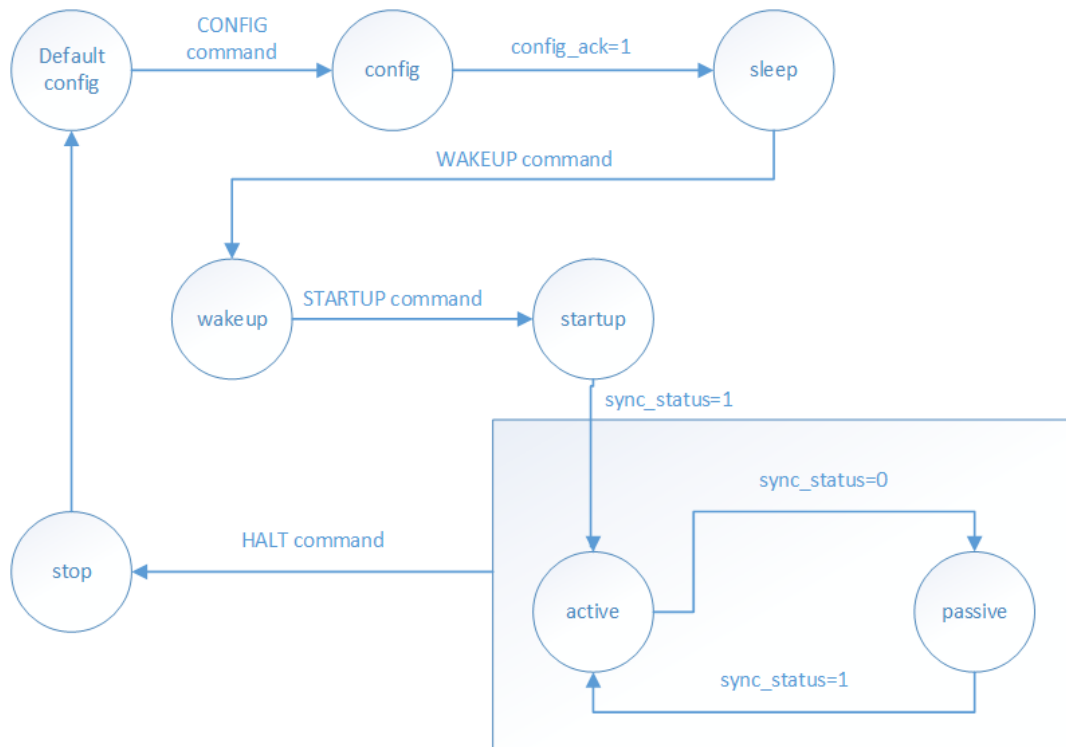


Figure 5.4: State diagram of the FSM

Figure 5.4 makes clear that the commands received from the Host perform the functions described in Subsection 5.3.1. The following points should, however, be noted:

- When the operation of the Communication Controller is terminated with the HALT command, the machine moves to a stop state. After some time it returns to the initial state of the system (Default Config), so that the system restarts.
- As mentioned earlier, once normal operation of the Controller begins (Startup state), the synchronisation process follows. After the node has successfully synchronised with the network, the machine enters a superstate (the solid box in Figure 5.4), in which it continuously checks

whether the node is synchronised with the rest of the network. Inside this superstate, the Controller keeps moving between the active and passive states, depending on whether synchronisation succeeds or not.

Finally, the continuous interaction between the Communication Controller and the Host/Microcontroller through the commands shows that the Host still has control of the Controller. Although the capabilities of the Communication Controller have been extended so that it performs some functions independently, the Host keeps control of it.

5.3.3 Testbench

State	Codeword
default_config	000
config	001
sleep	010
wakeup	011
startup	100
active	101
passive	110
stop	111

Command	Codeword
WAKEUP	00
STARTUP	01
CONFIG	10
HALT	11

Table 5.3: Encoding of the Host commands

Table 5.2: State encoding

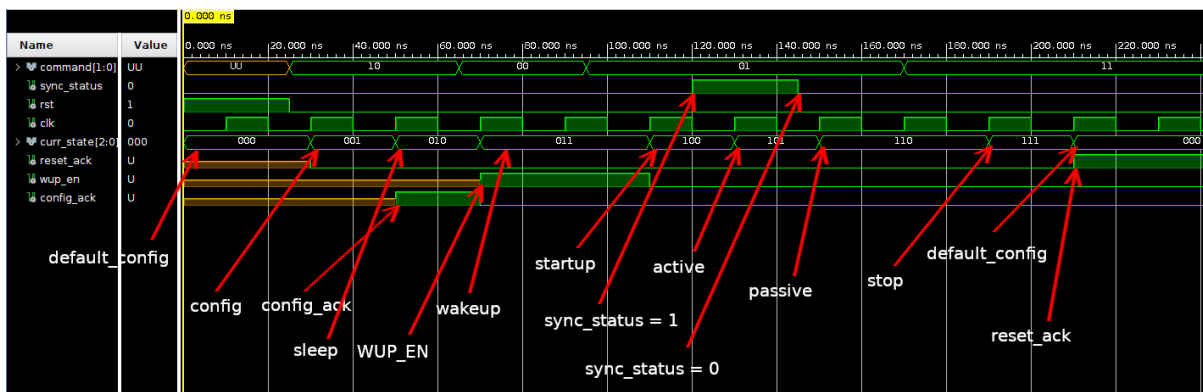


Figure 5.5: Testbench of the Next-state logic submodule (FSM), file: pmm/fsm_tb.vhd

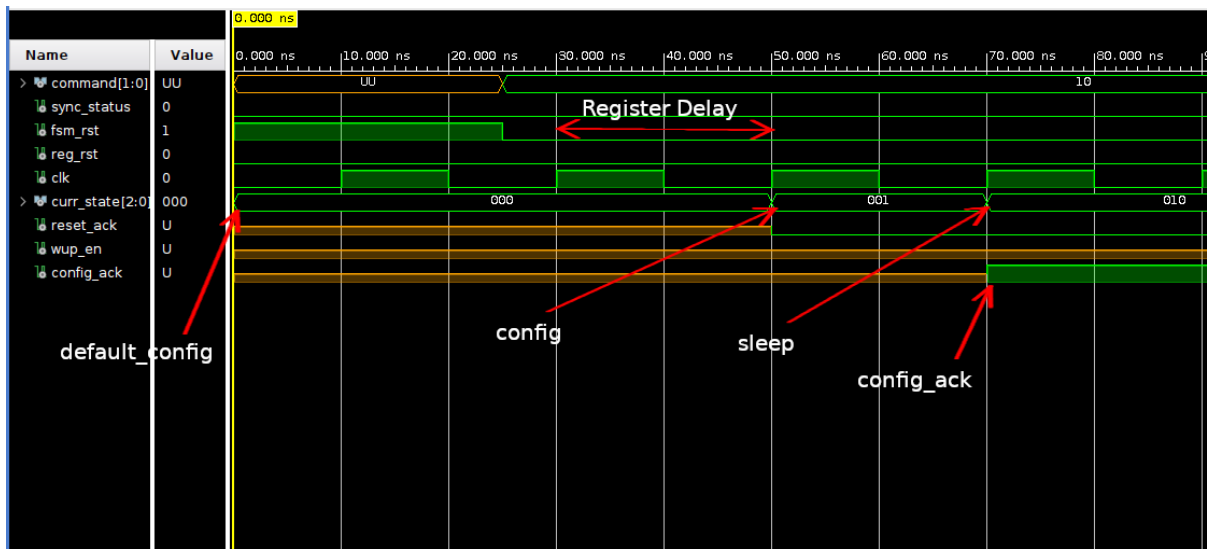


Figure 5.6: Testbench of the Protocol Management Module, file: pmm/pmm_tb.vhd

5.4 Controller Host Interface

The Controller Host Interface (CHI) is the means by which data pass between the Host and the Communication Controller. These data concern the current operating state, commands, and data frames. Figure 5.7 shows the architecture of the CHI. As the figure shows, its main functions are data buffering and the grouping of certain signals, which makes them easier to understand and to identify in the testbenches.

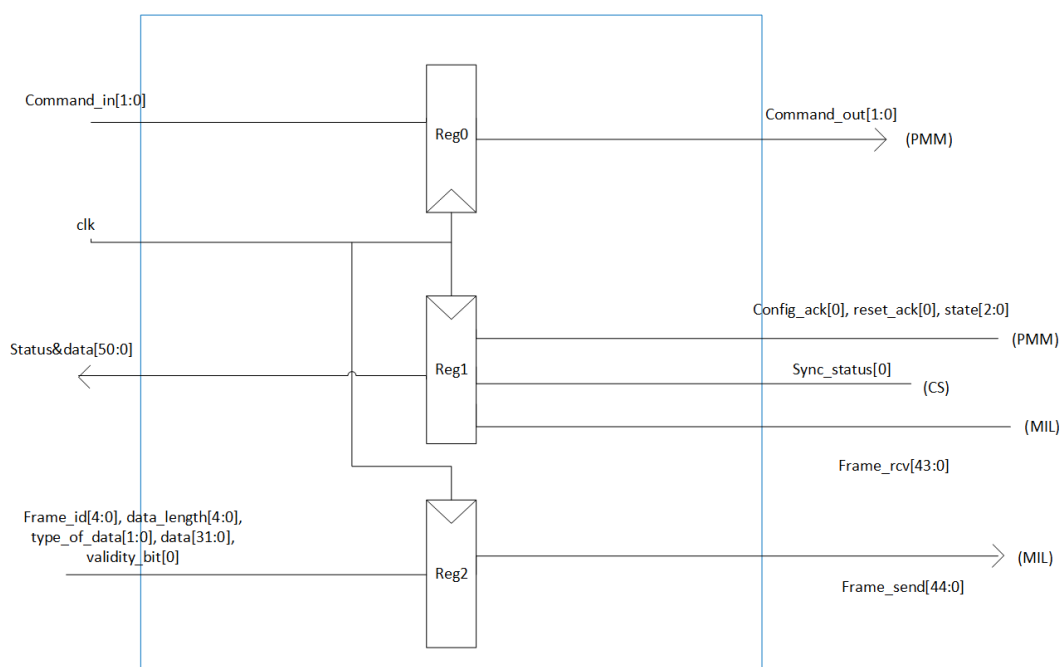


Figure 5.7: Architecture of the Controller Host Interface

5.4.1 Testbench

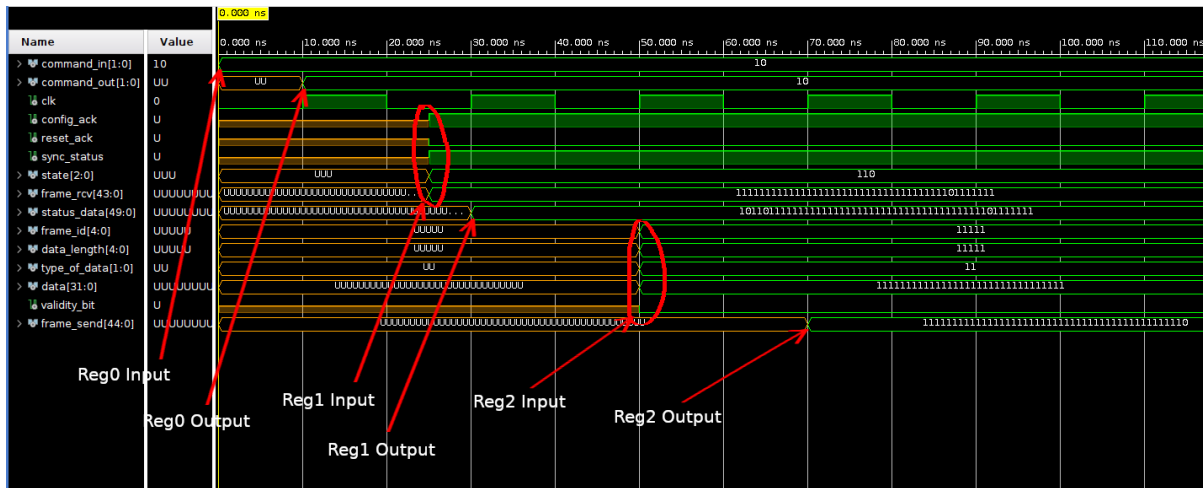


Figure 5.8: Testbench of the Controller Host Interface, file: chi/main_tb.vhd

5.5 Medium Interface Layer

The Medium Interface Layer (MIL) is responsible for sending data frames to, and receiving them from, the FlexRay Data Bus. It also performs various operations on the frames, among them encoding outgoing frames, decoding incoming frames, scheduling outgoing frames, and error detection. The architecture of the MIL is shown in the figure below.

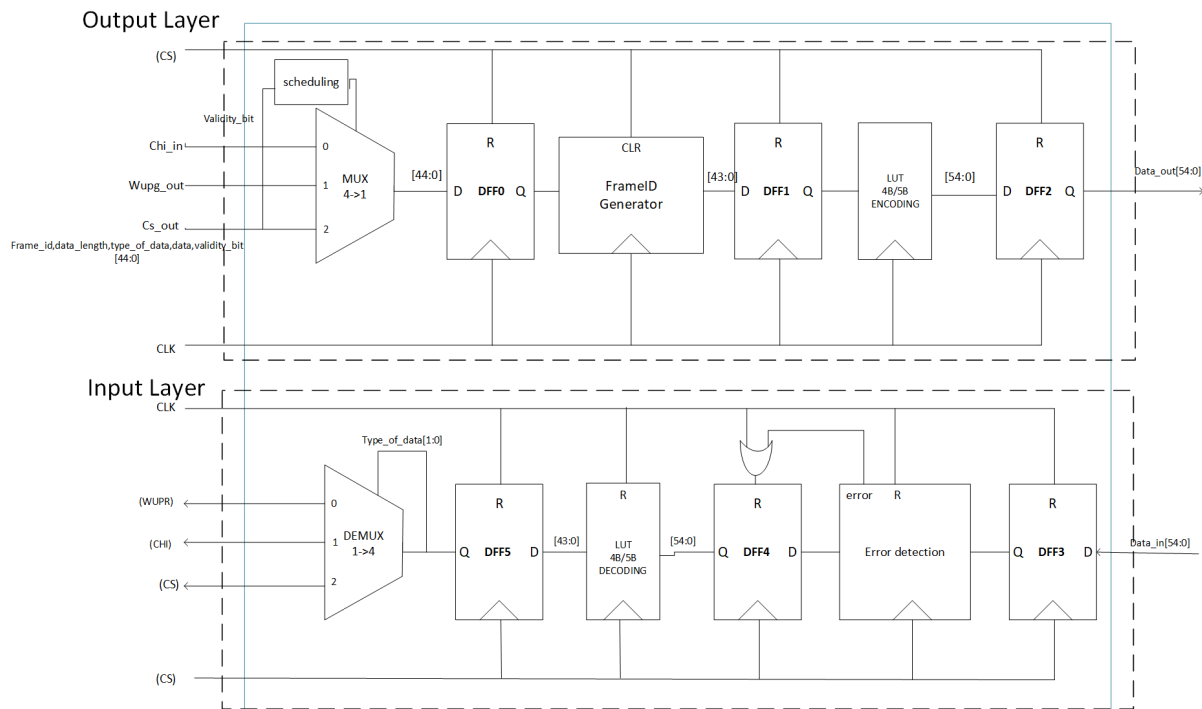


Figure 5.9: Architecture of the Medium Interface Layer

The *Output Layer* is the part that processes data frames destined for the network, while the *Input Layer* processes the frames received from the network.

Note also that DFFX (where X is an integer) denotes a D Flip Flop, while Mux and Demux stand for multiplexer and demultiplexer respectively.

5.5.1 Output Layer

As Figure 5.9 shows, frames entering the Output Layer first pass through a multiplexer (MUX). The multiplexer is necessary: without it there would be collisions, since several internal components of the Communication Controller could send frames in the same clock cycle.

Which data frame goes through first is decided by the *Scheduling* circuit. This circuit takes the validity bits of the frames (validity_bit in Figure 5.9) as inputs, so that it knows which frames are valid. It then forwards the frames according to where they come from. The order of priority is as follows:

1. Clock Synchronization (CS)
2. Controller Host Interface (CHI)
3. Wakeup Pattern Generator (WUPG)

This gives the following algorithm:

```

if (validity_bit_cs == 1) {
    output = frame_from_cs;
} else if (validity_bit_chi == 1) {
    output = frame_from_chi;
} else if (validity_bit_wup == 1) {
    output = frame_from_wup;
} else {
    output = Z;
}

```

In VHDL, 'Z' means high impedance.

From the algorithm, the truth table of the Scheduling circuit is filled in as follows:

Input (validity bits: cs, chi, wup (= a, b, c))	Output (select s1, s0)
000	ZZ
001	01
010	00
011	00
100	11
101	11
110	11
111	11

Table 5.4: Truth table of the frame scheduling circuit

The following logic equations follow from the table above:

$$S1 = ab'c' + ab'c + abc' + abc = a$$

$$S0 = a + a'b'c$$

Now that the logic equations show how the output of the circuit follows from its inputs, the circuit can be designed at gate level. It is shown in Figure 5.10.

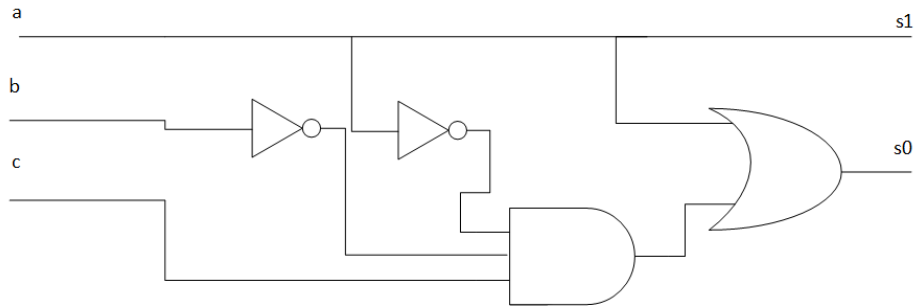


Figure 5.10: Frame scheduling circuit

The *Frame ID Generator* is essentially a counter that assigns each outgoing frame a number as its unique identifier. As mentioned in Section 5.2, the Frame ID serves no purpose in this design, but it could be used in future applications, for example to send an acknowledgement (ACK) of the successful reception of a frame.

We also see that the signal at the output of the Frame ID Generator has one bit fewer. This is because the validity bit is removed, since it only matters locally, inside the Communication Controller.

The *LUT 4B/5B Encoding* submodule encodes the data frame to be forwarded. 4B/5B encoding is explained in detail in Section D.1. As Figure 5.9 shows, the final frame is 55 bits long, since this encoding replaces every 4 bits with 5.

5.5.2 Input Layer

When a data frame enters the node, it should be checked for errors that may have occurred during its transmission over the network. This is done by the *Error-detection* component. Error detection relies on the 4B/5B encoding. As mentioned in Section D.1, every possible 4-bit sequence corresponds to a unique 5-bit sequence. Checking whether an encoded block is valid is therefore simple, since it only has to be looked up in the dictionary (Table D.1). Note also that this design does not use the control sequences (Table D.2), so their presence is treated as an error. The architecture of the Error-Detection submodule follows.

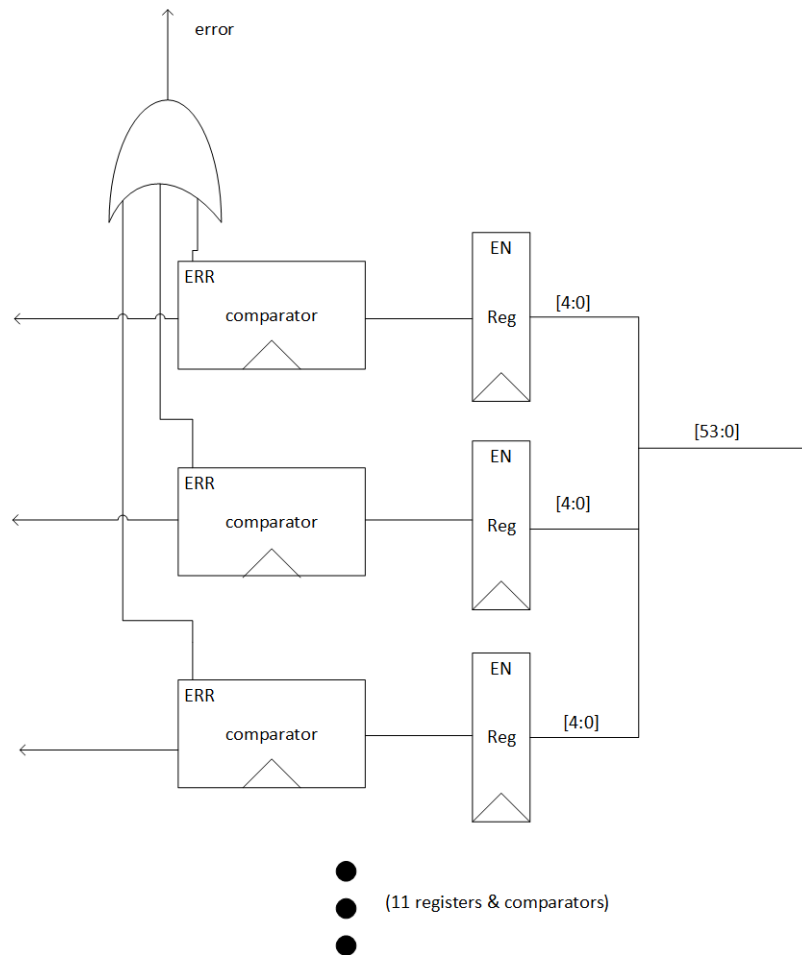


Figure 5.11: Architecture of the error detection submodule

As Figure 5.11 shows, each 5-bit block is checked separately by a comparator. If even one erroneous block is found, the comparator sets its ERR output to 1. The output of the OR gate then becomes 1. Going back to Figure 5.9, we see that with the error output of the Error-detection circuit at 1, the next Flip-Flop is reset in the next clock cycle. In this way the frame with the erroneous data block is ‘dropped’, that is, discarded.

The *LUT 4B/5B Decoding* submodule decodes the data frame. Decoding is the exact reverse of encoding: every 5-bit block is translated back into a 4-bit one.

In Figure 5.9 we see that, before the data frame leaves the Input Layer, a demultiplexer sends it to the component it is intended for. The destination is decided by the Type of Data field, explained in Section 5.2.

5.5.3 Testbench

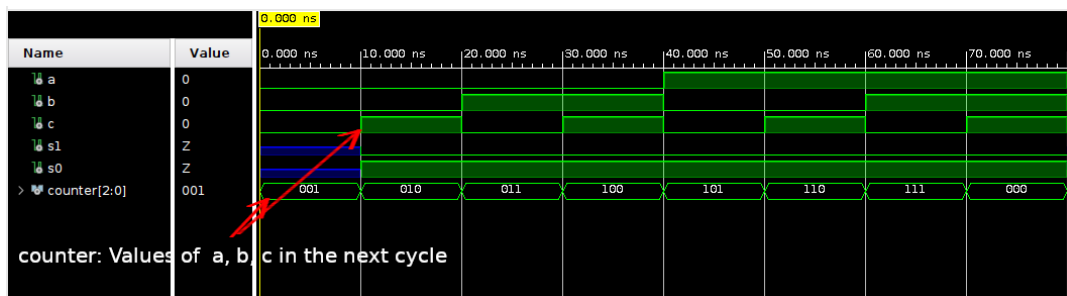


Figure 5.12: Testbench for Scheduling (Output Layer), file: mil/scheduler_tb.vhd

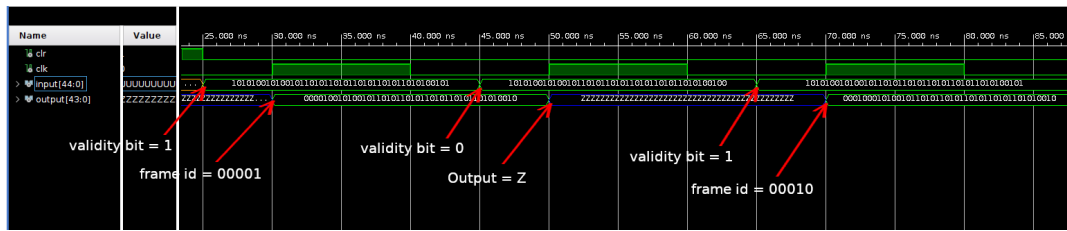


Figure 5.13: Testbench for the Frame ID Generator (Output Layer), file: mil/fid_gen_tb.vhd

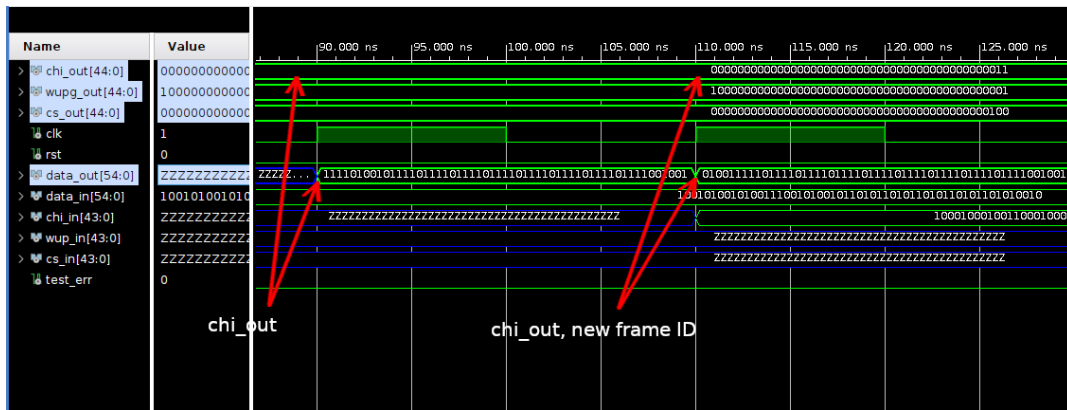


Figure 5.14: Testbench for the MIL (Output Layer), file: mil/mil_main_tb.vhd

Verification of Figure 5.14

We see that the data frame from the Controller Host Interface appears at the output, since it has a higher priority than the frame from the Wakeup Pattern Generator. The frame from Clock Synchronization is ignored, because its validity bit (the last one) is 0.

Before the frame is encoded, the frame id is prepended. It is 00001, since this is the first frame to be sent. The content of the frame is therefore now:

00001|00000000...0001

We see that the validity bit has been removed. Using the 4B/5B encoding (Table D.1), the contents of the frame are transformed as follows:

Grouped into 4-bit blocks:

0000 1000 0000 ... 0001

Each 4-bit block mapped to a 5-bit block:

11110 10010 11110 11110 ... 01001

This bit string matches the one in Figure 5.14, which confirms that the circuit operates correctly.

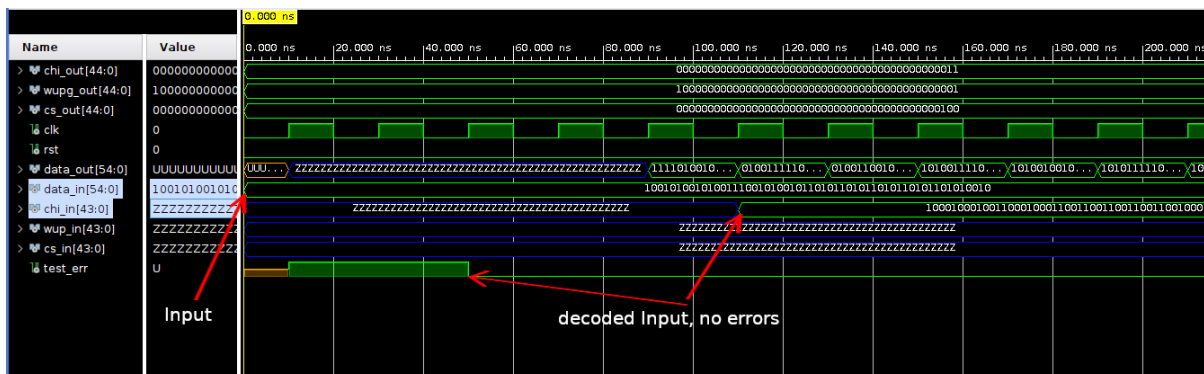


Figure 5.15: Testbench #1 for the MIL (Input Layer), file: mil/mil_main_tb.vhd

Before the WUPG starts, the S-R Flip-Flop (top left in Figure 5.17) must be initialised. This is done by setting the `rst_wire` signal to 1 through the multiplexer on the left. From then on, the output of the AND gate is used for as long as the component operates.

When the `wupg_en` signal is set to 1, the WUPG starts operating in the next clock cycle. The symbols are read from a ROM using a 2-bit counter. Since the counter is 2 bits wide, 4 symbols are read in total.

The Flip-Flops DFF5, DFF4 and DFF2 at the bottom of the figure are used to stop the WUPG. When the counter reaches its last value (11), the `rst_wire` signal becomes 1 three clock cycles later. In the next clock cycle the `ctrl_write` signal therefore becomes 0, disabling the counter and the ROM. The Flip-Flop DFF3 is used to allow the last symbol to be sent.

At the top right of Figure 5.17 we also see that the AND gate takes the validity bit from the ROM as an input, in addition to the output of DFF3. In this way DFF1 prevents invalid data frames from being sent.

The data (the symbol) is also framed inside the Wakeup Pattern Generator. This is done with the ‘Additional signals for framing the data’, which are concatenated with the data at the input of DFF1. The figure below shows the structure of the final frame and where its fields come from.

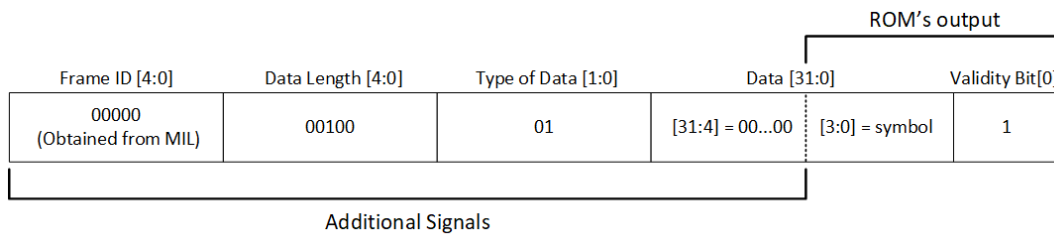


Figure 5.18: Data frame of the Wakeup Pattern Generator

5.6.1 Testbench

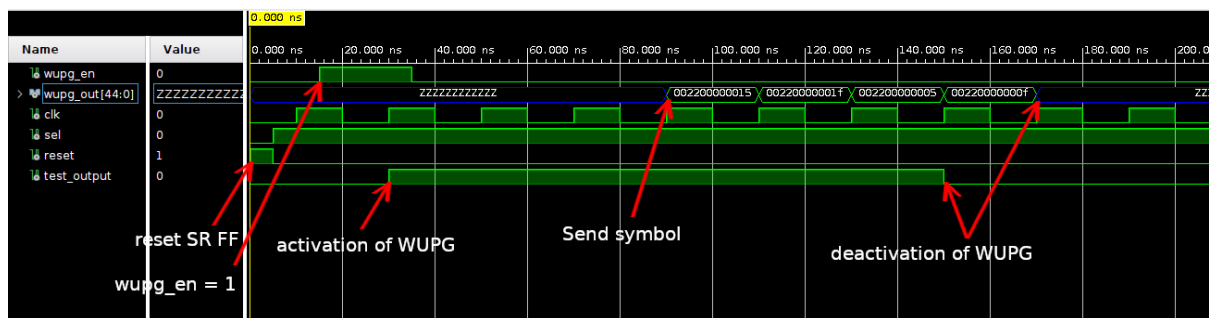


Figure 5.19: Testbench of the Wakeup Pattern Generator, file: `wupg/wupg_main_tb.vhd`

5.7 Wakeup Pattern Recognition

The Wakeup Pattern Recognition (WUPR) recognises a predefined sequence of symbols. If the symbol sequence is the correct one, the node moves from the sleep state to the wakeup state. The architecture of the WUPR is shown in the figure below.

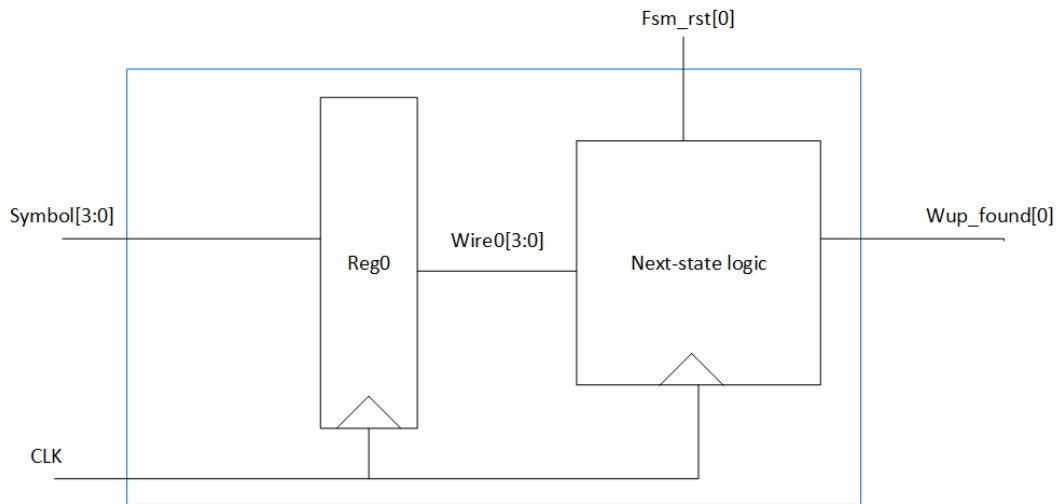


Figure 5.20: Architecture of the Wakeup Pattern Recognition

In Figure 5.20, Reg0 is a register, while the Next-state logic contains a finite state machine that recognises the pattern (or symbol sequence).

The symbol sequence to be recognised is taken to be 0xa, 0xf, 0x2, 0x7 (the hexadecimal form of the numbers 10, 15, 2 and 7 respectively). The state table of the FSM follows, giving the next state, the conditions required, and the output for each transition.

Current State	Input: Symbol	Next State	Output: wup_found
m0	a / not a	m1/m0	0/0
m1	f / not f	m2/m0	0/0
m2	2 / not 2	m3/m0	0/0
m3	7 / not 7	m4/m0	1/0
m4	X	m0	0

Table 5.5: State table of the FSM in the WUPR

As Table 5.5 shows, the slash separates alternative inputs, next states or outputs. For example, if the symbol received in state m0 is a, the machine moves to state m1 and the output is 0. For any other input it stays in state m0, and the output is again 0. When the whole sequence has been recognised, the output is 1. 'X' stands for don't care, meaning that the state changes in the next clock cycle regardless. This is shown more clearly in the state diagram below.

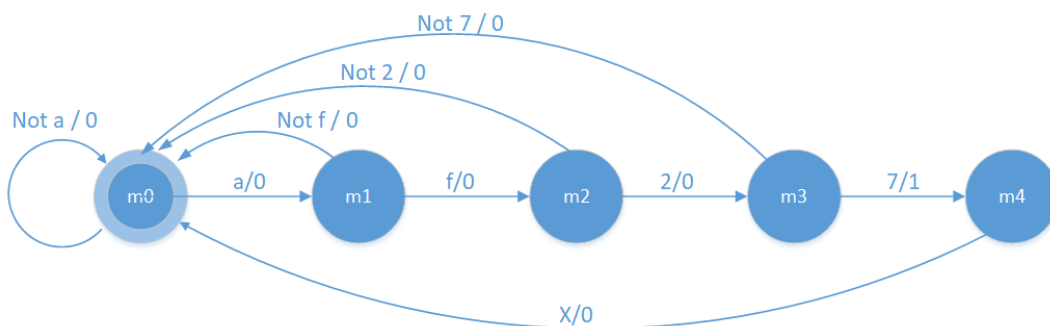


Figure 5.21: State diagram of the FSM in the WUPR

5.7.1 Testbench

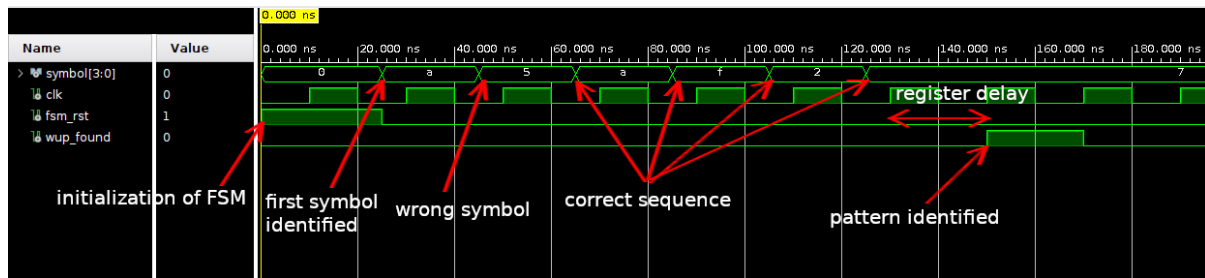


Figure 5.22: Testbench of the WUPR, file: wup_recognition/wupr_main_tb.vhd

5.8 Clock Synchronization

Clock Synchronization is responsible for synchronising the node with the rest of the network. A submodule that checks synchronisation was created, but no clock correction mechanism was designed. Whether the node is synchronised with the rest of the network is checked by the *Microtick Counter* submodule, so this section is devoted to that submodule.

According to the paper [3], Microticks and Macrocks relate to the alignment of the local and the global clock respectively. For the present design it is assumed that the Microticks are the number of local clock cycles that should correspond to one Macrocks, that is, to one cycle of the global clock. If the correspondence is correct, the output of the circuit is 1, and otherwise 0.

The figure below shows the architecture of the Microtick Counter submodule.

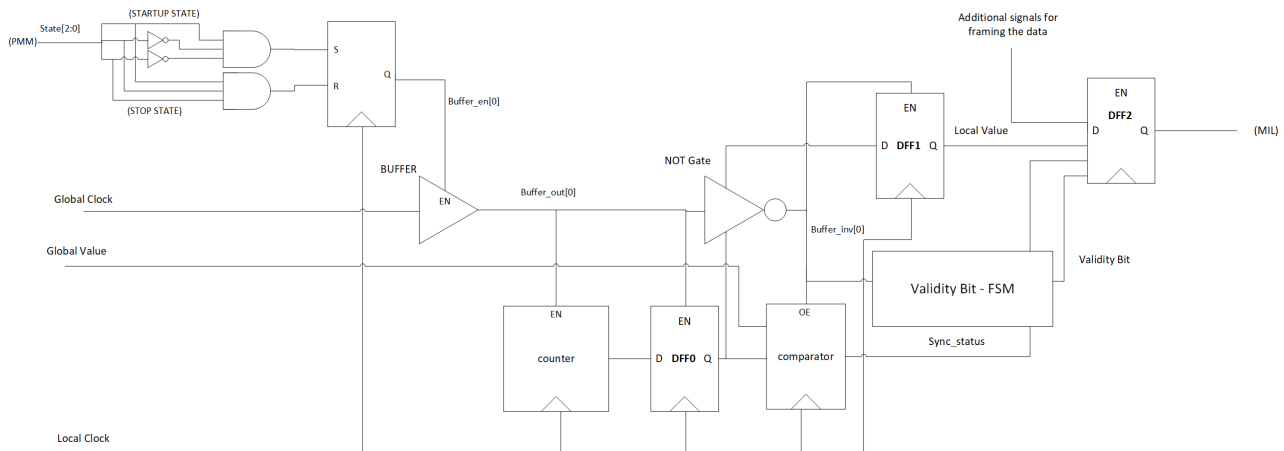


Figure 5.23: Architecture of the Microtick Counter

5.8.1 Principle of Operation

Figure 5.23 shows that the Microtick Counter is enabled in the STARTUP operating state, where the S-R Flip-Flop is set and its output goes to 1. When the STOP operating state is detected, the Microtick Counter stops, since the S-R Flip-Flop is reset and its output therefore goes to 0.

The output of the S-R FF drives the enable signal of a buffer that lets the global clock signal through. When the global clock signal goes to 1, a counter is enabled. The counter increments on every rising edge of the local clock (Microtick), for as long as the global clock signal stays at 1 (Macrocks).

On the falling edge of the global clock, the output of the buffer goes to 0 and the counter is disabled. Thanks to the NOT gate, a comparator is also enabled, which checks whether the counted number of Microticks matches the correct one (Global Value in Figure 5.23). The correct number of

Microticks could be supplied by a network arbiter. If the values match, the output of the comparator is 1, and otherwise 0.

In the next clock cycle, DFF2 lets the data through to the output. The data consist of the synchronisation status of the node (the output of the comparator) and the local value, that is, the counted number of Microticks (the output of DFF1).

5.8.2 Data Framing

Some additional signals are used to send the data inside a frame. They appear at the top right of Figure 5.23 as ‘Additional signals for framing the data’, and are added to the inputs of register DFF2. The figure below shows the structure of the output frame.

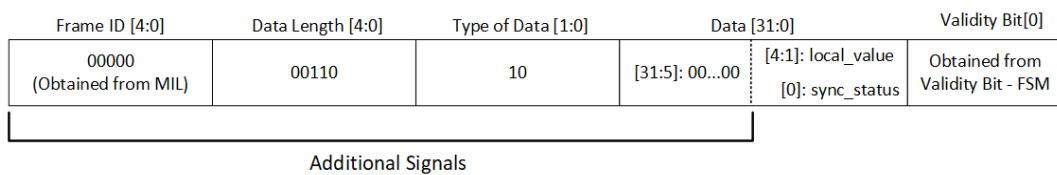


Figure 5.24: Data frame of Clock Synchronization

As Figures 5.23 and 5.24 show, the validity bit comes from a finite state machine, the ‘Validity Bit FSM’. This machine is needed so that the data are sent only in the first clock cycle after the Microtick count has finished. The end of the count is signalled by the falling edge of the global clock signal. Figure 5.25 shows the state diagram of this FSM.

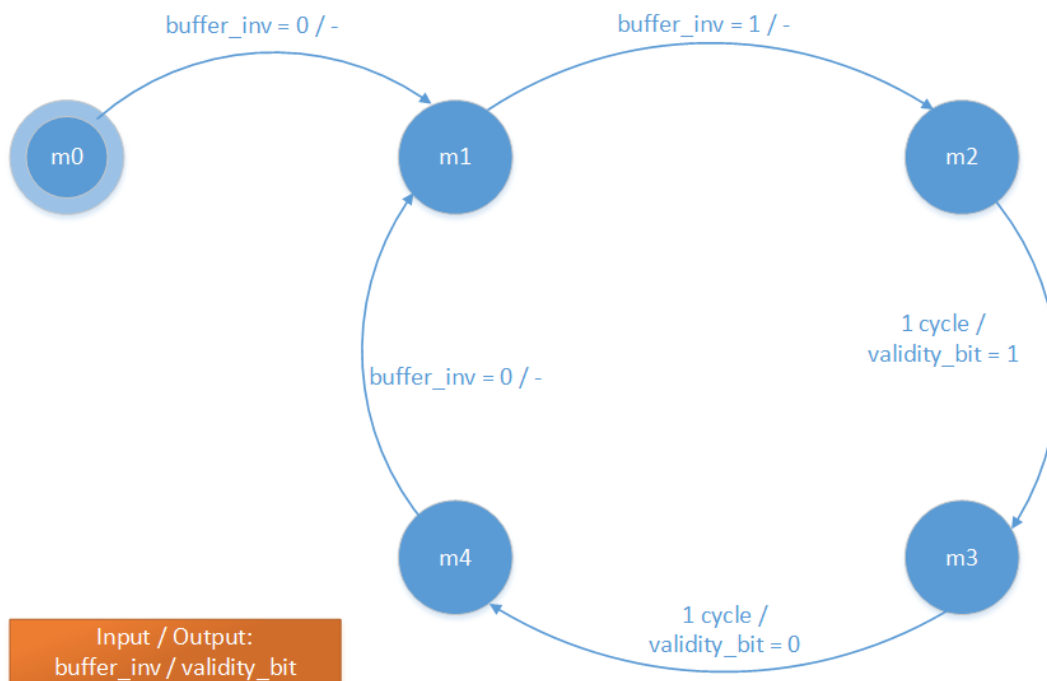


Figure 5.25: Finite state machine for the validity bit

m0: Initial state, waiting for the Microtick Counter to be enabled.

m1: The Microtick Counter has been enabled, since `buffer_inv = 0` (the output of the NOT gate in Figure 5.23). The Microticks are being counted.

m2: The Microtick count is complete, since `buffer_inv = 1`. One more clock cycle is needed for the data to appear at the output of DFF1.

m3: The validity bit has been set to 1, since the data are available to send.

m4: The validity bit goes to 0, since the data were sent in the previous clock cycle.

m1: The counting process has restarted, since `buffer_inv = 0`.

5.8.3 Testbench

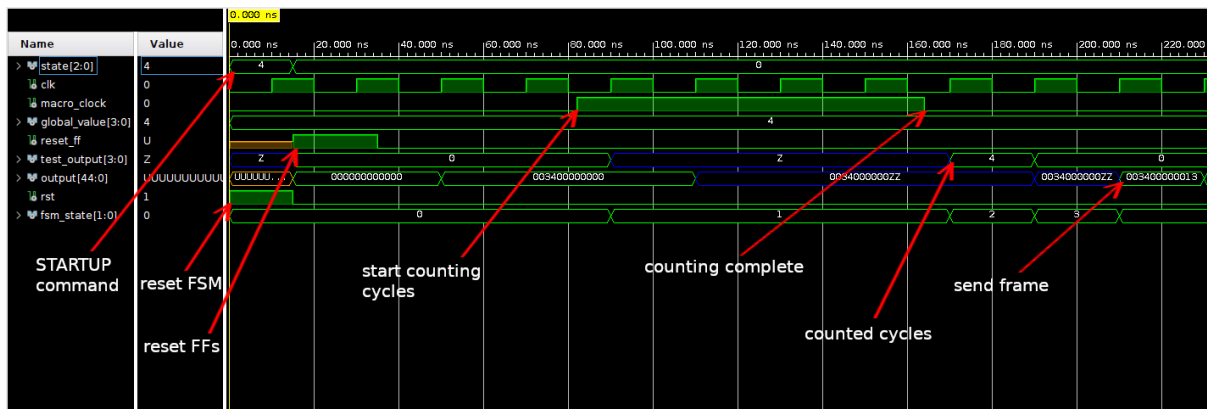


Figure 5.26: Testbench #1 of the Microtick Counter, file: `microtick_counter_tb.vhd`

From the figure above we see that the content of the data frame is: `0x03400000013`

The leading digit (0) has been added by the simulation tool. The last hexadecimal digit is 3, that is, `0011` in binary. The frame is therefore valid, since the last bit (the validity bit) is 1.

Converting the contents of the frame to binary gives:

`0000 0011 0100 0000...0000 0001 0011`

In Figure 5.26, `global_value` represents the correct number of local clock cycles (Microticks).

According to the figure, the total number of cycles counted is 100 in binary, that is, four (= `global_value`). We also see that the second-to-last bit (`sync_status`) is 1, which means the node is synchronised.

Changing the value of `global_value` to 5 gives the following results.

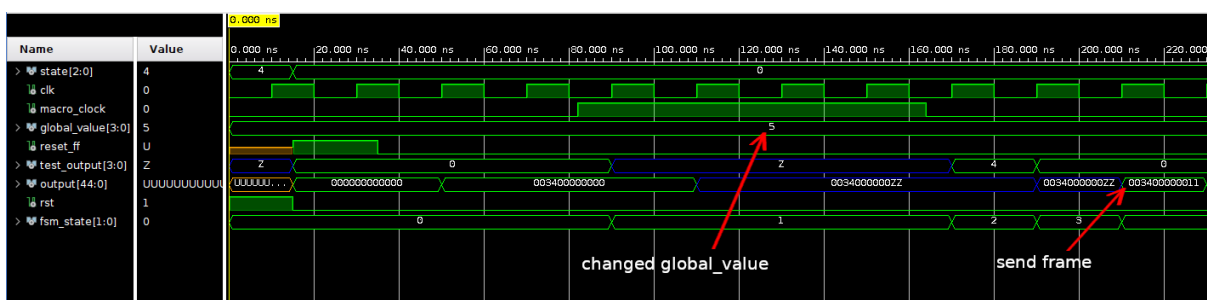


Figure 5.27: Testbench #2 of the Microtick Counter (changed `global_value`), file: `microtick_counter_tb.vhd`

In this case the content of the data frame is:

`0x034000000011 = 0000 0011 0100 0000...0000 0001 0001`

The number of cycles counted is again 4 ($\neq$ `global_value`), but the second-to-last bit is now 0 (`sync_status`), because the count disagrees with `global_value`.

5.9 System Testing

In this section a testbench is run to confirm that the complete system, obtained by integrating the submodules described in the previous sections, operates correctly.

To confirm that the system operates correctly, its signals have to be observed during the transitions between different operating states. The current operating state of the system is set by the finite state machine in the Protocol Management Module, which was presented in Subsection 5.3.2.

The figure below shows the diagram of the FSM in the PMM. The execution flow targeted by the testbench is highlighted in red.

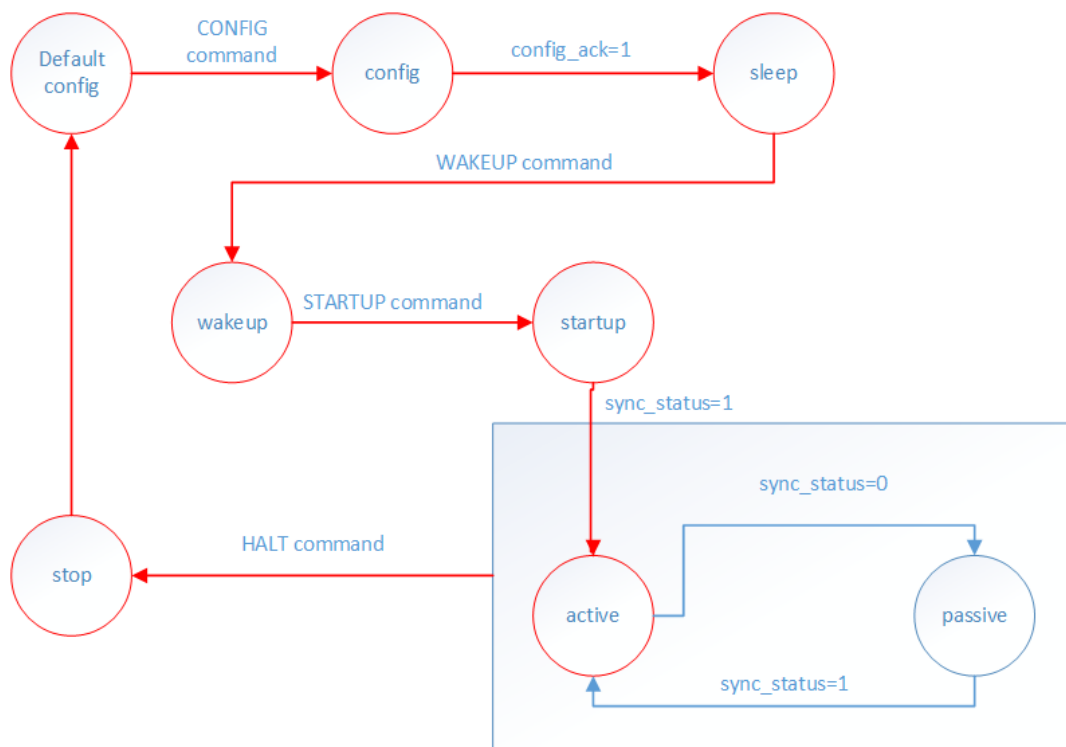


Figure 5.28: Execution flow during system testing

Figure 5.30: System testbench (2/5)

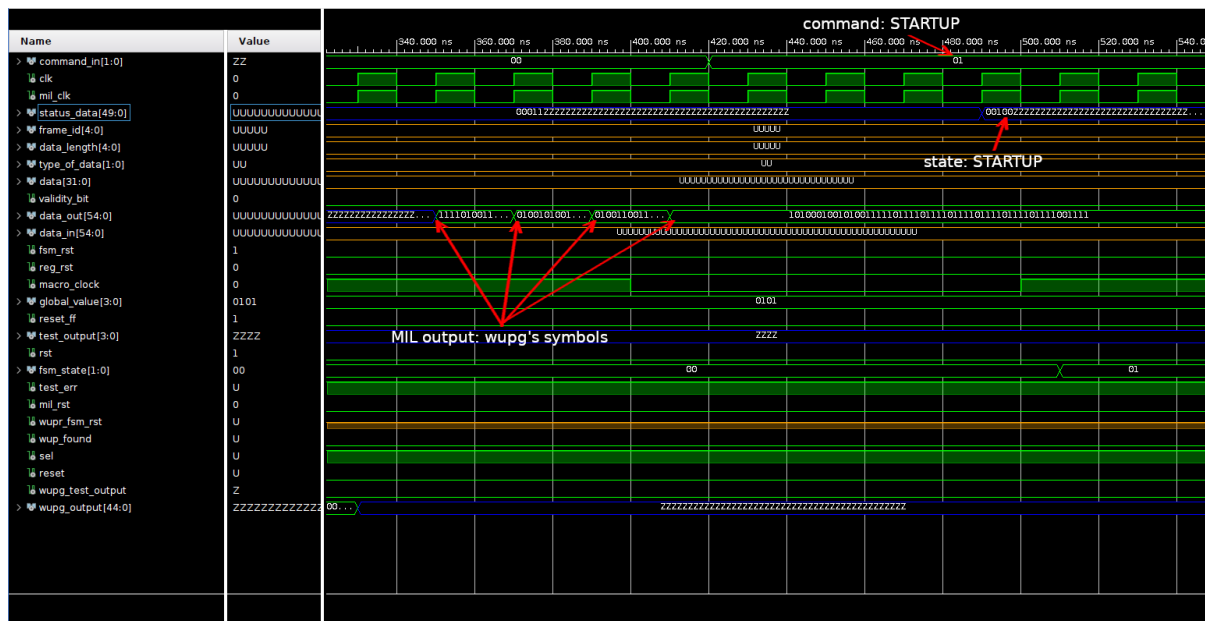


Figure 5.31: System testbench (3/5)

In Figure 5.31 we see that, after a few clock cycles, the symbols produced by the Wakeup Pattern Generator have appeared at the outputs of the Medium Interface Layer (the data_out signal), in frames that have been encoded.

The contents of the last frame are:

10100|01001|01001|11110|11110|11110|11110|11110|11110|11110|01111

Decoding them gives:

0010|0001|0001|0000|0000|0000|0000|0000|0000|0000|0111

Recall that the structure of the WUPG frame is as follows:

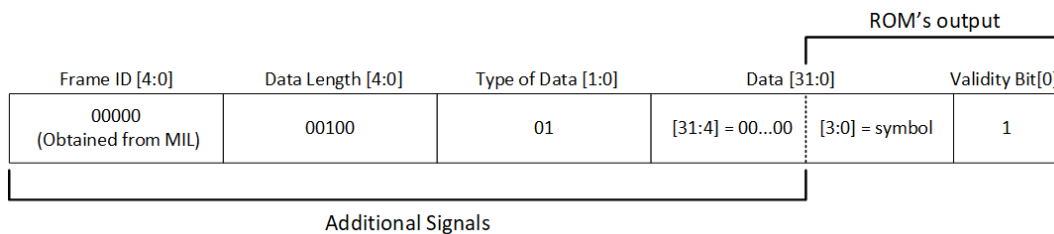


Figure 5.32: Data frame of the Wakeup Pattern Generator

It remains to confirm that the contents of the data frame are correct. Mapping the contents of the last frame onto the fields in the figure above gives:

- Frame ID = 00100 = 4, meaning this is the 4th data frame sent by the MIL.
- Data Length = 00100 = 4, meaning the data are 4 bits long.
- Type of Data = 01, meaning the data are indeed one of the wakeup symbols.
- Data[31:4] = 00...00
Data[3:0] = 0111 = 7, which is indeed the last wakeup symbol, as expected.
- The validity bit has been removed in the MIL, because it only matters locally.

It follows that the symbols appear, successfully encoded, at the outputs of the Medium Interface Layer, ready to be sent to the rest of the network.

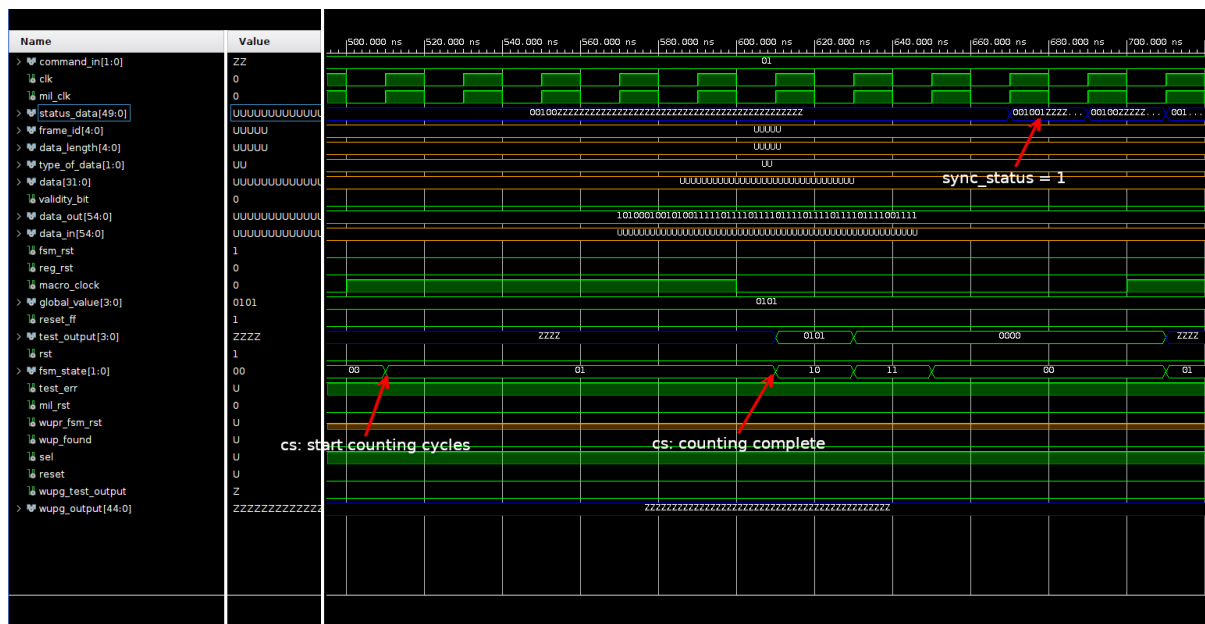


Figure 5.33: System testbench (4/5)

In Figure 5.31, the system entered the STARTUP state. This means that a synchronisation check with the network should now follow. In Figure 5.33, counting of the local clock cycles begins, and 5 are counted in total. We see that the `global_value` signal has the value `0101 = 5`. Since the two values match, synchronisation succeeds, and the `sync_status` signal (the 6th bit of the `status_data` signal) is set to 1.

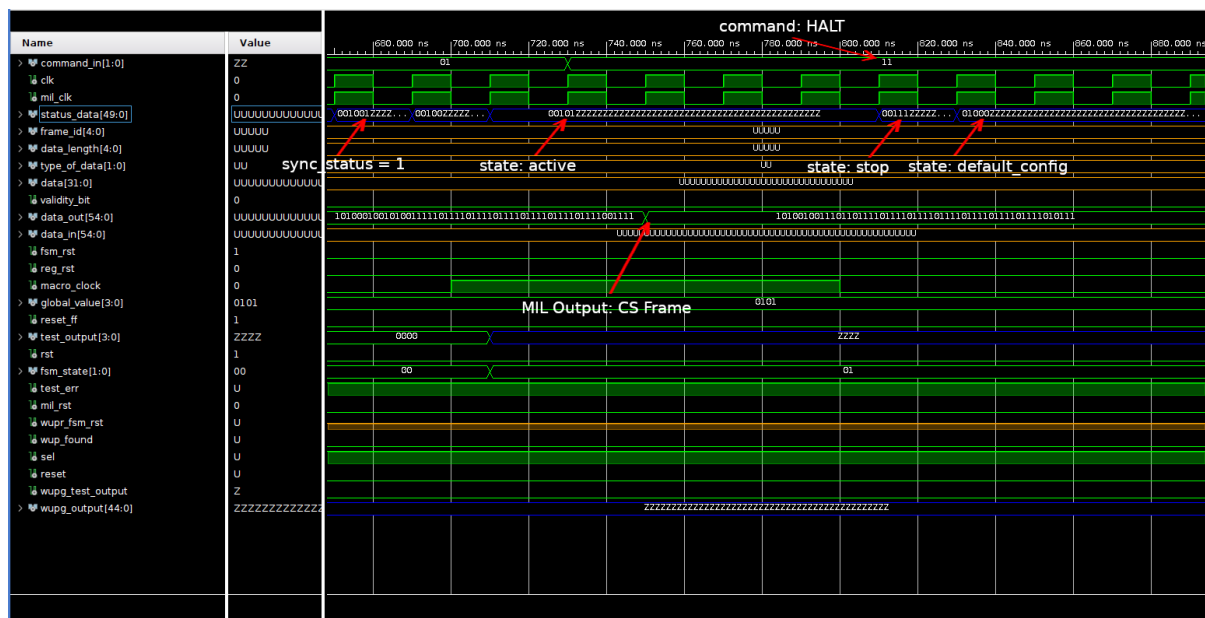


Figure 5.34: System testbench (5/5)

Since synchronisation with the network succeeded, the system moves to the active state, as Figure 5.34 shows.

In the same figure we also see that the data frame from the Clock Synchronization submodule appears at the outputs of the Medium Interface Layer.

The contents of the data frame are:

10100|10011|10110|11110|11110|11110|11110|11110|11110|11110|10111

Decoding them gives:

0010|1001|1010|0000|0000|0000|0000|0000|0000|0000|1011

Recall that the structure of the CS data frame is as follows:

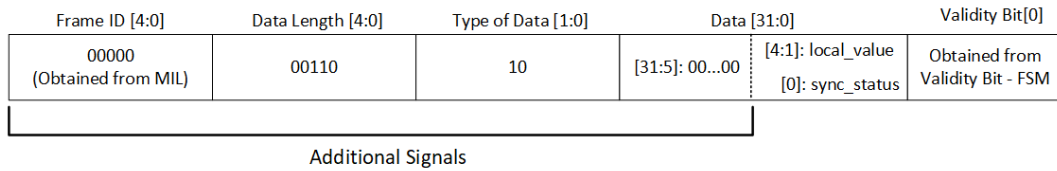


Figure 5.35: Data frame of Clock Synchronization

It remains to confirm that the contents of the data frame are correct. Mapping the contents onto the fields in the figure above gives:

- Frame ID = 00101 = 5, meaning this is the 5th data frame sent by the MIL.
- Data Length = 00110 = 6, meaning the data are 6 bits long.
- Type of Data = 10, meaning the data do indeed concern the Clock Synchronization submodule.
- Data[31:5] = 00...00
Data[4:1] = 0101 = 5, the number of clock cycles counted.
Data[0] = 1, meaning synchronisation succeeded.
- The validity bit has been removed in the MIL, because it only matters locally.

This confirms that the data from the Clock Synchronization submodule are sent correctly.

Finally, as Figure 5.34 shows, issuing the HALT command moves the Communication Controller to the stop state, and then to the default_config state so that it is reset.

5.10 System Synthesis

5.10.1 General Information

Synthesis is one of the stages of the design flow for an FPGA. The design flow (for RTL, or Register-Transfer Level, descriptions) consists of the following steps:

1. Logic Synthesis
2. Technology Mapping
3. Place and Route
4. Generating Configuration Data
5. FPGA Programming
6. Execution on the FPGA

Synthesis in the tool used (Xilinx Vivado) performs the first two steps of the design flow.

Logic Synthesis is the process that creates the logic and sequential circuitry from an RTL description (a VHDL description, for example). The result of synthesis is known as a ‘netlist’, which consists of a set of logic elements, such as logic gates and Flip-Flops, and the interconnections between them.

Technology Mapping is the process in which the logic elements of the netlist are assigned to actual logic elements of the FPGA.[6]

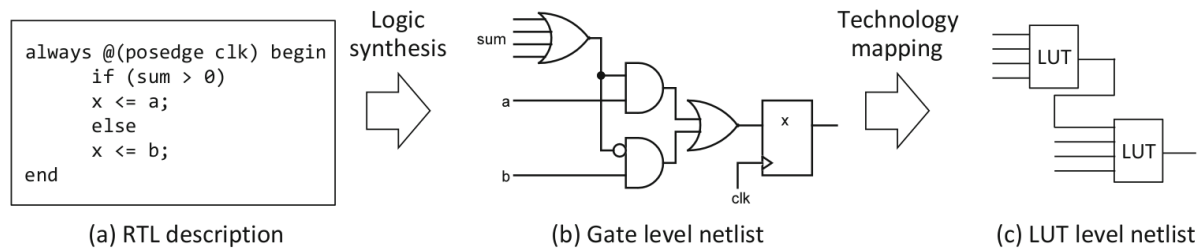


Figure 5.36: Logic synthesis & technology mapping from an RTL description

5.10.2 Synthesis Results

The Artix-7 FPGA xc7a15tfgg484-3 was used for synthesis, chosen with a view to optimising resource utilisation. The synthesis results follow.

Component	Slice LUTs	Slice Registers	Bonded IOB	BUFGCTRL
Clock Synchronization	52	22	0	0
Medium Interface Layer	50	364	0	0
Protocol Management Module	7	9	0	0
Wakeup Pattern Generator	5	11	0	0
System (Total)	114	406	228	2

Table 5.6: Resource utilisation by hierarchy

Resource	Utilization	Available	Utilization (%)
LUT	114	10400	1.10
FF	406	20800	1.95
IO	228	250	91.20

Table 5.7: Resource utilisation summary

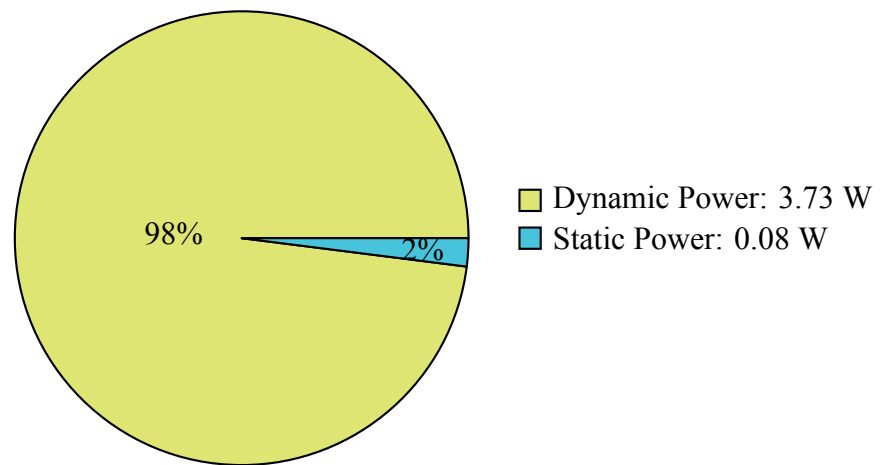


Figure 5.37: Power consumption

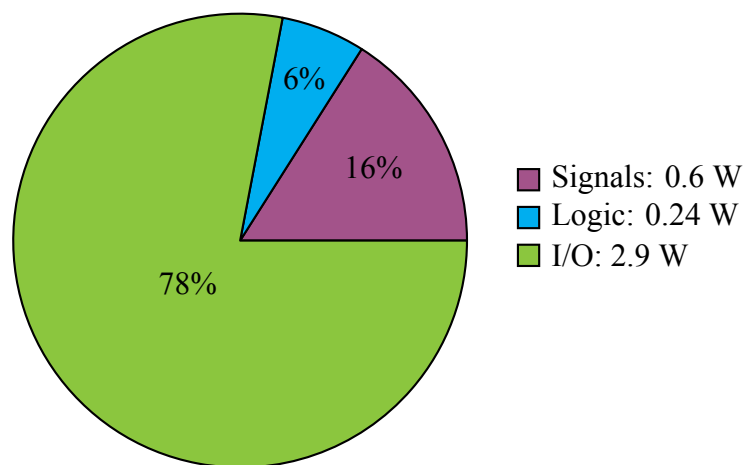


Figure 5.38: Dynamic power consumption

6. Conclusions & Future Work

6.1 Conclusions

The rapid progress of technology has led to embedded systems being used in many fields, and therefore to people depending on them more and more. The need to add new applications, especially real-time and safety-critical ones, calls for these embedded systems to adopt different operating protocols.

This can be seen in cars, which use numerous embedded systems (Electronic Control Units, or ECUs) for a variety of applications. It has become clear that a more advanced communication protocol with higher data rates, such as FlexRay, needs to be adopted. Deploying it is a challenge, however, because it leads to higher energy and bandwidth consumption.

Extending the capabilities of the Communication Controller in a network node (ECU) could solve this problem. It would improve the quality and efficiency of the system, and at the same time allow more functions to be integrated. Chapter 4 presented such an approach in detail, together with experimental results showing that the proposal is sound.

In this thesis, a Communication Controller was designed and simulated according to the descriptions above. The additional submodules inside the Communication Controller give it extra functionality that would normally be carried out on the Host/Microcontroller. Their functionality was described, possible architectures for them were presented, and each was simulated to confirm that it operates correctly. Finally, the submodules were integrated into the complete system, whose correct operation was confirmed with a suitable testbench.

6.2 Future Work

The first priority for continuing this work is to simulate the functionality of this Communication Controller in a virtual network, in order to measure its efficiency. A similar Communication Controller with fewer functions could then be developed, with the missing functions carried out by the Host. The efficiency and energy consumption of the two cases could then be compared, demonstrating the effectiveness of the proposed solution.

Further improvements could also be made to the architectures and functionality of the submodules presented in this thesis.

For example, the synchronisation check of a node in the active or the passive state could be carried out over a different number of clock cycles, depending on the application running on that node. This could save energy, since some applications are not particularly critical and their nodes therefore do not need to be synchronised immediately.

In the Medium Interface Layer (Section 5.5), once an error has been detected in a data frame, a notification (an ACK message) could also be sent back to the sender to report the error, so that the frame is retransmitted.

Finally, the architectures could be revised for lower energy consumption, higher processing speed and minimum area, based on the chosen FPGA.

Appendices

A. Source Code

This appendix presents the code written for this thesis. Besides the circuit descriptions, there are corresponding testbenches for testing them (file names ending in `_tb`). Section A.7 contains the code of the system, which integrates all the submodules of the Communication Controller.

Some components (`dff.vhd`, for example) are listed under only one section but may also be used by other modules. The ‘main’ files describe the complete system of each submodule, since they integrate its other components.

A.1 Protocol Management Module

fsm.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY fsm IS
    PORT (command      : IN std_logic_vector(1 DOWNTO 0);
          sync_status  : IN std_logic;
          clk, rst      : IN std_logic;

          curr_state    : OUT std_logic_vector(2 DOWNTO 0);
          reset_ack     : OUT std_logic;
          wup_en        : OUT std_logic;
          config_ack    : OUT std_logic);
END fsm;

-- the architecture follows design style #2 from Pedroni's book
ARCHITECTURE fsm_default OF fsm IS
    TYPE states IS (default_config, config, sleep, wakeup, startup, active, passive, stop);
    SIGNAL prv_state, nxt_state : states;
    SIGNAL temp_reset_ack, temp_wup_en, temp_config_ack : std_logic;
BEGIN

    PROCESS (rst, clk)
    BEGIN
        IF (rst = '1') THEN
            curr_state <= std_logic_vector(to_unsigned(states'pos(default_config), 3));
            prv_state <= default_config;
        ELSIF (clk'event AND clk = '1') THEN
            curr_state <= std_logic_vector(to_unsigned(states'pos(nxt_state), 3));
            reset_ack <= temp_reset_ack;
            wup_en <= temp_wup_en;
            config_ack <= temp_config_ack;
            prv_state <= nxt_state;
        END IF;
    END PROCESS;

    PROCESS (prv_state, command, sync_status)
    BEGIN
        CASE prv_state IS
            WHEN default_config =>
                IF (command = "10") THEN
                    temp_reset_ack <= '0';
```

```

        nxt_state      <= config;
    END IF;

    WHEN config =>
        temp_config_ack <= '1';
        nxt_state      <= sleep;

    WHEN sleep =>
        IF (command = "00") THEN
            temp_config_ack <= '0';
            temp_wup_en     <= '1';
            nxt_state      <= wakeup;
        END IF;

    WHEN wakeup =>
        temp_wup_en <= '0'; -- so that it stays at 1 for only 1 clock cycle
        IF (command = "01") THEN
            nxt_state <= startup;
        END IF;

    WHEN startup =>
        IF (sync_status = '1') THEN
            nxt_state <= active;
        END IF;

    WHEN active =>
        IF (command = "11") THEN
            nxt_state <= stop;
        ELSIF (sync_status = '0') THEN
            nxt_state <= passive;
        END IF;

    WHEN passive =>
        IF (command = "11") THEN
            nxt_state <= stop;
        ELSIF (sync_status = '1') THEN
            nxt_state <= active;
        END IF;

    WHEN stop =>
        temp_reset_ack <= '1';
        nxt_state      <= default_config;

    END CASE;
END PROCESS;

END fsm_default;

```

fsm_tb.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY testbench IS
END testbench;

ARCHITECTURE tb OF testbench IS
    SIGNAL command      : std_logic_vector(1 DOWNTO 0);
    SIGNAL sync_status, rst : std_logic;
    SIGNAL clk          : std_logic := '0';

    SIGNAL curr_state    : std_logic_vector(2 DOWNTO 0);
    SIGNAL reset_ack, wup_en, config_ack : std_logic;
BEGIN
    fsm : ENTITY work.fsm PORT MAP (command => command, sync_status => sync_status, clk => clk, rst =>
        rst, curr_state => curr_state, reset_ack => reset_ack, wup_en => wup_en, config_ack =>
        config_ack);

    clk <= NOT clk AFTER 10 ns;

    PROCESS
    BEGIN
        sync_status <= '0';
    END PROCESS;

```

```

    rst      <= '1';
    WAIT FOR 25 ns;

    rst      <= '0';
    command <= "10";
    WAIT FOR 40 ns;

    command <= "00";
    WAIT FOR 30 ns;

    command <= "01";
    WAIT FOR 25 ns;

    sync_status <= '1';
    WAIT FOR 25 ns;

    sync_status <= '0';
    WAIT FOR 25 ns;

    command <= "11";
    WAIT FOR 80 ns;

    END PROCESS;

END tb;

```

dff.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY dff IS
    GENERIC (d_len : integer := 1;
             q_len : integer := 1);
    PORT (d      : IN  std_logic_vector(d_len-1 DOWNTO 0);
          q      : OUT std_logic_vector(q_len-1 DOWNTO 0);
          clk, rst, en : IN  std_logic);
END dff;

ARCHITECTURE default OF dff IS
    -- architecture with asynchronous reset
    BEGIN

    PROCESS(rst, clk)
    BEGIN
        CASE en IS
            WHEN '1' =>
                CASE rst IS
                    WHEN '1' => q <= (OTHERS => '0');
                    WHEN '0' =>
                        IF (clk'event AND clk = '1') THEN
                            q <= d;
                        END IF;
                    WHEN OTHERS => NULL;
                END CASE;
            WHEN OTHERS =>
                q <= (OTHERS => 'Z');
            END CASE;
        END PROCESS;

    END default;

```

pmm.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY pmm IS

```

```

PORT (command      : IN std_logic_vector(1 DOWNTO 0);
      sync_status   : IN std_logic;
      clk, fsm_rst, reg_rst : IN std_logic;

      curr_state : OUT std_logic_vector(2 DOWNTO 0);
      reset_ack  : OUT std_logic;
      wup_en     : OUT std_logic;
      config_ack : OUT std_logic);
END pmm;

ARCHITECTURE pmm_arch OF pmm IS
  COMPONENT dff IS
    GENERIC (d_len : integer := 1;
             q_len : integer := 1);
    PORT (d      : IN std_logic_vector(d_len-1 DOWNTO 0);
          q      : OUT std_logic_vector(q_len-1 DOWNTO 0);
          clk, rst, en : IN std_logic);
  END COMPONENT;
  COMPONENT fsm IS
    PORT (command      : IN std_logic_vector(1 DOWNTO 0);
          sync_status   : IN std_logic;
          clk, rst      : IN std_logic;

          curr_state : OUT std_logic_vector(2 DOWNTO 0);
          reset_ack  : OUT std_logic;
          wup_en     : OUT std_logic;
          config_ack : OUT std_logic);
  END COMPONENT;

  SIGNAL wire0 : std_logic_vector(2 DOWNTO 0);
BEGIN
  reg0 : dff GENERIC MAP (d_len => 3, q_len => 3) PORT MAP (d(2 DOWNTO 1) => command, d(0) =>
    sync_status, q => wire0, clk => clk, rst => reg_rst, en => '1');

  fsm0 : fsm PORT MAP (command => wire0(2 DOWNTO 1), sync_status => wire0(0), clk => clk, rst =>
    fsm_rst, curr_state => curr_state, reset_ack => reset_ack, wup_en => wup_en, config_ack =>
    config_ack);

END pmm_arch;

```

pmm_tb.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY pmm_testbench IS
END pmm_testbench;

ARCHITECTURE pmm_tb OF pmm_testbench IS
  SIGNAL command      : std_logic_vector(1 DOWNTO 0);
  SIGNAL sync_status   : std_logic;
  SIGNAL fsm_rst, reg_rst : std_logic;
  SIGNAL clk           : std_logic := '0';

  SIGNAL curr_state : std_logic_vector(2 DOWNTO 0);
  SIGNAL reset_ack  : std_logic;
  SIGNAL wup_en     : std_logic;
  SIGNAL config_ack : std_logic;
BEGIN
  pmm : ENTITY work.pmm PORT MAP (command => command, sync_status => sync_status, clk => clk,
    fsm_rst => fsm_rst, reg_rst => reg_rst, curr_state => curr_state, reset_ack => reset_ack, wup_en
    => wup_en, config_ack => config_ack);

  clk      <= NOT clk AFTER 10 ns;
  reg_rst <= '0';

  PROCESS
  BEGIN
    sync_status <= '0';
    fsm_rst     <= '1';
  END PROCESS

```

```

    WAIT FOR 25 ns;

    fsm_rst <= '0';
    command <= "10";
    WAIT FOR 80 ns;

    END PROCESS;

END pmm_tb;

```

A.2 Controller Host Interface

chi.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY chi IS
    PORT (
        -- input, direction: micro => communication
        command_in : IN std_logic_vector(1 DOWNTO 0);
        --corresponding output
        command_out : OUT std_logic_vector(1 DOWNTO 0);
        -- input, origin: micro
        clk : IN std_logic;
        -- inputs, direction: communication => micro
        config_ack : IN std_logic;
        reset_ack : IN std_logic;
        state : IN std_logic_vector(2 DOWNTO 0);
        sync_status : IN std_logic;
        frame_rcv : IN std_logic_vector(43 DOWNTO 0);
        -- corresponding output
        status_data : OUT std_logic_vector(49 DOWNTO 0);
        -- inputs, direction: micro => communication
        frame_id : IN std_logic_vector(4 DOWNTO 0);
        data_length : IN std_logic_vector(4 DOWNTO 0);
        type_of_data : IN std_logic_vector(1 DOWNTO 0);
        data : IN std_logic_vector(31 DOWNTO 0);
        validity_bit : IN std_logic;
        -- corresponding output
        frame_send : OUT std_logic_vector(44 DOWNTO 0));
END chi;

ARCHITECTURE chi_interface OF chi IS
    COMPONENT dff IS
        GENERIC (d_len : integer := 1;
                 q_len : integer := 1);
        PORT (d : IN std_logic_vector(d_len-1 DOWNTO 0);
              q : OUT std_logic_vector(q_len-1 DOWNTO 0);
              clk, rst, en : IN std_logic);
    END COMPONENT;

    signal temp : std_logic_vector(4 downto 0);
BEGIN
    temp <= config_ack & reset_ack & state;

    reg0 : dff GENERIC MAP (d_len => 2, q_len => 2) PORT MAP (d => command_in, q => command_out, clk
    => clk, rst => '0', en => '1');

    reg1 : dff GENERIC MAP (d_len => 50, q_len => 50) PORT MAP (d(49 DOWNTO 45) => temp, d(44) =>
    sync_status, d(43 DOWNTO 0) => frame_rcv, q => status_data, clk => clk, rst => '0', en => '1');

    reg2 : dff GENERIC MAP (d_len => 45, q_len => 45) PORT MAP (d(44 DOWNTO 40) => frame_id, d(39
    DOWNTO 35) => data_length, d(34 DOWNTO 33) => type_of_data, d(32 DOWNTO 1) => data, d(0) =>
    validity_bit, q => frame_send, clk => clk, rst => '0', en => '1');

```

```
END chi_interface;
```

main_tb.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY testbench IS
END testbench;

ARCHITECTURE tb OF testbench IS
    SIGNAL command_in  : std_logic_vector(1 DOWNTO 0);
    SIGNAL command_out : std_logic_vector(1 DOWNTO 0);

    SIGNAL clk : std_logic := '0';

    SIGNAL config_ack, reset_ack, sync_status : std_logic;
    SIGNAL state : std_logic_vector(2 DOWNTO 0);
    SIGNAL frame_rcv : std_logic_vector(43 DOWNTO 0);

    SIGNAL status_data : std_logic_vector(49 DOWNTO 0);

    SIGNAL frame_id, data_length : std_logic_vector(4 DOWNTO 0);
    SIGNAL type_of_data : std_logic_vector(1 DOWNTO 0);
    SIGNAL data : std_logic_vector(31 DOWNTO 0);
    SIGNAL validity_bit : std_logic;

    SIGNAL frame_send : std_logic_vector(44 DOWNTO 0);

BEGIN
    interface : ENTITY work.chi PORT MAP (command_in => command_in, command_out => command_out, clk =>
        clk, config_ack => config_ack, reset_ack => reset_ack, sync_status => sync_status, state =>
        state, frame_rcv => frame_rcv, status_data => status_data, frame_id => frame_id, data_length =>
        data_length, type_of_data => type_of_data, data => data, validity_bit => validity_bit,
        frame_send => frame_send);

    clk <= NOT clk AFTER 10 ns;

    PROCESS
    BEGIN
        -- reg0
        command_in <= "10";
        WAIT FOR 25 ns;
        -- reg1
        config_ack <= '1';
        reset_ack <= '0';
        state <= "110";
        sync_status <= '1';
        frame_rcv <= (7 => '0', OTHERS => '1');
        WAIT FOR 25 ns;
        -- reg2
        frame_id <= (OTHERS => '1');
        data_length <= (OTHERS => '1');
        type_of_data <= (OTHERS => '1');
        data <= (OTHERS => '1');
        validity_bit <= '0';
        WAIT FOR 25 ns;

    END PROCESS;

END tb;
```

A.3 Medium Interface Layer

comparator.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE work.encoding.ALL;

ENTITY comparator IS
  PORT (input      : IN  std_logic_vector(4 DOWNTO 0);
        output     : OUT std_logic_vector(4 DOWNTO 0);
        en, clk     : IN  std_logic;
        found_error : OUT std_logic);
END comparator;

ARCHITECTURE default OF comparator IS
BEGIN
  PROCESS (clk)
  BEGIN
    CASE en IS
      WHEN '1' =>
        IF (clk'event AND clk = '1') THEN
          check(input, output, found_error);
        END IF;
      WHEN OTHERS =>
        output      <= (OTHERS => 'Z');
        found_error <= 'Z';
    END CASE;
  END PROCESS;
END default;

```

encoding.vhd (package)

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

PACKAGE encoding IS

  -- tables storing the 4b5b encoding/decoding mappings
  TYPE array4b IS ARRAY (0 TO 15) OF std_logic_vector(3 DOWNTO 0);
  CONSTANT array0 : array4b := (
    ("0000"), ("0001"), ("0010"), ("0011"), ("0100"), ("0101"),
    ("0110"), ("0111"), ("1000"), ("1001"), ("1010"), ("1011"),
    ("1100"), ("1101"), ("1110"), ("1111")
  );
  TYPE array5b IS ARRAY (0 TO 15) OF std_logic_vector(4 DOWNTO 0);
  CONSTANT array1 : array5b := (
    ("11110"), ("01001"), ("10100"), ("10101"), ("01010"),
    ("01011"), ("01110"), ("01111"), ("10010"), ("10011"),
    ("10110"), ("10111"), ("11010"), ("11011"), ("11100"),
    ("11101")
  );

  -- encoding
  PROCEDURE encode(SIGNAL input  : IN  std_logic_vector(3 DOWNTO 0);
                  SIGNAL output : OUT std_logic_vector(4 DOWNTO 0));

  -- decoding
  PROCEDURE decode(SIGNAL input  : IN  std_logic_vector(4 DOWNTO 0);
                  SIGNAL output : OUT std_logic_vector(3 DOWNTO 0));

  -- error check
  PROCEDURE check(SIGNAL input      : IN  std_logic_vector(4 DOWNTO 0);
                  SIGNAL output     : OUT std_logic_vector(4 DOWNTO 0);
                  SIGNAL found_error : OUT std_logic);

END encoding;

PACKAGE BODY encoding IS

  PROCEDURE check(SIGNAL input : IN std_logic_vector(4 DOWNTO 0); SIGNAL output : OUT
    std_logic_vector(4 DOWNTO 0); SIGNAL found_error : OUT std_logic) IS
    VARIABLE found_match : boolean := false; --the variable shows whether a match
      --was found, initially we assume that
      -- none was found

    VARIABLE counter      : integer := 0;

```

```

BEGIN
  c0 : FOR i IN 0 TO 15 LOOP
    IF (input = array1(counter)) THEN      -- if there is a match
      output <= input;
      found_match := true;
      found_error <= '0';                  -- flag that there is no error
    EXIT;
  END IF;
  counter := counter + 1;
END LOOP;
IF (NOT found_match) THEN                  --if no match was found
  found_error <= '1';                      -- flag that there is an error
  output <= (OTHERS => 'Z');
END IF;
END check;

PROCEDURE encode(SIGNAL input : std_logic_vector(3 DOWNTO 0); SIGNAL output : OUT std_logic_vector
(4 DOWNTO 0)) IS
  VARIABLE counter : integer := 0;
BEGIN
  c0 : FOR i IN 0 TO 15 LOOP
    IF (input = array0(counter)) THEN
      output <= array1(counter);
    EXIT;
  END IF;
  counter := counter + 1;
END LOOP;
END encode;

PROCEDURE decode(SIGNAL input : std_logic_vector(4 DOWNTO 0); SIGNAL output : OUT std_logic_vector
(3 DOWNTO 0)) IS
  VARIABLE counter : integer := 0;
BEGIN
  c0 : FOR i IN 0 TO 15 LOOP
    IF (input = array1(counter)) THEN
      output <= array0(counter);
    EXIT;
  END IF;
  counter := counter + 1;
END LOOP;
END decode;
END encoding;

```

encoder.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE work.encoding.ALL;

ENTITY encoder IS
  PORT (input  : IN  std_logic_vector(43 DOWNTO 0);
        en, clk : IN  std_logic;
        output  : OUT std_logic_vector(54 DOWNTO 0) := (OTHERS => 'Z'));
END encoder;

ARCHITECTURE default OF encoder IS
BEGIN
  PROCESS (clk)
  BEGIN
    CASE en IS
      WHEN '1' =>
        IF (clk'event AND clk = '1') THEN
          encode(input(43 DOWNTO 40), output(54 DOWNTO 50));
          encode(input(39 DOWNTO 36), output(49 DOWNTO 45));
          encode(input(35 DOWNTO 32), output(44 DOWNTO 40));
          encode(input(31 DOWNTO 28), output(39 DOWNTO 35));
          encode(input(27 DOWNTO 24), output(34 DOWNTO 30));
          encode(input(23 DOWNTO 20), output(29 DOWNTO 25));
          encode(input(19 DOWNTO 16), output(24 DOWNTO 20));
          encode(input(15 DOWNTO 12), output(19 DOWNTO 15));

```

```

        encode(input(11 DOWNTO 8), output(14 DOWNTO 10));
        encode(input(7 DOWNTO 4), output(9 DOWNTO 5));
        encode(input(3 DOWNTO 0), output(4 DOWNTO 0));
    END IF;
    WHEN OTHERS =>
        output <= (OTHERS => 'Z');
    END CASE;
END PROCESS;

END default;

```

decoder.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE work.encoding.ALL;

ENTITY decoder IS
    PORT (input  : IN  std_logic_vector(54 DOWNTO 0);
          en, clk : IN  std_logic;
          output  : OUT std_logic_vector(43 DOWNTO 0) := (OTHERS => 'Z'));
END decoder;

ARCHITECTURE default OF decoder IS
BEGIN
    PROCESS (clk)
    BEGIN
        CASE en IS
            WHEN '1' =>
                IF (clk'event AND clk = '1') THEN
                    decode(input(54 DOWNTO 50), output(43 DOWNTO 40));
                    decode(input(49 DOWNTO 45), output(39 DOWNTO 36));
                    decode(input(44 DOWNTO 40), output(35 DOWNTO 32));
                    decode(input(39 DOWNTO 35), output(31 DOWNTO 28));
                    decode(input(34 DOWNTO 30), output(27 DOWNTO 24));
                    decode(input(29 DOWNTO 25), output(23 DOWNTO 20));
                    decode(input(24 DOWNTO 20), output(19 DOWNTO 16));
                    decode(input(19 DOWNTO 15), output(15 DOWNTO 12));
                    decode(input(14 DOWNTO 10), output(11 DOWNTO 8));
                    decode(input(9 DOWNTO 5), output(7 DOWNTO 4));
                    decode(input(4 DOWNTO 0), output(3 DOWNTO 0));
                END IF;
            WHEN OTHERS =>
                output <= (OTHERS => 'Z');
            END CASE;
        END PROCESS;
    END default;

```

demux.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY demux IS
    PORT (sel  : IN  std_logic_vector(1 DOWNTO 0);
          input : IN  std_logic_vector(43 DOWNTO 0);

          a, b, c : OUT std_logic_vector(43 DOWNTO 0));
END demux;

ARCHITECTURE default OF demux IS
BEGIN
    PROCESS(sel)
    BEGIN
        CASE sel IS
            WHEN "00" =>
                a <= input;

```

```

        b <= (OTHERS => 'Z');
        c <= (OTHERS => 'Z');
    WHEN "01" =>
        a <= (OTHERS => 'Z');
        b <= input;
        c <= (OTHERS => 'Z');
    WHEN "10" =>
        a <= (OTHERS => 'Z');
        b <= (OTHERS => 'Z');
        c <= input;
    WHEN OTHERS =>
        a <= (OTHERS => 'Z');
        b <= (OTHERS => 'Z');
        c <= (OTHERS => 'Z');

    END CASE;
    END PROCESS;

END default;

```

mux.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY mux IS
    PORT (sel      : IN  std_logic_vector(1 DOWNTO 0);
          a, b, c  : IN  std_logic_vector(44 DOWNTO 0);

          output   : OUT std_logic_vector(44 DOWNTO 0));
END mux;

ARCHITECTURE default OF mux IS
BEGIN

    WITH sel SELECT
        output <= a      WHEN "00",
        b WHEN "01",
        c WHEN "11", -- changed to "11" to match the multiplexer
        (OTHERS => 'Z') WHEN OTHERS;

END default;

```

error_detection.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY error_detect IS
    PORT (input      : IN  std_logic_vector(54 DOWNTO 0);
          output     : OUT std_logic_vector(54 DOWNTO 0);
          en, rst, clk : IN  std_logic;
          found_error : OUT std_logic);
END error_detect;

ARCHITECTURE default OF error_detect IS
    COMPONENT comparator IS
        PORT (input      : IN  std_logic_vector(4 DOWNTO 0);
              output     : OUT std_logic_vector(4 DOWNTO 0);
              en, clk    : IN  std_logic;
              found_error : OUT std_logic);
    END COMPONENT;

    COMPONENT dff IS
        GENERIC (d_len : integer := 5;
                 q_len : integer := 5);
        PORT (d      : IN  std_logic_vector(d_len-1 DOWNTO 0);

```

```

        q          : OUT std_logic_vector(q_len-1 DOWNTO 0);
        clk, rst, en : IN  std_logic);
END COMPONENT;

-- wires for flip flops' output
SIGNAL dff0_out : std_logic_vector(4 DOWNTO 0);
SIGNAL dff1_out : std_logic_vector(4 DOWNTO 0);
SIGNAL dff2_out : std_logic_vector(4 DOWNTO 0);
SIGNAL dff3_out : std_logic_vector(4 DOWNTO 0);
SIGNAL dff4_out : std_logic_vector(4 DOWNTO 0);
SIGNAL dff5_out : std_logic_vector(4 DOWNTO 0);
SIGNAL dff6_out : std_logic_vector(4 DOWNTO 0);
SIGNAL dff7_out : std_logic_vector(4 DOWNTO 0);
SIGNAL dff8_out : std_logic_vector(4 DOWNTO 0);
SIGNAL dff9_out : std_logic_vector(4 DOWNTO 0);
SIGNAL dff10_out : std_logic_vector(4 DOWNTO 0);

-- wires for "found_error" output of comparators
SIGNAL error0 : std_logic;
SIGNAL error1 : std_logic;
SIGNAL error2 : std_logic;
SIGNAL error3 : std_logic;
SIGNAL error4 : std_logic;
SIGNAL error5 : std_logic;
SIGNAL error6 : std_logic;
SIGNAL error7 : std_logic;
SIGNAL error8 : std_logic;
SIGNAL error9 : std_logic;
SIGNAL error10 : std_logic;

BEGIN

-- flip flops before comparators
dff0 : dff PORT MAP (d => input(54 DOWNTO 50), q => dff0_out, clk => clk, rst => rst, en => en);
dff1 : dff PORT MAP (d => input(49 DOWNTO 45), q => dff1_out, clk => clk, rst => rst, en => en);
dff2 : dff PORT MAP (d => input(44 DOWNTO 40), q => dff2_out, clk => clk, rst => rst, en => en);
dff3 : dff PORT MAP (d => input(39 DOWNTO 35), q => dff3_out, clk => clk, rst => rst, en => en);
dff4 : dff PORT MAP (d => input(34 DOWNTO 30), q => dff4_out, clk => clk, rst => rst, en => en);
dff5 : dff PORT MAP (d => input(29 DOWNTO 25), q => dff5_out, clk => clk, rst => rst, en => en);
dff6 : dff PORT MAP (d => input(24 DOWNTO 20), q => dff6_out, clk => clk, rst => rst, en => en);
dff7 : dff PORT MAP (d => input(19 DOWNTO 15), q => dff7_out, clk => clk, rst => rst, en => en);
dff8 : dff PORT MAP (d => input(14 DOWNTO 10), q => dff8_out, clk => clk, rst => rst, en => en);
dff9 : dff PORT MAP (d => input(9 DOWNTO 5), q => dff9_out, clk => clk, rst => rst, en => en);
dff10 : dff PORT MAP (d => input(4 DOWNTO 0), q => dff10_out, clk => clk, rst => rst, en => en);

-- comparators
comp0 : comparator PORT MAP (input => dff0_out, output => output(54 DOWNTO 50), en => en, clk =>
    clk, found_error => error0);
comp1 : comparator PORT MAP (input => dff1_out, output => output(49 DOWNTO 45), en => en, clk =>
    clk, found_error => error1);
comp2 : comparator PORT MAP (input => dff2_out, output => output(44 DOWNTO 40), en => en, clk =>
    clk, found_error => error2);
comp3 : comparator PORT MAP (input => dff3_out, output => output(39 DOWNTO 35), en => en, clk =>
    clk, found_error => error3);
comp4 : comparator PORT MAP (input => dff4_out, output => output(34 DOWNTO 30), en => en, clk =>
    clk, found_error => error4);
comp5 : comparator PORT MAP (input => dff5_out, output => output(29 DOWNTO 25), en => en, clk =>
    clk, found_error => error5);
comp6 : comparator PORT MAP (input => dff6_out, output => output(24 DOWNTO 20), en => en, clk =>
    clk, found_error => error6);
comp7 : comparator PORT MAP (input => dff7_out, output => output(19 DOWNTO 15), en => en, clk =>
    clk, found_error => error7);
comp8 : comparator PORT MAP (input => dff8_out, output => output(14 DOWNTO 10), en => en, clk =>
    clk, found_error => error8);
comp9 : comparator PORT MAP (input => dff9_out, output => output(9 DOWNTO 5), en => en, clk =>
    clk, found_error => error9);
comp10 : comparator PORT MAP (input => dff10_out, output => output(4 DOWNTO 0), en => en, clk =>
    clk, found_error => error10);

-- OR gate for "found_error" signals
found_error <= error0 OR error1 OR error2 OR error3 OR error4 OR error5 OR error6 OR error7 OR
    error8 OR error9 OR error10;

END default;
```

fid_gen.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY fid_gen IS
  PORT (
    clr, clk : IN std_logic;
    input     : IN std_logic_vector(44 DOWNTO 0);
    output    : OUT std_logic_vector(43 DOWNTO 0)
  );
END fid_gen;

ARCHITECTURE default OF fid_gen IS
  SIGNAL en : std_logic; --internal signal, because it is taken from the validity bit
BEGIN
  en <= input(0);        -- the last bit is the validity bit

  PROCESS (clk, clr)
    VARIABLE temp : integer RANGE 0 TO 32;
  BEGIN
    CASE en IS
      WHEN '1' =>
        CASE clr IS
          WHEN '0' =>
            IF (clk'event AND clk = '1') THEN
              temp      := temp + 1;
              output(38 DOWNTO 0) <= input(39 DOWNTO 1); -- the last bit
                                                           -- was used earlier
              output(43 DOWNTO 39) <= std_logic_vector(to_unsigned(temp, 5));
            END IF;
          WHEN OTHERS =>
            temp := 0;
          END CASE;
        WHEN OTHERS =>
          --output <= (OTHERS => 'Z');
          NULL;
        END CASE;

    END PROCESS;
END default;

```

fid_gen_tb.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY testbench IS
END testbench;

ARCHITECTURE tb OF testbench IS
  SIGNAL clr : std_logic;
  SIGNAL clk : std_logic := '0';
  SIGNAL input : std_logic_vector(44 DOWNTO 0);
  SIGNAL output : std_logic_vector(43 DOWNTO 0);
BEGIN
  fid_gen0 : ENTITY work.fid_gen PORT MAP (clr => clr, clk => clk, input => input, output => output)
    ;

  clk <= NOT clk AFTER 10 ns;

  PROCESS
  BEGIN
    --clear counter
    clr <= '1';
    WAIT FOR 25 ns;

    clr <= '0';
    input <= "101010010100101101011010110101101011010100101";
    WAIT FOR 20 ns;
  END PROCESS

```

```

    input <= "1010100101001011010110101101011010110100100";
    WAIT FOR 20 ns;

    input <= "1010100101001011010110101101011010110100101";
    WAIT FOR 20 ns;

    input <= "1010100101001011010110101101011010110100101";
    WAIT FOR 20 ns;
END PROCESS;

END tb;

```

scheduler.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY scheduler IS
    PORT (a, b, c : IN std_logic;
          s1, s0 : OUT std_logic);
END scheduler;

ARCHITECTURE default OF scheduler IS
    SIGNAL check_null : std_logic;
    SIGNAL input0, input1, input2 : std_logic;
BEGIN
    input0 <= '0' WHEN a = 'Z' ELSE a;
    input1 <= '0' WHEN b = 'Z' ELSE b;
    input2 <= '0' WHEN c = 'Z' ELSE c;

    check_null <= input0 OR input1 OR input2;
    PROCESS(check_null)
    BEGIN
        CASE check_null IS
            WHEN '0' =>
                s1 <= 'Z';
                s0 <= 'Z';
            WHEN OTHERS =>
                s1 <= input0;
                s0 <= input0 OR ((NOT input0) AND (NOT input1) AND input2);
        END CASE;
    END PROCESS;
END default;

```

scheduler_tb.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY testbench IS
END testbench;

ARCHITECTURE tb OF testbench IS
    SIGNAL a, b, c, s1, s0 : std_logic;
    SIGNAL counter : std_logic_vector(2 DOWNTO 0) := "000"; -- counter for loop
BEGIN

    scheduler0: entity work.scheduler port map (a=>a, b=>b, c=>c, s1=>s1, s0=>s0);

    PROCESS
    BEGIN
        FOR i IN 0 TO 7 LOOP
            a <= counter(2);
            b <= counter(1);
            c <= counter(0);
            counter <= std_logic_vector(unsigned(counter) + 1);
            WAIT FOR 10 ns;
        END LOOP
    END PROCESS

```

```

    END LOOP;
    END PROCESS;

END tb;

```

mil_main.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY mil_main IS
    PORT (
        clk : IN std_logic;
        rst : IN std_logic := '0'; --will change, as it will be controlled by the clock
        --synchronization module

        -- output layer
        chi_out, wupg_out, cs_out : IN std_logic_vector(44 DOWNTO 0);
        data_out : OUT std_logic_vector(54 DOWNTO 0);

        -- input layer
        data_in : IN std_logic_vector(54 DOWNTO 0);
        -- wup_in leads to the controller host interface
        -- chi_in is for the data
        chi_in, wup_in, cs_in : OUT std_logic_vector(43 DOWNTO 0);

        -- testing signals
        test_err : OUT std_logic
    );
END mil_main;

ARCHITECTURE default OF mil_main IS
    -- components
    COMPONENT scheduler IS
        PORT (a, b, c : IN std_logic;
              s1, s0 : OUT std_logic);
    END COMPONENT;

    COMPONENT mux IS
        PORT (sel : IN std_logic_vector(1 DOWNTO 0);
              a, b, c : IN std_logic_vector(44 DOWNTO 0);

              output : OUT std_logic_vector(44 DOWNTO 0));
    END COMPONENT;

    COMPONENT dff IS
        GENERIC (d_len : integer := 1;
                 q_len : integer := 1);
        PORT (d : IN std_logic_vector(d_len-1 DOWNTO 0);
              q : OUT std_logic_vector(q_len-1 DOWNTO 0);
              clk, rst, en : IN std_logic);
    END COMPONENT;

    COMPONENT encoder IS
        PORT (input : IN std_logic_vector(43 DOWNTO 0);
              en, clk : IN std_logic;
              output : OUT std_logic_vector(54 DOWNTO 0) := (OTHERS => 'Z'));
    END COMPONENT;

    COMPONENT error_detect IS
        PORT (input : IN std_logic_vector(54 DOWNTO 0);
              output : OUT std_logic_vector(54 DOWNTO 0);
              en, rst, clk : IN std_logic;
              found_error : OUT std_logic);
    END COMPONENT;

    COMPONENT decoder IS
        PORT (input : IN std_logic_vector(54 DOWNTO 0);
              en, clk : IN std_logic;
              output : OUT std_logic_vector(43 DOWNTO 0) := (OTHERS => 'Z'));
    END COMPONENT;

```

```

COMPONENT demux IS
  PORT (sel      : IN   std_logic_vector(1 DOWNTO 0);
        input    : IN   std_logic_vector(43 DOWNTO 0);
        a, b, c  : OUT  std_logic_vector(43 DOWNTO 0));
END COMPONENT;

COMPONENT fid_gen IS
  PORT (
    clr, clk : IN   std_logic;
    input    : IN   std_logic_vector(44 DOWNTO 0);
    output   : OUT  std_logic_vector(43 DOWNTO 0)
  );
END COMPONENT;

-- WIRES

-- output layer
SIGNAL mux_s1, mux_s0      : std_logic;
SIGNAL mux_out            : std_logic_vector(44 DOWNTO 0);
SIGNAL dff0_out           : std_logic_vector(44 DOWNTO 0);
SIGNAL encoder_out        : std_logic_vector(54 DOWNTO 0);
SIGNAL dff1_out           : std_logic_vector(43 DOWNTO 0);
SIGNAL fid_gen_out        : std_logic_vector(43 DOWNTO 0);
--input layer
SIGNAL dff3_out           : std_logic_vector(54 DOWNTO 0);
SIGNAL error_status       : std_logic;
SIGNAL error_detect_output : std_logic_vector(54 DOWNTO 0);
SIGNAL dff4_out           : std_logic_vector(54 DOWNTO 0);
SIGNAL OR_gate            : std_logic;
SIGNAL decoder_output     : std_logic_vector(43 DOWNTO 0);
SIGNAL dff5_out           : std_logic_vector(43 DOWNTO 0);
SIGNAL typeofservice      : std_logic_vector(1 DOWNTO 0); --sets the path
                                                              --of the incoming frame through the
                                                              demultiplexer

BEGIN

-- OUTPUT LAYER

-- scheduler and multiplexer
sched0 : scheduler PORT MAP (a => cs_out(0), b => chi_out(0), c => wupg_out(0), s1 => mux_s1, s0
=> mux_s0);
mux0   : mux PORT MAP (sel(1) => mux_s1, sel(0) => mux_s0, a => chi_out, b => wupg_out, c =>
cs_out, output => mux_out);

-- flip flop
dff0 : dff GENERIC MAP (d_len => 45, q_len => 45) PORT MAP (d => mux_out(44 DOWNTO 0), q =>
dff0_out, clk => clk, rst => rst, en => '1');

-- frame id generator
fid_gen0 : fid_gen PORT MAP (clr => '0', clk => clk, input => dff0_out, output => fid_gen_out);

-- flip flop
dff1 : dff GENERIC MAP (d_len => 44, q_len => 44) PORT MAP (d => fid_gen_out, q => dff1_out, clk
=> clk, rst => rst, en => '1');

-- encoder
encoder0 : encoder PORT MAP (input => dff1_out, en => '1', clk => clk, output => encoder_out);

-- flip flop
dff2 : dff GENERIC MAP (d_len => 55, q_len => 55) PORT MAP (d => encoder_out, q => data_out, clk
=> clk, rst => rst, en => '1');

-- INPUT LAYER

-- direction: input => output (right to left in the figure)
-- flip flop
dff3 : dff GENERIC MAP (d_len => 55, q_len => 55) PORT MAP (d => data_in, q => dff3_out, clk =>
clk, rst => rst, en => '1');

-- error detection
error_detect0 : error_detect PORT MAP (input => dff3_out, output => error_detect_output, en =>
'1', clk => clk, rst => rst, found_error => error_status);

-- flip flop
dff4 : dff GENERIC MAP (d_len => 55, q_len => 55) PORT MAP (d => error_detect_output, q =>
dff4_out, clk => clk, rst => OR_gate, en => '1');

```

```

-- or gate
OR_gate <= error_status OR rst;

-- decoder
decoder0 : decoder PORT MAP (input => dff4_out, output => decoder_output, clk => clk, en => '1');

-- flip flop
dff5 : dff GENERIC MAP (d_len => 44, q_len => 44) PORT MAP (d => decoder_output, q => dff5_out,
    clk => clk, rst => rst, en => '1');

-- demux's selection bits
typeofservice <= dff5_out(33 DOWNTO 32); -- 11o-12o bits

-- demux
demux0 : demux PORT MAP (sel => typeofservice, input => dff5_out, a => wup_in, b => chi_in, c =>
    cs_in);

test_err    <= error_status;

END default;

```

mil_main_tb.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY testbench IS
END testbench;

ARCHITECTURE tb OF testbench IS
    -- input layer
    SIGNAL chi_out, wupg_out, cs_out : std_logic_vector(44 DOWNTO 0);
    SIGNAL clk                       : std_logic := '0';
    SIGNAL rst                       : std_logic;
    SIGNAL data_out                  : std_logic_vector(54 DOWNTO 0);

    -- output layer
    SIGNAL data_in                   : std_logic_vector(54 DOWNTO 0);
    SIGNAL chi_in, wup_in, cs_in    : std_logic_vector(43 DOWNTO 0);

    -- testing
    SIGNAL test_err : std_logic;
BEGIN

    mil : ENTITY work.mil_main PORT MAP (chi_out => chi_out, wupg_out => wupg_out, cs_out => cs_out,
        clk => clk, rst => rst, data_out => data_out, data_in => data_in, chi_in => chi_in, wup_in =>
        wup_in, cs_in => cs_in, test_err => test_err);

    clk <= NOT clk AFTER 10 ns;
    rst <= '0';

    -- output layer testing
    chi_out <= (0 => '1', 1 => '1', OTHERS => '0');
    wupg_out <= (44 => '1', 0 => '1', OTHERS => '0');
    cs_out <= (2 => '1', OTHERS => '0');

    -- input layer testing
    --data_in <= ("10010100100111001010010110101101011010110101101010010");
    data_in <= ("1111110010100111001010010110101101011010110101101010010");

    --change the first 5-bit group to "11111" for an error

```

```
END tb;
```

A.4 Wakeup Pattern Generator

counter.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY counter IS
  PORT (
    en, clr, clk : IN std_logic;
    output       : OUT std_logic_vector(1 DOWNTO 0) := (OTHERS => 'Z')
  );
END counter;

ARCHITECTURE default OF counter IS
BEGIN

  PROCESS (clk, clr)
    VARIABLE temp : integer RANGE 0 TO 3;
  BEGIN
    CASE en IS
      WHEN '1' =>
        CASE clr IS
          WHEN '0' =>
            IF (clk'event AND clk = '1') THEN
              temp := temp + 1;
            END IF;
          WHEN OTHERS =>
            temp := 0;
          END CASE;
        WHEN OTHERS =>
            temp := 0;
        END CASE;
      output <= std_logic_vector(to_unsigned(temp, 2));
    END PROCESS;
  END default;
```

wupg_mux.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY wupg_mux IS
  PORT (sel : IN std_logic;
        a, b : IN std_logic;

        output : OUT std_logic);
END wupg_mux;

ARCHITECTURE default OF wupg_mux IS
BEGIN

  WITH sel SELECT
    output <= a WHEN '0',
             b WHEN '1',
             'Z' WHEN OTHERS;

END default;
```

rom.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY rom IS
  PORT (
    addr    : IN  std_logic_vector(1 DOWNTO 0);
    output   : OUT std_logic_vector(4 DOWNTO 0);
    oe, clk  : IN  std_logic
  );
END rom;

ARCHITECTURE default OF rom IS
  TYPE memory4 IS ARRAY (0 TO 3) OF std_logic_vector(4 DOWNTO 0);
  -- the first 3 bits represent the symbol and the last bit is the validity bit
  CONSTANT rom4 : memory4 := (
    "10101", "11111", "00101", "01111"
  );
BEGIN
  PROCESS(clk)
  BEGIN
    CASE oe IS
      WHEN '1' =>
        CASE addr IS
          WHEN (OTHERS => 'Z') =>
            output <= (OTHERS => 'Z');
          WHEN OTHERS =>
            IF (clk'event AND clk = '1') THEN
              output <= rom4(to_integer(unsigned(addr)));
            END IF;
          END CASE;
        WHEN OTHERS =>
          output <= (OTHERS => 'Z');
        END CASE;
      END PROCESS;
    END default;

```

srff.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

-- always enabled
ENTITY srff IS
  PORT (
    set, rst, clk : IN  std_logic;
    q              : OUT std_logic
  );
END srff;

ARCHITECTURE default OF srff IS
BEGIN
  PROCESS (rst, clk, set)
  BEGIN
    CASE rst IS
      WHEN '1' =>
        IF (set = '0') THEN
          q <= '0';
        ELSIF (set = '1') THEN
          q <= 'Z';
        END IF;
      WHEN '0' =>
        IF (clk'event AND clk = '1' AND set = '1') THEN
          q <= '1';
        END IF;
      WHEN OTHERS =>
        q <= 'Z';
      END CASE;
    END PROCESS;

```

```
END default;
```

mainv2.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY wupg_main IS
  PORT (
    wupg_en   : IN  std_logic;
    wupg_out  : OUT std_logic_vector(44 DOWNTO 0);
    clk       : IN  std_logic;

    sel       : IN  std_logic;
    reset     : IN  std_logic;
    test_output : OUT std_logic
  );
END wupg_main;

ARCHITECTURE default OF wupg_main IS

-- components
  COMPONENT srff IS
    PORT (
      set, rst, clk : IN  std_logic;
      q             : OUT std_logic
    );
  END COMPONENT;

  COMPONENT dff IS
    GENERIC (d_len : integer := 1;
             q_len : integer := 1);
    PORT (d       : IN  std_logic_vector(d_len-1 DOWNTO 0);
          q       : OUT std_logic_vector(q_len-1 DOWNTO 0);
          clk, rst, en : IN  std_logic);
  END COMPONENT;

  COMPONENT counter IS
    PORT (
      en, clr, clk : IN  std_logic;
      output       : OUT std_logic_vector(1 DOWNTO 0)
    );
  END COMPONENT;

  COMPONENT rom IS
    PORT (
      addr   : IN  std_logic_vector(1 DOWNTO 0);
      output : OUT std_logic_vector(4 DOWNTO 0);
      oe, clk : IN  std_logic
    );
  END COMPONENT;

  COMPONENT wupg_mux IS
    PORT (sel : IN  std_logic;
          a, b : IN  std_logic;

          output : OUT std_logic);
  END COMPONENT;

-- wires
  SIGNAL ctrl_wire : std_logic := '0';
  SIGNAL cnt_out   : std_logic_vector(1 DOWNTO 0);
  SIGNAL dff0_out  : std_logic_vector(1 DOWNTO 0);
  SIGNAL rom_out   : std_logic_vector(4 DOWNTO 0);
  SIGNAL dff3_out  : std_logic;
  SIGNAL rst_wire  : std_logic;
  SIGNAL dff2_out  : std_logic_vector(1 DOWNTO 0);
  SIGNAL AND_output : std_logic;
  SIGNAL dff4_out  : std_logic_vector(1 DOWNTO 0);
  SIGNAL dff5_out  : std_logic_vector(1 DOWNTO 0);
```

```

SIGNAL dff1_en      : std_logic;

BEGIN

    srff0 : srff PORT MAP (set => wupg_en, rst => rst_wire, clk => clk, q => ctrl_wire);

    counter0 : counter PORT MAP (en => ctrl_wire, clr => '0', clk => clk, output => cnt_out);

    dff0 : dff GENERIC MAP (d_len => 2, q_len => 2) PORT MAP (d => cnt_out, q => dff0_out, clk => clk,
        rst => '0', en => ctrl_wire);

    rom0 : rom PORT MAP (addr => dff0_out, output => rom_out, oe => ctrl_wire, clk => clk);

    -- DFF1
    -- here the symbol is sent inside a frame
    -- frame id is set to 0, because it is added in the medium interface layer
    dff1 : dff GENERIC MAP (d_len => 45, q_len => 45) PORT MAP (d(44 DOWNTO 40) => (OTHERS => '0'), d
        (39 DOWNTO 35) => "00100", d(34 DOWNTO 33) => "01", d(32 DOWNTO 5) => (OTHERS => '0'), d(4
        DOWNTO 0) => rom_out, q => wupg_out, clk => clk, rst => '0', en => dff1_en);

    -- enable signal for DFF1
    -- rom_out(0) = validity bit
    dff1_en <= dff3_out AND rom_out(0);

    -----

    dff3 : dff GENERIC MAP (d_len => 1, q_len => 1) PORT MAP (d(0) => ctrl_wire, q(0) => dff3_out, clk
        => clk, rst => '0', en => '1');

    dff2 : dff GENERIC MAP (d_len => 2, q_len => 2) PORT MAP (d => cnt_out, q => dff2_out, clk => clk,
        rst => reset, en => '1');

    dff4 : dff GENERIC MAP (d_len => 2, q_len => 2) PORT MAP (d => dff2_out, q => dff4_out, clk => clk
        , rst => reset, en => '1');

    dff5 : dff GENERIC MAP (d_len => 2, q_len => 2) PORT MAP (d => dff4_out, q => dff5_out, clk => clk
        , rst => reset, en => '1');

    mux0 : wupg_mux PORT MAP (sel => sel, a => '1', b => AND_output, output => rst_wire);

    -- AND gate
    AND_output <= dff5_out(1) AND dff5_out(0);

    test_output <= ctrl_wire;

END default;

```

wupg_main_tb.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY testbench IS
END testbench;

ARCHITECTURE tb OF testbench IS
    SIGNAL wupg_en      : std_logic;
    SIGNAL wupg_out    : std_logic_vector(44 DOWNTO 0);
    SIGNAL clk         : std_logic := '0';

    SIGNAL sel         : std_logic;
    SIGNAL reset       : std_logic;
    SIGNAL test_output : std_logic;

BEGIN

    wupg_main0 : ENTITY work.wupg_main PORT MAP (wupg_en => wupg_en, wupg_out => wupg_out, clk => clk,
        test_output => test_output, reset => reset, sel => sel);

    clk <= NOT clk AFTER 10 ns;

PROCESS

```

```

BEGIN
    -- reset SR FF
    sel <= '0'; -- multiplexer output: '1'
    wupg_en <= '0';
    reset <= '1'; -- reset 2, 4, 5 DFFs
    WAIT FOR 5 ns;
    reset <= '0'; --the reset is asynchronous
    sel <= '1'; -- multiplexer output: AND gate output
    WAIT FOR 10 ns;
    -- set SR FF
    wupg_en <= '1';
    WAIT FOR 20 ns;
    wupg_en <= '0';
    WAIT FOR 200 ns;

    END PROCESS;

END tb;

```

A.5 Wakeup Pattern Recognition

wupr_fsm.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY wupr_fsm IS
    PORT (symbol : IN std_logic_vector(3 DOWNTO 0);
          clk, rst : IN std_logic;

          wup_found : OUT std_logic;

          --testing signal
          curr_state : OUT std_logic_vector(2 DOWNTO 0));
END wupr_fsm;

-- the architecture follows design style #2 from Pedroni's book
ARCHITECTURE default OF wupr_fsm IS
    TYPE states IS (m0, m1, m2, m3, m4);
    SIGNAL prv_state, nxt_state : states;
    SIGNAL temp_wup_found : std_logic;
BEGIN

    PROCESS (clk, rst)
    BEGIN
        IF (rst = '1') THEN
            prv_state <= m0;
            curr_state <= std_logic_vector(to_unsigned(states'pos(m0), 3));
            wup_found <= '0';
        ELSIF (clk'event AND clk = '1') THEN
            wup_found <= temp_wup_found;
            prv_state <= nxt_state;
            curr_state <= std_logic_vector(to_unsigned(states'pos(nxt_state), 3));
        END IF;
    END PROCESS;

    PROCESS (prv_state, symbol)
    BEGIN
        CASE prv_state IS

            WHEN m0 =>
                CASE symbol IS
                    WHEN "1010" =>
                        temp_wup_found <= '0';
                        nxt_state <= m1;
                    WHEN OTHERS =>
                        temp_wup_found <= '0';
                END CASE;
            -- other states follow a similar pattern
        END CASE;
    END PROCESS;

```

```

    WHEN m1 =>
        CASE symbol IS
            WHEN "1111" =>
                nxt_state <= m2;
            WHEN OTHERS =>
                nxt_state <= m0;
        END CASE;

    WHEN m2 =>
        CASE symbol IS
            WHEN "0010" =>
                nxt_state <= m3;
            WHEN OTHERS =>
                nxt_state <= m0;
        END CASE;

    WHEN m3 =>
        CASE symbol IS
            WHEN "0111" =>
                nxt_state <= m4;
                temp_wup_found <= '1';
            WHEN OTHERS =>
                nxt_state <= m0;
        END CASE;

    WHEN m4 =>
        nxt_state <= m0;
        temp_wup_found <= '0';

    WHEN OTHERS =>
        NULL;

    END CASE;
END PROCESS;

```

```
END default;
```

wupr_fsm_tb.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY testbench IS
END testbench;

ARCHITECTURE tb OF testbench IS
    SIGNAL symbol      : std_logic_vector(3 DOWNTO 0);
    SIGNAL clk         : std_logic := '0';
    SIGNAL rst         : std_logic;
    SIGNAL wup_found   : std_logic;
    SIGNAL curr_state  : std_logic_vector(2 DOWNTO 0);
BEGIN

    wupr_fsm0 : ENTITY work.wupr_fsm PORT MAP (symbol => symbol, clk => clk, rst => rst, wup_found =>
        wup_found, curr_state => curr_state);

    clk <= NOT clk AFTER 10 ns;

    PROCESS
    BEGIN

        rst <= '1';
        symbol <= "0000";
        WAIT FOR 25 ns;

        rst <= '0';
        symbol <= "1010";
        WAIT FOR 20 ns;

        symbol <= "0101";
    END PROCESS;

```

```

    WAIT FOR 20 ns;

    symbol <= "1010";
    WAIT FOR 20 ns;

    symbol <= "1111";
    WAIT FOR 20 ns;

    symbol <= "0010";
    WAIT FOR 20 ns;

    symbol <= "0111";
    WAIT FOR 200 ns;

    END PROCESS;

END tb;

```

wupr_main.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY wupr_main IS
    PORT (symbol      : IN  std_logic_vector(3 DOWNTO 0);
          clk, fsm_rst : IN  std_logic;
          wup_found    : OUT std_logic);
END wupr_main;

ARCHITECTURE default OF wupr_main IS
    --components
    COMPONENT wupr_fsm IS
        PORT (symbol      : IN  std_logic_vector(3 DOWNTO 0);
              clk, rst    : IN  std_logic;

              wup_found    : OUT std_logic;

              --testing signal
              curr_state    : OUT std_logic_vector(2 DOWNTO 0));
    END COMPONENT;

    COMPONENT dff IS
        GENERIC (d_len : integer := 1;
                 q_len : integer := 1);
        PORT (d      : IN  std_logic_vector(d_len-1 DOWNTO 0);
              q      : OUT std_logic_vector(q_len-1 DOWNTO 0);
              clk, rst, en : IN  std_logic);
    END COMPONENT;

    --wires
    SIGNAL wire0      : std_logic_vector(3 DOWNTO 0);
    --testing signal
    SIGNAL curr_state : std_logic_vector(2 DOWNTO 0);
BEGIN
    reg0 : dff GENERIC MAP (d_len => 4, q_len => 4) PORT MAP (d => symbol, q => wire0, clk => clk, rst
        => '0', en => '1');

    wupr_fsm0 : wupr_fsm PORT MAP (symbol => wire0, clk => clk, rst => fsm_rst, wup_found => wup_found
        , curr_state => curr_state);

END default;

```

wupr_main_tb.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

```

```

ENTITY testbench IS
END testbench;

ARCHITECTURE tb OF testbench IS
    SIGNAL symbol      : std_logic_vector(3 DOWNTO 0);
    SIGNAL clk         : std_logic := '0';
    SIGNAL fsm_rst     : std_logic;
    SIGNAL wup_found   : std_logic;
BEGIN

    wupr_main0 : ENTITY work.wupr_main PORT MAP (symbol => symbol, clk => clk, fsm_rst => fsm_rst,
        wup_found => wup_found);

    clk <= NOT clk AFTER 10 ns;

    PROCESS
    BEGIN

        fsm_rst    <= '1';
        symbol <= "0000";
        WAIT FOR 25 ns;

        fsm_rst    <= '0';
        symbol <= "1010";
        WAIT FOR 20 ns;

        symbol <= "0101";
        WAIT FOR 20 ns;

        symbol <= "1010";
        WAIT FOR 20 ns;

        symbol <= "1111";
        WAIT FOR 20 ns;

        symbol <= "0010";
        WAIT FOR 20 ns;

        symbol <= "0111";
        WAIT FOR 200 ns;

    END PROCESS;

END tb;

```

A.6 Clock Synchronization

buffer.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY tri_state_buffer IS
    PORT (en      : IN  std_logic;
          input   : IN  std_logic;
          output  : OUT std_logic
        );
END tri_state_buffer;

ARCHITECTURE default OF tri_state_buffer IS
BEGIN

    output <= input WHEN en = '1' ELSE
        'Z';

END default;

```

micro_comparator.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY micro_comparator IS
  PORT (local_value  : IN  std_logic_vector(3 DOWNTO 0);  --microticks counted
        global_value : IN  std_logic_vector(3 DOWNTO 0);  -- required microticks from the arbiter
        en, clk       : IN  std_logic;
        sync_status   : OUT std_logic);
END micro_comparator;

ARCHITECTURE default OF micro_comparator IS
BEGIN
  PROCESS (clk)
  BEGIN
    CASE en IS
      WHEN '1' =>
        IF (clk'event AND clk = '1') THEN
          IF (global_value = local_value) THEN
            sync_status <= '1';
          ELSE
            sync_status <= '0';
          END IF;
        END IF;
      WHEN OTHERS =>
        sync_status <= 'Z';
    END CASE;
  END PROCESS;
END default;

```

micro_counter.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY micro_counter IS
  PORT (
    en, clr, clk : IN  std_logic;
    output       : OUT std_logic_vector(3 DOWNTO 0) := (OTHERS => 'Z')
  );
END micro_counter;

ARCHITECTURE default OF micro_counter IS
BEGIN

  PROCESS (clk, clr)
    VARIABLE temp : integer RANGE 0 TO 3;
  BEGIN
    CASE en IS
      WHEN '1' =>
        CASE clr IS
          WHEN '0' =>
            IF (clk'event AND clk = '1') THEN
              temp := temp + 1;
            END IF;
          WHEN OTHERS =>
            temp := 0;
          END CASE;
        WHEN OTHERS =>
            temp := 0;
        END CASE;
      output <= std_logic_vector(to_unsigned(temp, 4));
    END PROCESS;

END default;

```

srff.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

-- always enabled
ENTITY srff IS
  PORT (
    set, rst, clk : IN std_logic;
    q              : OUT std_logic
  );
END srff;

ARCHITECTURE default OF srff IS

BEGIN

  PROCESS (rst, clk, set)
  BEGIN
    CASE rst IS
      WHEN '1' =>
        IF (set = '0') THEN
          q <= '0';
        ELSIF (set = '1') THEN
          q <= 'Z';
        END IF;
      WHEN '0' =>
        IF (clk'event AND clk = '1' AND set = '1') THEN
          q <= '1';
        END IF;
      WHEN OTHERS =>
        q <= 'Z';
    END CASE;
  END PROCESS;

END default;

```

vbit_fsm.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY vbit_fsm IS
  PORT (
    buffer_inv : IN std_logic;
    validity_bit : OUT std_logic;

    clk, rst : IN std_logic;

    --testing signal
    curr_state : OUT std_logic_vector(1 DOWNTO 0)
  );
END vbit_fsm;

-- the architecture follows design style #1 from Pedroni's book
-- (asynchronous output)

ARCHITECTURE default OF vbit_fsm IS
  TYPE states IS (m0, m1, m2, m3, m4);
  SIGNAL prv_state, nxt_state : states;
BEGIN

  PROCESS (clk, rst)
  BEGIN
    IF (rst = '1') THEN
      prv_state <= m0;
      curr_state <= std_logic_vector(to_unsigned(states'pos(m0), 2));
    ELSIF (clk'event AND clk = '1') THEN
      prv_state <= nxt_state;
      curr_state <= std_logic_vector(to_unsigned(states'pos(nxt_state), 2));
    END IF;
  END PROCESS;

  PROCESS (prv_state, buffer_inv)

```

```

BEGIN
  CASE prv_state IS
    WHEN m0 =>
      validity_bit <= '0';
      CASE buffer_inv IS
        WHEN '0' =>
          nxt_state <= m1;
        WHEN OTHERS =>
          NULL;
      END CASE;
    WHEN m1 =>
      CASE buffer_inv IS
        WHEN '1' =>
          nxt_state <= m2;
        WHEN OTHERS =>
          NULL;
      END CASE;
    WHEN m2 =>
      -- wait state
      nxt_state <= m3;
    WHEN m3 =>
      validity_bit <= '1';
      nxt_state <= m4;
    WHEN m4 =>
      validity_bit <= '0';
      CASE buffer_inv IS
        WHEN '0' =>
          nxt_state <= m1;
        WHEN OTHERS =>
          NULL;
      END CASE;
    WHEN OTHERS =>
      NULL;
  END CASE;
END PROCESS;

END default;

```

vbit_fsm_tb.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY testbench IS
END testbench;

ARCHITECTURE tb OF testbench IS
  SIGNAL buffer_inv : std_logic := '0';
  SIGNAL validity_bit : std_logic;
  SIGNAL clk : std_logic := '0';
  SIGNAL rst : std_logic;
  SIGNAL curr_state : std_logic_vector(1 DOWNTO 0);
BEGIN

  vbit_fsm0 : ENTITY work.vbit_fsm PORT MAP (buffer_inv => buffer_inv, validity_bit => validity_bit,
    clk => clk, rst => rst, curr_state => curr_state);

  clk <= NOT clk AFTER 10 ns;

  PROCESS
  BEGIN
    rst <= '1';
    WAIT FOR 25 ns;

    rst <= '0';
    buffer_inv <= '1';
    WAIT FOR 40 ns;

    buffer_inv <= '0';
    WAIT FOR 25 ns;
  END PROCESS;

END tb;

```

microtick_counter.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY microtick_counter IS
  PORT (
    state          : IN  std_logic_vector(2 DOWNTO 0);
    clk, macro_clock : IN  std_logic;
    global_value    : IN  std_logic_vector(3 DOWNTO 0); --required macroticks
                                                         --from the arbiter

    reset_ff        : IN  std_logic;
    test_output      : OUT std_logic_vector(3 DOWNTO 0);

    -- output frame
    output : OUT std_logic_vector(44 DOWNTO 0);

    --reset signal for the vbit fsm
    rst : IN std_logic;

    -- test signal for the vbit fsm
    fsm_state : OUT std_logic_vector(1 DOWNTO 0)

  );
END microtick_counter;

ARCHITECTURE default OF microtick_counter IS
  --components
  COMPONENT srff IS
    PORT (
      set, rst, clk : IN  std_logic;
      q              : OUT std_logic
    );
  END COMPONENT;

  COMPONENT dff IS
    GENERIC (d_len : integer := 1;
             q_len : integer := 1);
    PORT (d      : IN  std_logic_vector(d_len-1 DOWNTO 0);
          q      : OUT std_logic_vector(q_len-1 DOWNTO 0);
          clk, rst, en : IN  std_logic);
  END COMPONENT;

  COMPONENT micro_comparator IS
    PORT (local_value : IN  std_logic_vector(3 DOWNTO 0); --microticks counted
          global_value : IN  std_logic_vector(3 DOWNTO 0); -- required microticks from the arbiter
          en, clk      : IN  std_logic;
          sync_status  : OUT std_logic);
  END COMPONENT;

  COMPONENT micro_counter IS
    PORT (
      en, clr, clk : IN  std_logic;
      output       : OUT std_logic_vector(3 DOWNTO 0) := (OTHERS => 'Z')
    );
  END COMPONENT;

  COMPONENT tri_state_buffer IS
    PORT (en   : IN  std_logic;
          input : IN  std_logic;
          output : OUT std_logic
    );
  END COMPONENT;

  COMPONENT vbit_fsm IS
    PORT (
      buffer_inv : IN  std_logic;
      validity_bit : OUT std_logic;

      clk, rst : IN std_logic;

      --testing signal
      curr_state : OUT std_logic_vector(1 DOWNTO 0)
    );
  END COMPONENT;

```

```

-- wires
SIGNAL and0_out   : std_logic;
SIGNAL and1_out   : std_logic;
SIGNAL buffer_en  : std_logic;
SIGNAL buffer_out : std_logic;
SIGNAL buffer_inv : std_logic;
SIGNAL cnt_out    : std_logic_vector(3 DOWNTO 0);
SIGNAL dff0_out   : std_logic_vector(3 DOWNTO 0);

SIGNAL local_value : std_logic_vector(3 DOWNTO 0); --microticks counted,
                                                    --received from
                                                    --DFF1

SIGNAL sync_status : std_logic;    -- taken from the comparator
SIGNAL validity_bit : std_logic;   -- taken from vbit_fsm

BEGIN

-- set signal for SR FF
and0_out <= state(2) AND (NOT state(1)) AND (NOT (state(0)));

-- reset signal for SR FF
and1_out <= state(2) AND state(1) AND state(0);

-- sr FF
srff0 : srff PORT MAP (set => and0_out, rst => and1_out, clk => clk, q => buffer_en);

-- buffer
buffer0 : tri_state_buffer PORT MAP (en => buffer_en, input => macro_clock, output => buffer_out);

-- NOT gate
buffer_inv <= NOT buffer_out;

-- counter
counter0 : micro_counter PORT MAP (en => buffer_out, clr => '0', clk => clk, output => cnt_out);

-- D FF 0
dff0 : dff GENERIC MAP (d_len => 4, q_len => 4) PORT MAP (d => cnt_out, q => dff0_out, clk => clk,
    rst => reset_ff, en => buffer_inv);

test_output <= dff0_out;

-- comparator
comparator0 : micro_comparator PORT MAP (local_value => dff0_out, global_value => global_value, en
    => buffer_inv, clk => clk, sync_status => sync_status);

-- D FF 1
dff1 : dff GENERIC MAP (d_len => 4, q_len => 4) PORT MAP (d => dff0_out, q => local_value, clk =>
    clk, rst => reset_ff, en => buffer_inv);

-- D FF 2
dff2 : dff GENERIC MAP (d_len => 45, q_len => 45) PORT MAP (d(44 DOWNTO 40) => (OTHERS => '0'), d
    (39 DOWNTO 35) => "00110", d(34 DOWNTO 33) => "10", d(32 DOWNTO 6) => (OTHERS => '0'), d(5
    DOWNTO 2) => local_value, d(1) => sync_status, d(0) => validity_bit, q => output, clk => clk,
    rst => reset_ff, en => '1');

-- vbit fsm, used to set the validity bit to 1
vbit_fsm0 : vbit_fsm PORT MAP (buffer_inv => buffer_inv, validity_bit => validity_bit, clk => clk,
    rst => rst, curr_state => fsm_state);

END default;

```

microtick_counter_tb.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY testbench IS
END testbench;

```

```

ARCHITECTURE tb OF testbench IS
  SIGNAL state      : std_logic_vector(2 DOWNTO 0);
  SIGNAL clk        : std_logic := '0';
  SIGNAL macro_clock : std_logic := '0';
  SIGNAL global_value : std_logic_vector(3 DOWNTO 0);
  SIGNAL reset_ff    : std_logic;
  SIGNAL test_output : std_logic_vector(3 DOWNTO 0);
  SIGNAL output      : std_logic_vector(44 DOWNTO 0);
  -- fsm's signals
  SIGNAL rst        : std_logic;
  SIGNAL fsm_state   : std_logic_vector(1 DOWNTO 0);
BEGIN

  microtick_counter0 : ENTITY work.microtick_counter PORT MAP (state => state, clk => clk,
    macro_clock => macro_clock, global_value => global_value, reset_ff => reset_ff, test_output =>
    test_output, output => output, rst => rst, fsm_state => fsm_state);

  clk      <= NOT clk      AFTER 10 ns;
  macro_clock <= NOT macro_clock AFTER 82 ns;
  global_value <= "0100";

  PROCESS
  BEGIN
    state <= "100";           --set sr ff
    rst   <= '1';             -- reset vbit fsm
    WAIT FOR 15 ns;
    rst   <= '0';
    state <= "000";
    reset_ff <= '1';           -- reset dffs
    WAIT FOR 20 ns;
    reset_ff <= '0';
    -- the first rising edge of macro_clock arrives at 82 ns
    WAIT FOR 200 ns;
  END PROCESS;

END tb;

```

A.7 Communication Controller (Integration of the Components)

main.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY controller IS
  PORT (
    --PORTS
    -- ports on the Host/Microcontroller side
    command_in : IN std_logic_vector(1 DOWNTO 0);

    clk      : IN std_logic;
    mil_clk  : IN std_logic;

    status_data : OUT std_logic_vector(49 DOWNTO 0);

    frame_id   : IN std_logic_vector(4 DOWNTO 0);
    data_length : IN std_logic_vector(4 DOWNTO 0);
    type_of_data : IN std_logic_vector(1 DOWNTO 0);
    data       : IN std_logic_vector(31 DOWNTO 0);
    validity_bit : IN std_logic;

    -- ports on the FlexRay Bus Driver side
    data_out : OUT std_logic_vector(54 DOWNTO 0);
    data_in  : IN std_logic_vector(54 DOWNTO 0);
    -----

```

```

-- auxiliary internal signals for the testbench

-- pmm
ext_fsm_rst : IN std_logic;
ext_reg_rst : IN std_logic;

-- cs
ext_macro_clock : IN std_logic;
ext_global_value : IN std_logic_vector(3 DOWNTO 0);
ext_reset_ff : IN std_logic;
ext_test_output : OUT std_logic_vector(3 DOWNTO 0);
ext_rst : IN std_logic;
ext_fsm_state : OUT std_logic_vector(1 DOWNTO 0);

-- mil
ext_test_err : OUT std_logic;
ext_mil_rst : IN std_logic;

-- wupr
ext_wupr_fsm_rst : IN std_logic;
ext_wup_found : OUT std_logic;

-- wupg
ext_sel : IN std_logic;
ext_reset : IN std_logic;
ext_wupg_test_output : OUT std_logic;
ext_wupg_output : OUT std_logic_vector(44 DOWNTO 0)

);
END controller;

ARCHITECTURE default OF controller IS
  COMPONENT chi IS
    PORT (
      -- input, direction: micro => communication
      command_in : IN std_logic_vector(1 DOWNTO 0);
      --corresponding output
      command_out : OUT std_logic_vector(1 DOWNTO 0);
      -- input, origin: micro
      clk : IN std_logic;
      -- inputs, direction: communication => micro
      config_ack : IN std_logic;
      reset_ack : IN std_logic;
      state : IN std_logic_vector(2 DOWNTO 0);
      sync_status : IN std_logic;
      frame_rcv : IN std_logic_vector(43 DOWNTO 0);
      -- corresponding output
      status_data : OUT std_logic_vector(49 DOWNTO 0);
      -- inputs, direction: micro => communication
      frame_id : IN std_logic_vector(4 DOWNTO 0);
      data_length : IN std_logic_vector(4 DOWNTO 0);
      type_of_data : IN std_logic_vector(1 DOWNTO 0);
      data : IN std_logic_vector(31 DOWNTO 0);
      validity_bit : IN std_logic;
      -- corresponding output
      frame_send : OUT std_logic_vector(44 DOWNTO 0));
  END COMPONENT;

  COMPONENT pmm IS
    PORT (command : IN std_logic_vector(1 DOWNTO 0);
          sync_status : IN std_logic;
          clk, fsm_rst, reg_rst : IN std_logic;

          curr_state : OUT std_logic_vector(2 DOWNTO 0);
          reset_ack : OUT std_logic;
          wup_en : OUT std_logic;
          config_ack : OUT std_logic);
  END COMPONENT;

  -- Clock Synchronization Module
  COMPONENT microtick_counter IS
    PORT (
      state : IN std_logic_vector(2 DOWNTO 0);

```

```

    clk, macro_clock : IN  std_logic;
    global_value     : IN  std_logic_vector(3 DOWNTO 0);  --required macroticks
                                                           --from the arbiter

    reset_ff         : IN  std_logic;
    test_output      : OUT std_logic_vector(3 DOWNTO 0);

    -- output frame
    output : OUT std_logic_vector(44 DOWNTO 0);

    --reset signal for the vbit fsm
    rst : IN std_logic;

    -- test signal for the vbit fsm
    fsm_state : OUT std_logic_vector(1 DOWNTO 0)

);
END COMPONENT;

COMPONENT mil_main IS
  PORT (
    clk : IN std_logic;
    rst : IN std_logic := '0';  --will change, as it will be controlled by the clock
    --synchronization module

    -- output layer
    chi_out, wupg_out, cs_out : IN  std_logic_vector(44 DOWNTO 0);
    data_out                 : OUT std_logic_vector(54 DOWNTO 0);

    -- input layer
    data_in                 : IN  std_logic_vector(54 DOWNTO 0);
    -- wup_in leads to the controller host interface
    -- chi_in is for the data
    chi_in, wup_in, cs_in : OUT std_logic_vector(43 DOWNTO 0);

    -- testing
    test_err : OUT std_logic
  );
END COMPONENT;

COMPONENT wupr_main IS
  PORT (symbol : IN  std_logic_vector(3 DOWNTO 0);
        clk, fsm_rst : IN  std_logic;
        wup_found   : OUT std_logic);
END COMPONENT;

COMPONENT wupg_main IS
  PORT (
    wupg_en : IN  std_logic;
    wupg_out : OUT std_logic_vector(44 DOWNTO 0);
    clk      : IN  std_logic;

    sel      : IN  std_logic;
    reset    : IN  std_logic;
    test_output : OUT std_logic
  );
END COMPONENT;

-- signals interconnecting the components
----- CHI -----
SIGNAL command_out : std_logic_vector(1 DOWNTO 0);
SIGNAL config_ack  : std_logic;
SIGNAL reset_ack   : std_logic;
SIGNAL state       : std_logic_vector(2 DOWNTO 0);
SIGNAL sync_status : std_logic;
SIGNAL frame_rcv   : std_logic_vector(43 DOWNTO 0);
SIGNAL frame_send  : std_logic_vector(44 DOWNTO 0);

----- PMM -----
SIGNAL fsm_rst : std_logic;  -- external
SIGNAL reg_rst : std_logic;  -- external

```

```
SIGNAL wup_en : std_logic;
```

```
----- CS -----
SIGNAL macro_clock : std_logic;           -- external
SIGNAL global_value : std_logic_vector(3 DOWNTO 0); -- external
SIGNAL reset_ff : std_logic;               -- external
SIGNAL test_output : std_logic_vector(3 DOWNTO 0); -- external
SIGNAL output : std_logic_vector(44 DOWNTO 0);
SIGNAL rst : std_logic;                   -- external
SIGNAL fsm_state : std_logic_vector(1 DOWNTO 0); -- external
```

```
----- MIL -----
SIGNAL mil_rst : std_logic;
SIGNAL wupg_out : std_logic_vector(44 DOWNTO 0);
SIGNAL wup_in : std_logic_vector(43 DOWNTO 0);
SIGNAL cs_in : std_logic_vector(43 DOWNTO 0); -- unused
SIGNAL test_err : std_logic;                 -- external
```

```
----- WUPR -----
SIGNAL wupr_fsm_rst : std_logic; -- external
SIGNAL wup_found : std_logic; -- external, intended for the CHI
```

```
----- WUPG -----
SIGNAL sel : std_logic; -- external
SIGNAL reset : std_logic; -- external
SIGNAL wupg_test_output : std_logic; -- external
```

```
SIGNAL check_validity : std_logic;
```

```
BEGIN
```

```
-- validity check of sync_status
PROCESS (output)
BEGIN
CASE output(0) IS -- the validity bit
WHEN '1' =>
sync_status <= output(1); -- then equal to the output of clock synchronization
WHEN OTHERS =>
sync_status <= 'Z';
END CASE;
END PROCESS;
```

```
chi0 : chi PORT MAP (command_in => command_in, command_out => command_out, clk => clk, config_ack
=> config_ack, reset_ack => reset_ack, sync_status => sync_status, state => state, frame_rcv =>
frame_rcv, status_data => status_data, frame_id => frame_id, data_length => data_length,
type_of_data => type_of_data, data => data, validity_bit => validity_bit, frame_send =>
frame_send);
```

```
pmm0 : pmm PORT MAP (command => command_out, sync_status => sync_status, clk => clk, fsm_rst =>
fsm_rst, reg_rst => reg_rst, curr_state => state, reset_ack => reset_ack, wup_en => wup_en,
config_ack => config_ack);
```

```
cs0 : microtick_counter PORT MAP (state => state, clk => clk, macro_clock => macro_clock,
global_value => global_value, reset_ff => reset_ff, test_output => test_output, output => output
, rst => rst, fsm_state => fsm_state);
```

```
mil0 : mil_main PORT MAP (clk => mil_clk, rst => mil_rst, chi_out => frame_send, wupg_out =>
wupg_out, cs_out => output, data_out => data_out, data_in => data_in, chi_in => frame_rcv,
wup_in => wup_in, cs_in => cs_in, test_err => test_err);
```

```
wupr0 : wupr_main PORT MAP (symbol => wup_in(4 DOWNTO 1), clk => clk, fsm_rst => wupr_fsm_rst,
wup_found => wup_found);
```

```
wupg0 : wupg_main PORT MAP (wupg_en => wup_en, wupg_out => wupg_out, clk => clk, sel => sel, reset
=> reset, test_output => wupg_test_output);
```

```
-- drive the auxiliary signals for the testbench
-- pmm
fsm_rst <= ext_fsm_rst;
```

```

reg_rst <= ext_reg_rst;

-- cs
macro_clock      <= ext_macro_clock;
global_value     <= ext_global_value;
reset_ff         <= ext_reset_ff;
ext_test_output  <= test_output;
rst              <= ext_rst;
ext_fsm_state    <= fsm_state;

-- mil
ext_test_err     <= test_err;
mil_rst          <= ext_mil_rst;

-- wupr
wupr_fsm_rst     <= ext_wupr_fsm_rst;
ext_wup_found    <= wup_found;

--wupg
sel              <= ext_sel;
reset            <= ext_reset;
ext_wupg_test_output <= wupg_test_output;
ext_wupg_output  <= wupg_out;

END default;

```

main_tb.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY testbench IS
END testbench;

ARCHITECTURE tb OF testbench IS

--PORTS

-- ports on the Host/Microcontroller side
SIGNAL command_in : std_logic_vector(1 DOWNTO 0);

SIGNAL clk        : std_logic := '0';
SIGNAL mil_clk    : std_logic := '0';    -- clock for the Medium Interface Layer

SIGNAL status_data : std_logic_vector(49 DOWNTO 0);

SIGNAL frame_id    : std_logic_vector(4 DOWNTO 0);
SIGNAL data_length : std_logic_vector(4 DOWNTO 0);
SIGNAL type_of_data : std_logic_vector(1 DOWNTO 0);
SIGNAL data        : std_logic_vector(31 DOWNTO 0);
SIGNAL validity_bit : std_logic;

-- ports on the FlexRay Bus Driver side
SIGNAL data_out : std_logic_vector(54 DOWNTO 0);
SIGNAL data_in  : std_logic_vector(54 DOWNTO 0);

-----

-- auxiliary internal signals for the testbench

-- pmm
SIGNAL fsm_rst : std_logic;    -- external
SIGNAL reg_rst : std_logic;    -- external

-- cs
SIGNAL macro_clock : std_logic := '0';    -- external
SIGNAL global_value : std_logic_vector(3 DOWNTO 0); --external
SIGNAL reset_ff     : std_logic;          -- external
SIGNAL test_output  : std_logic_vector(3 DOWNTO 0); --external
SIGNAL rst          : std_logic;          -- external
SIGNAL fsm_state    : std_logic_vector(1 DOWNTO 0); -- external

```

```

-- mil
SIGNAL test_err : std_logic;           -- external
SIGNAL mil_rst  : std_logic;           --external

-- wupr
SIGNAL wupr_fsm_rst : std_logic;       -- external
SIGNAL wup_found   : std_logic;       -- external, intended for the CHI

-- wupg
SIGNAL sel          : std_logic;       -- external
SIGNAL reset        : std_logic;       -- external
SIGNAL wupg_test_output : std_logic;   -- external
SIGNAL wupg_output  : std_logic_vector(44 DOWNTO 0); --external

BEGIN
  controller : ENTITY work.controller PORT MAP (command_in => command_in, clk => clk, status_data =>
    status_data, frame_id => frame_id, data_length => data_length, type_of_data => type_of_data,
    data => data, validity_bit => validity_bit, data_out => data_out, data_in => data_in,
    ext_fsm_rst => fsm_rst, ext_reg_rst => reg_rst, ext_macro_clock => macro_clock, ext_global_value
    => global_value, ext_reset_ff => reset_ff, ext_test_output => test_output, ext_rst => rst,
    ext_fsm_state => fsm_state, ext_test_err => test_err, ext_mil_rst => mil_rst, ext_wupr_fsm_rst
    => wupr_fsm_rst, ext_wup_found => wup_found, ext_sel => sel, ext_reset => reset,
    ext_wupg_test_output => wupg_test_output, ext_wupg_output => wupg_output, mil_clk => mil_clk);

  clk      <= NOT clk      AFTER 10 ns;
  mil_clk  <= clk;
  macro_clock <= NOT macro_clock AFTER 100 ns;
-- PROCESS (clk)
-- BEGIN
--   IF (clk'event) THEN
--     mil_clk <= NOT mil_clk AFTER 5 ns;
--   END IF;
-- END PROCESS;

  reg_rst      <= '0';           -- for the register in the pmm
  mil_rst      <= '0';           -- for the registers inside the mil
  validity_bit <= '0'; -- validity bit for a frame from the host

  global_value <= "0101"; -- number of microticks required for successful synchronisation

PROCESS
BEGIN
  --reset to set state to default_config
  fsm_rst <= '1';

  -- reset dffs inside cs (to get validity bit = 0/Z in mil)
  reset_ff <= '1';
  -- reset vbit fsm
  rst      <= '1';

  command_in <= "ZZ";

  WAIT FOR 25 ns;

  reset_ff <= '0';
  rst      <= '0';

  -- command: config
  fsm_rst <= '0';
  command_in <= "10";

  WAIT FOR 100 ns; -- once in the sleep state and config_ack=1 has been received

  -- command: wakeup
  command_in <= "00";
  WAIT FOR 30 ns;

  -- initialise the wakeup pattern generator
  sel <= '0';
  reset <= '1';
  WAIT FOR 5 ns;
  reset <= '0';
  sel <= '1';

```

```
WAIT FOR 260 ns; -- after the frames have been sent
-- (the wait is not necessary, it is only there
-- to make the results easier to visualise)

-- command: startup
command_in <= "01";
WAIT FOR 310 ns;

--since it is in the active state
-- command: halt
command_in <= "11";

WAIT FOR 400 ns;

END PROCESS;

END tb;
```

B. Notes on the Source Code

This appendix records some remarks on the code used in this thesis, which was presented in Appendix A.

B.1 The Finite State Machines

The finite state machines in the files [fsm.vhd](#) (Protocol Management Module) and [wupr_fsm.vhd](#) (Wakeup Pattern Recognition) follow design style #2 from Pedroni's book [7]. The FSM in the file [vbit_fsm.vhd](#) (Clock Synchronization), in contrast, follows design style #1.

Design style #1 stores only the previous state. As a result, when the output depends on the current input, a change in the input changes the output immediately. The output is therefore asynchronous (it does not depend on the clock). The overall circuit is summarised in the figure below.

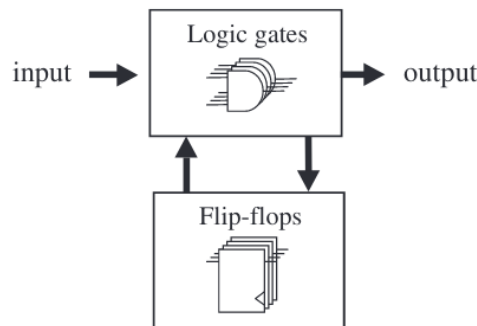


Figure B.1: Circuit diagram of design style #1

In design style #2, by contrast, the output of the circuit is stored as well, so the output is synchronous. The output can therefore change only on the rising edge of the clock.¹ In this case the circuit diagram is as follows.

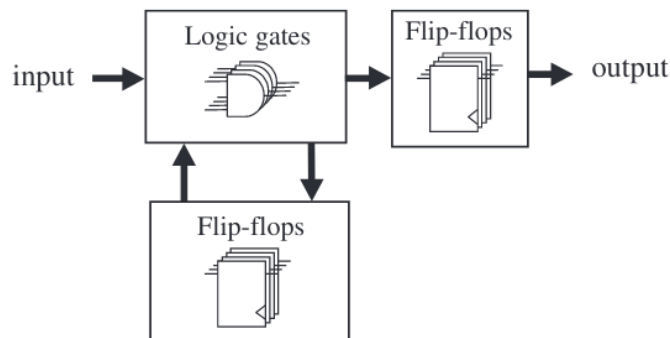


Figure B.2: Circuit diagram of design style #2

¹The type of edge naturally depends on the design, and it could, for example, be the falling edge. This code uses the rising edge, which is why that is the one mentioned.

B.2 Signal Mappings in the System

This section presents some of the signal mappings between different components, to make it easier to see how they are connected in the code. The underlined signals are those whose names were used in the integration code ([main.vhd](#)).

CHI	PMM
<u>command_out</u>	command
<u>config_ack</u>	config_ack
<u>reset_ack</u>	reset_ack
<u>state</u>	curr_state

Table B.1: CHI-PMM signal mapping

CHI	CS
<u>sync_status</u>	output(1)

Table B.2: CHI-CS signal mapping

PMM	CS
<u>sync_status</u>	output(1)
curr_state	<u>state</u>

Table B.3: PMM-CS signal mapping

CHI	MIL
<u>Frame_rcv</u>	chi_in
<u>Frame_send</u>	chi_out

Table B.4: CHI-MIL signal mapping

CS	MIL
output	cs_out
—	cs_in

Table B.5: CS-MIL signal mapping

WUPR	MIL
symbol	<u>wup_in(4 downto 1)</u>

Table B.6: WUPR-MIL signal mapping

WUPG	MIL
wupg_out	<u>wupg_out</u>

Table B.7: WUPG-MIL signal mapping

WUPG	PMM
wupg_en	<u>wup_en</u>

Table B.8: WUPG-PMM signal mapping

In Table B.5, the signal `cs_in` does not correspond to any CS signal. This is because reading data frames inside the CS was not part of the design. It is, however, a subject for future work, since synchronisation takes place inside Clock Synchronization and exchanging synchronisation data requires communication between the different nodes.

B.3 The Validity of the `sync_status` Signal

The `sync_status` signal indicates whether the node is synchronised with the network. It is one of the outputs of Clock Synchronization, and it is sent inside a data frame to inform the other nodes, the network arbiter, or both. The signal is also read continuously by the Controller Host Interface and Protocol Management Module submodules.

The signal must, however, only be read when it is valid. The data frame always contains the correct value of the signal, thanks to the finite state machine (Validity Bit FSM) inside Clock Synchronization. Having the CHI and PMM submodules read it directly, on the other hand, requires additional digital logic.

This digital logic does not appear in the architecture diagrams, because it was added when the components were integrated. It consists of the following code:

```
-- validity check of sync_status
PROCESS (output)
BEGIN
CASE output(0) IS -- the validity bit
WHEN '1' =>
    sync_status <= output(1); -- then equal to the output of clock synchronization
WHEN OTHERS =>
    sync_status <= 'Z';
END CASE;
END PROCESS;
```

In this way the correct value of the signal is always read, and when the signal is in high impedance (value 'Z'), it is ignored.

C. FPGA Technology

C.1 What Is an FPGA?

An FPGA (Field-Programmable Gate Array) is a programmable logic device, and more specifically a type of integrated circuit that can be used to implement any digital circuit. The name comes from the fact that the user works with a gate array that can be programmed in the field, that is, at the user's own workplace.

C.2 The Classification of FPGAs

Logic devices are divided into two categories, standard logic devices and custom integrated circuits (ICs), as shown in Figure C.1.

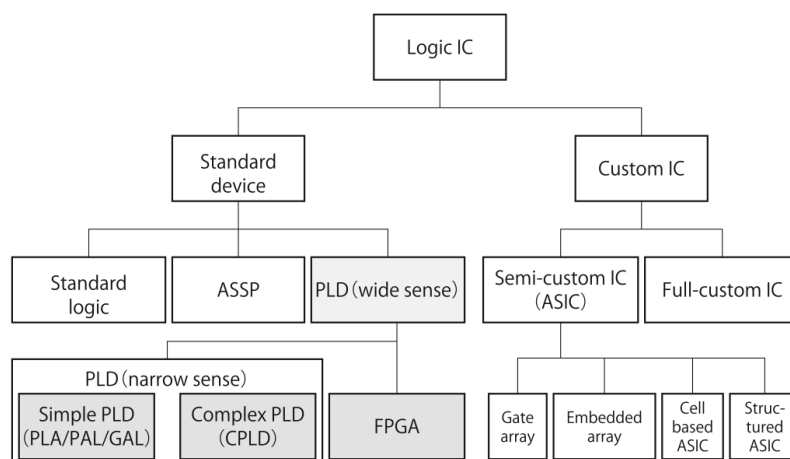


Figure C.1: The position of FPGAs among semiconductor devices

In most cases custom ICs are superior in performance (operating speed), density (number of gates) and flexibility. Nevertheless, the non-recurring engineering cost of custom ICs keeps rising, and the time from order to delivery keeps getting longer.

Custom ICs are classified as full-custom or semi-custom. The former are designed from scratch, while the latter use standard cells.¹

Unlike application-specific standard parts (ASSP), a programmable logic device (PLD) can implement various logic circuits according to the user's application. Programmable logic devices have become widespread because they exploit field programming and the freedom to be reconfigured.

An FPGA is a programmable logic device that combines logic blocks in a single device for a high degree of programmability. It can be mass-produced in a blank (initialised) state, which makes it an ideal standard device for semiconductor vendors. It can also be regarded as an easily made

¹A standard cell is any predesigned logic element, such as a logic gate, a multiplexer or a flip-flop.

Application-Specific Integrated Circuit (ASIC), since its non-recurring engineering cost is low and it requires no mask to be designed or manufactured.²

C.3 The Structure of an FPGA

The figure below shows the structure of an island-style FPGA.

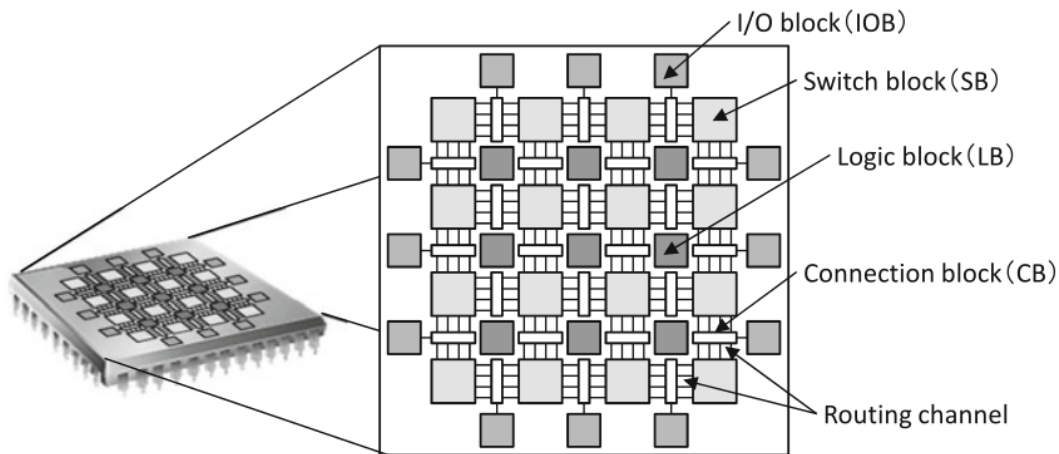


Figure C.2: Structure of a traditional FPGA

The core of an FPGA consists of three parts. The first contains the logic blocks (LBs), which implement logic circuits. The second is the input/output blocks (IOBs), which drive the input and output signals. The third is the routing elements, namely the switch blocks (SBs) and the connection blocks (CBs), which connect the logic blocks and the input/output blocks. There is also a clock network, a configuration chain and a test circuit. Commercial FPGAs include additional circuits for specialised functions, such as processors, memory blocks and multipliers.

The elements mentioned above are described in more detail below.

Logic Block: There are three main schemes for implementing programmable logic: the lookup table, the multiplexer and the product term (as defined in Boolean algebra). Each of them consists of a programmable part that can be used to implement any logic circuit.

Input/Output Block: A block that connects the input/output pins to the routing elements. It also includes some control circuits, such as pull-up, pull-down and input/output direction control, as well as a circuit for storing values, such as a flip-flop.

Routing Element: It consists of routing channels, connection blocks, and switch blocks at the points where logic blocks and input/output blocks meet. The switch blocks are programmable, so any route can be set up with the available routing resources.

Other Elements: The logic functions and interconnections of all the blocks are determined by the values stored in the configuration memory. The configuration chain is a path that writes the configuration data sequentially into all the configuration memory modules.^[6]

²A mask is the arrangement of the materials that make up the circuit (layers of metal, oxide and so on).

D. Block Coding

Block coding is a type of data coding that converts every block of m bits into a block of n bits. It adds redundancy that guarantees synchronisation, and it makes error detection possible.

Block coding usually involves three steps: *division*, *substitution* and *combination*. In the division step, a sequence of bits is divided into groups of m bits. The substitution step follows, in which each group of m bits is replaced by a group of n bits. Finally, the n -bit blocks are combined to form the final data stream. Figure D.1 illustrates the encoding process. Decoding naturally follows exactly the reverse process.

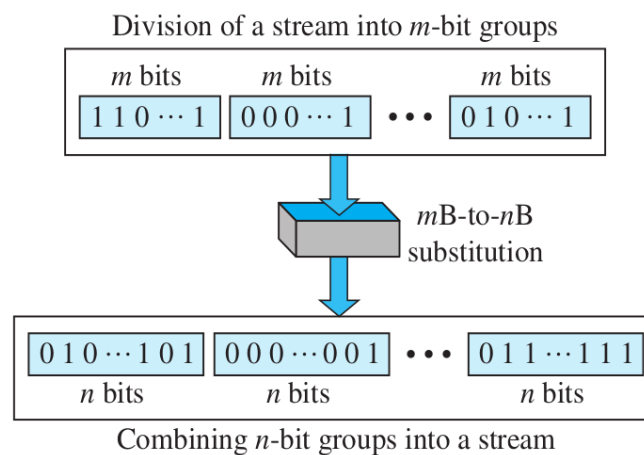


Figure D.1: Block coding

This thesis uses 4B/5B block coding, which is presented in detail below.

D.1 4B/5B Encoding

In this encoding $m = 4$ and $n = 5$, so every 4-bit block is mapped to a 5-bit block. The 5-bit groups have at most one 0 at the start and at most two 0s at the end. As a result, the new data sequence never contains more than three zeros in a row. This avoids the synchronisation problems that could otherwise arise from the NRZ-I encoding that follows.

The tables below show the mappings of the bit sequences in 4B/5B encoding.

Data Sequence	Encoded Sequence
0000	11110
0001	01001
0010	10100
0011	10101
0100	01010
0101	01011
0110	01110
0111	01111
1000	10010
1001	10011
1010	10110
1011	10111
1100	11010
1101	11011
1110	11100
1111	11101

Table D.1: 4B/5B encoding: data sequence mappings

Control Sequence	Encoded Sequence
Q (Quiet)	00000
I (Idle)	11111
H (Halt)	00100
J (Start delimiter)	11000
K (Start delimiter)	10001
T (End delimiter)	01101
S (Set)	11001
R (Reset)	00111

Table D.2: 4B/5B encoding: control sequence mappings

We see that a group of 4 bits has 16 possible combinations, while a group of 5 bits has 32. This means that 16 of the 5-bit groups are not used for encoding data. Eight of them are used for control purposes, as Table D.2 shows. The rest are used for error detection: if a 5-bit block turns out to belong to the unused part of the table, the receiver knows that an error occurred while the data were being transmitted.[8]

Note: This encoding is not the standard encoding of the FlexRay protocol. It is described here because it is the encoding method used in this design.

E. The OSI Model

E.1 Introduction

The Open System Interconnection (OSI) reference model is a framework that defines the conventions and processes that networked systems need in order to communicate with each other. The term ‘open systems’ refers to the fact that the specifications are available to everyone. The purpose of OSI is to help manufacturers and communications software engineers produce interoperable network systems. Figure E.1 shows the OSI model (excluding the physical transmission medium).

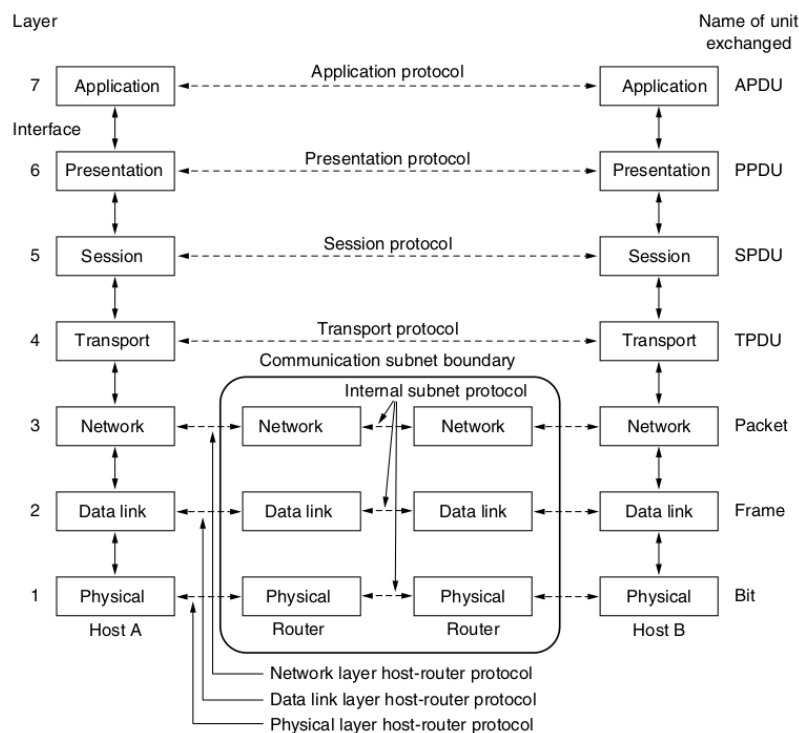


Figure E.1: The OSI reference model

The OSI model is based on the widely accepted technique of layering. In this approach, communication functions are partitioned into a vertical stack of layers. Each layer is associated with a set of functions that builds on, and makes use of, the services provided by the layer immediately below it. The layering approach was developed to achieve the following goals:

- To break a complex communication network down logically into smaller parts that are easier to understand and manage.
- To provide standard interfaces between network functions and modules.
- To provide a standard language for describing network functions, for use by network designers, managers, vendors and users. [9]

The principles on which the 7 layers of the OSI model were defined are the following:

1. A separate layer should be created wherever a different abstraction is needed.
2. Each layer should perform one well-defined function.
3. The function of each layer should be chosen so as to pave the way for internationally standardised protocols.
4. The boundaries of each layer should be chosen so that as little information as possible flows across the interfaces.
5. There should be enough layers that different functions do not have to be placed in the same layer. At the same time, there should be few enough of them that the architecture does not become hard to follow.

E.2 Description of the Layers

All the layers of the OSI model are described below, starting from the lowest.

E.2.1 Physical Layer

The **Physical Layer** handles the transmission of raw bits over a communication channel. Its design issues are about making sure the bits arrive correctly, so that when one side sends a 1 the other side receives a 1 and not a 0. Typical questions when implementing this layer are:

- Which electrical signals will represent 0 and 1.
- How many nanoseconds one bit will last.
- How communication is first established.
- How communication is ended once both sides have finished transmitting data.

These design issues cover the mechanical, electrical and timing interfaces as well as the physical transmission medium.

E.2.2 Data Link Layer

The main goal of the **Data Link Layer** is to turn a raw transmission into a stream of data that appears free of transmission errors. It does this by hiding the actual errors from the network layer, so that the network layer never sees them.

To do so, the sender breaks the input data up into *data frames*. The frames are then sent in sequence, and the receiver confirms that each one arrived successfully by sending an *acknowledgement frame* back to the sender.

E.2.3 Network Layer

The **Network Layer** controls the operation of the subnet. A key design issue is how packets are routed from source to destination. Routes can be based on static tables whose contents change rarely, or change often to avoid failed components in the network. Routes can also be decided when a session starts (for example, a login to a remote machine). Finally, the routes in the table can be changed continuously according to the current load on the network.

Another responsibility of the network layer is handling congestion, together with the higher layers, which determine the load on the network. In other words, the quality of service provided, in terms of delay, transmission time and so on, is a responsibility of the network layer.

The network layer must also allow heterogeneous networks to be interconnected, since addressing, packet size and so on may differ from one network to another.

E.2.4 Transport Layer

The basic job of the **Transport Layer** is to take data from the layer above (the Session Layer), break it into smaller segments and hand them to the network layer. It also has to confirm that the segments reach the other side correctly. All this must be done efficiently, and in a way that shields the upper layers from the changes in hardware that inevitably come with time.

The transport layer also decides which type of service to provide to the session layer, and ultimately to the users of the network. The type of service is decided when the connection is set up.

E.2.5 Session Layer

The **Session Layer** lets users on different machines set up sessions with one another. These sessions offer various services, such as *dialog control* (keeping track of whose turn it is to transmit), *token management* (preventing both parties from attempting the same critical operation at the same time), and *synchronisation* (adding checkpoints to long transmissions so that they can continue after a failure).

E.2.6 Presentation Layer

The **Presentation Layer** deals with the syntax and semantics of the data being transmitted. For two computers with different internal data representations to communicate, the data they exchange have to be represented in an abstract way. This can be achieved with a standard way of encoding the data. The presentation layer is responsible for managing these abstract data structures and for defining the higher-level data structures to be exchanged.

E.2.7 Application Layer

The **Application Layer** contains the protocols that users need. One such application protocol is HTTP (HyperText Transfer Protocol), which is the basis of the World Wide Web. Other application protocols in use cover file transfer, electronic mail and network news. [10]

Bibliography

- [1] Michael Barr. Programming embedded systems in C and C++. ” O’Reilly Media, Inc.”, 1999.
- [2] Steve Heath. Embedded systems design. Elsevier, 2002.
- [3] Shanker Shreejith and Suhaib A Fahmy. Extensible flexray communication controller for fpga-based automotive systems. IEEE transactions on vehicular technology, 64(2):453–465, 2014.
- [4] Peter Marwedel. Embedded system design: embedded systems foundations of cyber-physical systems, and the internet of things. Springer Nature, 2021.
- [5] Dominique Paret. FlexRay and its applications: real time multiplexed network. Wiley Online Library, 2012.
- [6] Hideharu Amano. Principles and structures of FPGAs. Springer, 2018.
- [7] Volnei A Pedroni. Circuit design with VHDL. MIT press, 2020.
- [8] B. A. Forouzan. Data communications and networking, fifth edition. McGraw-Hill, 2013.
- [9] Sumit Kumar, Sumit Dalal, and Vivek Dixit. The osi model: Overview on the seven layers of computer networks. International Journal of Computer Science and Information Technology Research, 2(3):461–466, 2014.
- [10] TANENBAUM ANDREW S and WETHERALL DAVID J. Computer networks fifth edition. pearson, 2011.

